

Contents

Every line is a link. The page numbers are read back from the rendered book, never typed by hand.

How to use this manual	2
MODULE ONE · INFORMATION MANAGEMENT AND DATABASE SYSTEMS	5
Unit 1 · Foundations of Data Management	6
Unit 2 · Database Systems and Architecture	9
MODULE TWO · CONCEPTUAL MODELS	19
Unit 1 · The Entity-Relationship Model	20
Unit 2 · The Enhanced E-R Model	36
Unit 3 · Semi-Structured Models: XML and JSON	46
Unit 4 · Worked Conceptual Models	48
MODULE THREE · LOGICAL DESIGN	51
Unit 1 · The Relational Model	52
Unit 2 · From Conceptual to Logical Design	56
MODULE FOUR · FUNCTIONAL DEPENDENCIES AND NORMALIZATION	63
Unit 1 · Normalization	64
MODULE FIVE · RELATIONAL THEORY AND QUERY LANGUAGES	75
Unit 1 · Relational Algebra	76
Unit 2 · Structured Query Language (SQL)	80
WORKED PAPER 2025/2026	90
Questions One to Three	91
Questions Four to Six	97
MOCK EXAMINATION ONE	105
Question Paper	106
Answers	107
MOCK EXAMINATION TWO	110
Question Paper	111
Answers	112
MOCK EXAMINATION THREE	115
Question Paper	116
Answers	117

How to use this manual

Read this page once. It is short, and it decides whether the rest of the manual works for you.

TEACH

THE BARGAIN

This manual assumes you have never designed a database, never drawn an entity-relationship diagram, and never written a line of SQL. It carries you from there to a full mark in the examination. It is also meant to be **enough on its own**: you should never need a lecturer, a textbook or a website to follow it. Anything it uses, it teaches first, including its own notation.

In exchange it asks one thing: **draw the diagrams and write the queries yourself**. Reading a finished ER diagram or a solved normalization creates a feeling of understanding that evaporates in the examination hall, where you face a blank page and a business description. Every diagram in this book was drawn by hand first, and every query was checked against the schema it runs on. Do the same, and the difference between your answer and the one printed here is information.

The boxes

Every page is built from a small set of boxes. Each has one job, and the colour tells you which before you read a word.

TEACH

New ideas, explained from nothing. Read these properly.

● MUST MEMORISE

Emerald green means **learn this by heart**. Definitions the examiner asks for word for word, the normal-form rules, the crow's-foot symbols, the six relational algebra operations. It is the only thing this colour is ever used for. If a page has green on it, there is something on that page you are expected to know cold.

WORKED

SOLVED RIGHT WHERE IT IS TAUGHT

A real question, solved in numbered steps, then a **now you try** at the end. Every worked box carries a small chip saying where the question comes from, so you know it is a question that was actually asked, not one invented to be easy:

PQ 25/26 Q6 a past paper (session and question); **TEST 1** the class test; **DECK** the practice slides; **TEXT M4U1** the course text's own exercise. Do the "now you try" before moving on.

SQL / RELATIONAL ALGEBRA

Indigo means a listing you would write on the answer sheet: an SQL statement or a relational algebra expression. It is always in a fixed-width font so the structure is exact. You are expected to be able to reproduce these, not just read them.

⚠ TRAP

A mistake that is easy to make and costly in marks. Most are taken from the exact places the examiner sets a snare: a partial dependency mistaken for transitive, a NULL that breaks a count, a many-to-many relationship with an attribute stranded on the line.

RECALL

Questions with the answers printed underneath, below a dashed line. **Cover the answers.** Say your answer out loud, or write it, and only then look. Checking whether you knew something is not the same as knowing it, and the difference is the whole point of this box.

EXAM QUESTION

A question reproduced as it appeared on the paper, plain and uncoloured, so you meet it the way the examiner set it. Its worked answer follows in a teal **Answer** box.

Reading a figure

● MUST MEMORISE

THE NOTATION IS THE EXAM

Half of this course is drawn, not written. An entity is a rectangle, a relationship a line with a verb on it, and the small marks at the ends of that line, the **crow's foot**, a circle, a single bar, carry the whole meaning of a business rule. Get a symbol wrong and the answer is wrong, however good the prose around it. Every symbol used in this manual is defined the first time it appears, and the full legend sits in Module Two. Where a figure is a diagram you must be able to draw, redraw it yourself until it comes out without looking.

One promise, and one label

TEACH

The promise. Every number, every normalized set of relations, and every query result in this manual was worked out independently, not copied from a source. The course's own materials contain arithmetic and design slips (you will meet one, a table whose printed candidate keys do not match its data), so nothing here is trusted just because a source printed it. Where a source is wrong, this manual says so and shows the correct working.

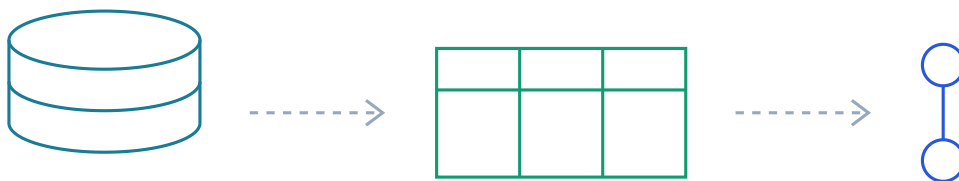
The label. A few questions in the past papers come from an older, wider syllabus (this course was once taught under a different code with extra "high performance" topics).

Where a question or a piece of content is outside the current syllabus, it carries a **NOT IN SYLLABUS** label. It is still taught and still solved in full, so that if it reappears you lose no marks, but the label tells you it is not core, so you spend your revision time accordingly.

MODULE ONE

Information Management and Database Systems

Where data comes from, why a pile of files is not a data-base, and the one idea, keeping the data and the programs that use it apart, that the rest of this book is built on.



Foundations of Data Management

Before a single table is drawn, three questions: what is data, what do we actually do with it, and what makes a system that holds it good rather than merely large.

By the end of this unit you can: tell data from information; describe how data is captured, represented, stored, and retrieved; explain the difference between privacy, integrity, and security; and judge a system by its scalability, efficiency, and effectiveness.

1.1 Data, and information

A single value, 08:00, or Glasgow, or 350, is **data**: a raw fact with no context. On its own it tells you nothing. The moment you put it in context, "the 08:00 inspection of property PG4 in Glasgow was priced at 350", it becomes **information**: data that has been organised so a person can act on it.

Everything in this course is machinery for turning the first thing into the second reliably, for many users at once, without the facts contradicting each other. That is what "data management" means.

● MUST MEMORISE

DATA VS INFORMATION

Data is raw, unprocessed facts. **Information** is data processed and placed in a context that gives it meaning. Data is the input; information is the output you can make a decision with.

1.2 The information lifecycle: capture, represent, store, retrieve

Data enters a system, is put into a form a computer can hold, is kept safely, and is fetched again on demand. Four stages, in order:

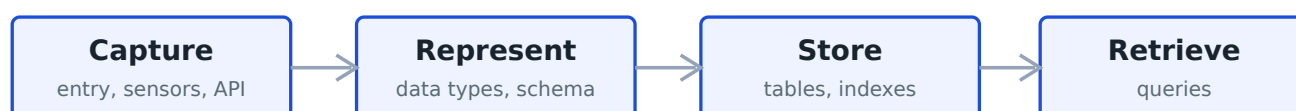


Figure 1.1 The information lifecycle. Data is captured, given a representation (a type and a place in a schema), stored, and later retrieved.

Capture is how data is collected from the real world: manual entry on a form, sensors and IoT devices, scanners and OCR, or an API pulling from another system. **Representation** is the form it is stored in: every value is given a **data type** that fixes what it may hold and how much space it takes.

TYPE	HOLDS	EXAMPLE VALUE
INT (INTEGER)	whole numbers	350
VARCHAR(n)	text up to n characters	'Glasgow'
CHAR(n)	fixed-length text	'M231 JGR'
DATE	a calendar date	'2004-10-18'
DECIMAL(p,s)	exact numbers with decimals	80000.00

You will meet these again the moment you write your first `CREATE TABLE` in Module Five. Text values go in single quotes; numbers do not.

Storing data well means being able to find it again quickly, which is the job of an **index**, and **retrieval** is fetching it with a query. Indexing earns its own question on the paper, so it earns a worked answer here.

Q. What is the purpose of indexing in a database, and how does it improve performance?

- 1 Say what an index is. An index is a separate lookup structure that maps the values of a column to the locations of the rows that hold them, in the way the index at the back of a book maps a topic to its page numbers.
- 2 Say what it replaces. Without an index, finding every row where `city = 'Glasgow'` means reading every row in the table, a **full table scan**. On a million-row table that is a million reads.
- 3 Say how it helps. With an index on `city`, the system jumps straight to the matching entries (using a structure such as a B-tree or a hash index) and reads only those rows. Search, sorting, and joining all get faster.
- 4 State the cost, because a complete answer names the trade-off. An index takes extra storage and must be updated on every insert, update, and delete, so indexing every column would slow writing down. You index the columns you search on.

Now you try: in one sentence each, give one column in a `STAFF(staffNo, fName, lName, branchNo)` table you *would* index and one you would not, and why.

1.3 Three things people confuse: privacy, integrity, security

These three words are used loosely in ordinary speech and precisely in this course. The examiner asks you to separate them, so separate them.

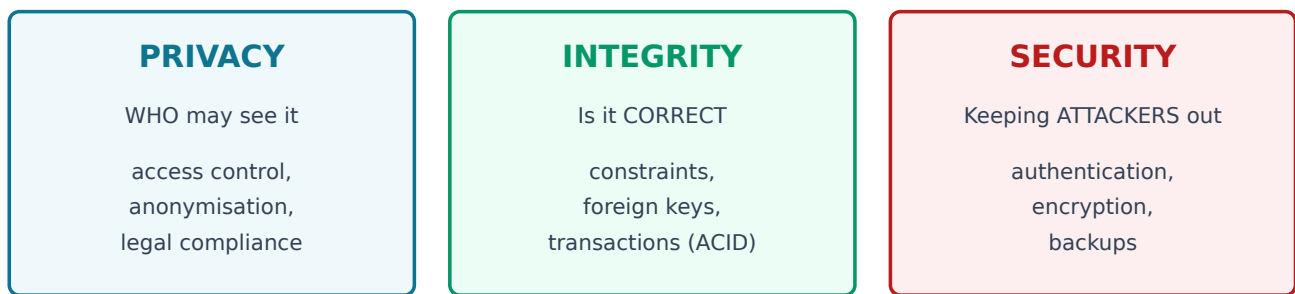


Figure 1.2 Privacy is about who is allowed to see data; integrity is about whether the data is correct and consistent; security is about keeping intruders out. They overlap in practice but answer three different questions.

WORKED

TEXT M1U1 SELF-ASSESSMENT 2

Q. Explain the differences between data privacy, data integrity, and data security.

Data privacy concerns *who is permitted to access* which data. It is enforced with access control (role-based permissions), anonymisation of sensitive fields, and compliance with data-protection law. Example: an ordinary student may see their own results but not a lecturer's salary.

Data integrity concerns whether the data is *accurate and consistent*. It is enforced with constraints (NOT NULL , UNIQUE , CHECK), referential integrity (foreign keys that must match a real row), and transaction management so a half-finished update never leaves the data in a broken state. Example: a booking cannot reference a guest number that does not exist.

Data security concerns protection against *unauthorised access and attack*. It is enforced with authentication, encryption, firewalls, database hardening, and backups for disaster recovery. Example: passwords are stored encrypted, not in plain text.

One-line hook to remember them: privacy = *who* may see it, integrity = is it *correct*, security = keeping *attackers* out.

MUST MEMORISE

ACID

Integrity leans on the transaction, and a good transaction has four properties, **ACID**: **Atomicity** (all of it happens or none of it does), **Consistency** (it moves the database from one valid state to another), **Isolation** (concurrent transactions do not tread on each other), **Durability** (once committed, it survives a crash). Learn the word and what each letter means.

1.4 Judging a system: scalability, efficiency, effectiveness

Three attributes decide whether a data system is any good. They sound similar and are not.

- **Scalability** is the ability to keep working as data and users grow. You scale **vertically** by giving one machine more power (more CPU, memory, disk), or **horizontally** by spreading the load across many machines (replication, sharding).

- ▶ **Efficiency** is how little resource the work costs: CPU, memory, storage, and query time. Good indexing and storage design buy efficiency.
- ▶ **Effectiveness** is whether the system actually meets the users' needs: accurate results, reliable uptime, and user satisfaction.

The distinction between the last two is a favourite short question, so here it is, answered.

AS PRINTED

19/20 Q6A

6 (a) Differentiate between Information Efficiency and Information Effectiveness [3 Marks]
DTS224 19/20, Q6a, as printed.

WORKED

PQ 19/20 Q6A

Q. Differentiate between Information Efficiency and Information Effectiveness.

NOT IN SYLLABUS (phrased in the older wording; the ideas are the efficiency/effectiveness pair above, so you can still answer it fully.)

Information efficiency is about *doing the thing right*: producing the required information with the least resource and time, cost per query, storage used, response time. A report that is generated in one second using little memory is efficient.

Information effectiveness is about *doing the right thing*: producing information that actually serves the user's purpose, that is accurate, relevant, timely and complete enough to decide on. A report delivered instantly but showing the wrong figures is efficient yet not effective.

Hook: efficiency = doing it right (cheaply, quickly); effectiveness = doing the right thing (usefully, correctly).

RECALL

COVER THE ANSWERS

1. Give one sentence separating data from information.
2. Name the four stages of the information lifecycle in order.
3. Which of privacy, integrity, security does a foreign key serve?
4. What is the difference between vertical and horizontal scaling?

ANSWERS

1. Data is raw facts; information is data given context so it can be acted on. 2. Capture, represent, store, retrieve. 3. Integrity (it keeps references correct). 4. Vertical = one bigger machine; horizontal = more machines sharing the load.

MODULE ONE · UNIT 2

Database Systems and Architecture

Why organisations abandoned loose files for databases, what a DBMS is and what sits inside it, and the layered architecture that lets the data and the programs change independently.

By the end of this unit you can: list the limitations of the file-based approach and the advantages of the database approach; define a database, a DBMS, a schema, and an instance; separate

DDL from DML; explain the three levels of data abstraction and the two kinds of data independence; and tell a personal database from an enterprise one.

2.1 The file-based approach, and why it failed

Before databases, each part of an organisation kept its own files and wrote its own programs to read them. The sales department had a sales file; accounts had an accounts file; each with its own copy of the customer list. This is the **file-based approach**, and it fails in ways the examiner returns to every year.

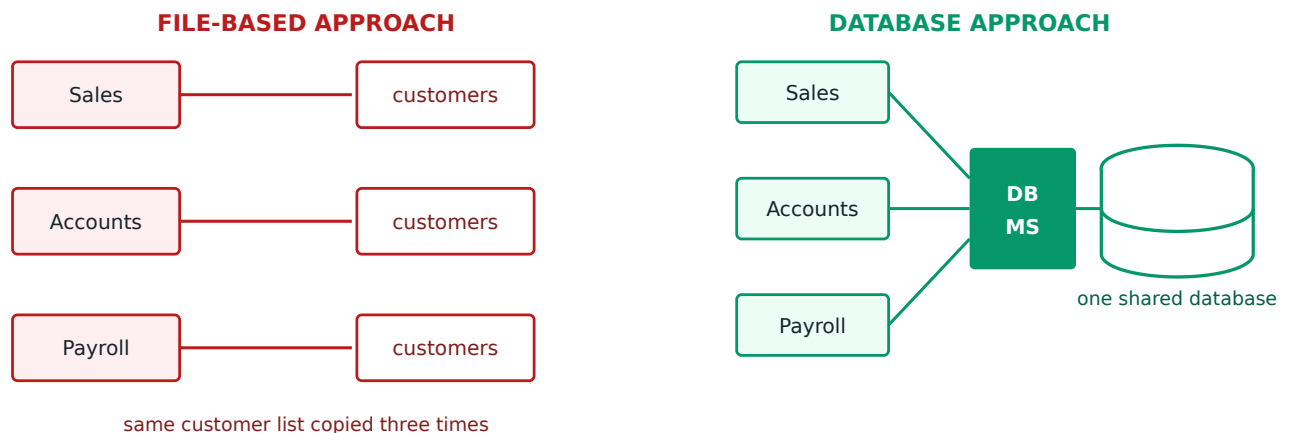


Figure 1.3 Left: every application owns its own files, so the customer list is duplicated and can disagree. Right: every application shares one database through the DBMS, so there is a single, consistent copy.

● MUST MEMORISE

LIMITATIONS OF THE FILE-BASED APPROACH

- ▶ **Data redundancy.** The same fact (a customer's address) is stored in several files.
- ▶ **Data inconsistency.** Update it in one file and not another, and the copies now disagree.
- ▶ **Data isolation.** Data is scattered across files in different formats, so writing a program to combine it is hard.
- ▶ **Program-data dependence.** The file's structure is written into every program, so changing the structure means rewriting the programs.
- ▶ **Difficult concurrent access.** Letting many users write at once safely is left to each program to solve, and mostly it is not solved.
- ▶ **Weak security and integrity.** Rules live in program code, applied unevenly, easy to bypass.

AS PRINTED

25/26 Q2A, 20/21 Q1A, 15/16 Q1C

- | a. Explain two (2) limitations of the file-based approach to data processing. (2 mks)
DTS224 25/26, Q2a, as printed.
- | (a) List 3 disadvantages of the File-Based System [3 marks]
DTS224 20/21, Q1a, as printed.
- | 1(c) Explain five limitation of file-Based Approach (5marks)
DTS224 15/16, Q1c, as printed.

Q. Explain two limitations of the file-based approach to data processing. (25/26 asks for two; 20/21 asks for three; 15/16 asks for five. One answer, scaled to the marks.)

Model answer (two, stated then explained)

1. Data redundancy and inconsistency. Because each program keeps its own files, the same data is duplicated across them. When one copy is updated and another is not, the files disagree, and there is no single correct version.

2. Program-data dependence. The physical structure of a file (its fields and their order) is embedded in the code of every program that reads it. A change as small as widening a field forces a change to every one of those programs.

For three, add **data isolation** (data scattered in incompatible formats is hard to combine). For five, add **difficulty of concurrent access** and **weak security and integrity control**. Naming the limitation and then explaining it in a sentence is what earns the mark; a bare list does not.

Now you try: the database approach fixes each of these. For redundancy and for program-data dependence, name the database feature that removes it. (Answers appear in 2.4 and 2.5 below.)

2.2 Database, DBMS, schema, instance

Four definitions the examiner asks for by name. Learn them in these words.

● MUST MEMORISE

THE FOUR CORE DEFINITIONS

Database: a shared collection of logically related data (and a description of that data), designed to meet the information needs of an organisation.

DBMS (Database Management System): the software that lets users define, create, maintain, and control access to the database. Examples: MySQL, Oracle, PostgreSQL, Microsoft SQL Server.

Schema: the description of the data using a given data model, that is, the structure. It changes rarely. Example: Student{matricNo, name, address, programme, age}.

Instance (database instance): the actual data in the database at a particular moment. It changes constantly, on every insert, update, and delete.

Schema is to instance as the empty form is to the filled-in form: the form's layout is the schema, today's stack of completed forms is the instance.

AS PRINTED

24/25 Q4C, 19/20 Q2C

(c) With two (2) examples, describe a database management system (3½ marks)
DTS224 24/25, Q4c, as printed.

2 (c) Define the following terms; (i) Data Model (ii) Database Instance (iii) DBMS
[3 Marks]

DTS224 19/20, Q2c, as printed.

Q. With two examples, describe a Database Management System. (24/25 Q4c)
And define: data model, database instance, DBMS. (19/20 Q2c)

A **DBMS** is the software layer between the users and the stored data. It lets them define the structure (through a data-definition language), put data in and take it out (through a data-manipulation language), and it controls access, enforces integrity, and manages concurrency and recovery, so the applications do not each have to. Two examples: **MySQL** and **Oracle Database** (also acceptable: PostgreSQL, Microsoft SQL Server, IBM Db2).

A **data model** is a collection of concepts used to describe the structure of a database, that is, the data, the relationships between it, and the constraints on it. The relational model (tables) is the one this course centres on.

A **database instance** is the information held in the database at a particular point in time, a snapshot of its current contents.

2.3 Two languages inside the DBMS: DDL and DML

You talk to a database in two registers. One defines the structure; the other moves the data around.

● MUST MEMORISE

DDL VS DML

	DDL (DATA DEFINITION LANGUAGE)	DML (DATA MANIPULATION LANGUAGE)
Defines	the structure: entities, attributes, relationships, constraints	the data: the rows themselves
Acts on	the schema	the instance
Statements	CREATE, ALTER, DROP	INSERT, UPDATE, DELETE, SELECT
Compiles into	tables stored in the data dictionary (system catalogue: metadata, "data about data")	changes to the stored rows

The retrieval-only part of DML (the `SELECT`) is called a **query language**. The four DML operations are retrieval, insertion, deletion, and modification.

(d) Differentiate between data definition language (DDL) and data manipulation language (DML)

(3 marks)

Question Five

DTS224 24/25, Q4d, as printed.

a. Given the following database concept in (I-III), answer question (i-ii):

I. The data definition language.

II. The data manipulation language.

III. The data model.

(i) Which of the following enumerations (I) to (III) plays an important role in representing information about the real world in a database? (½ mk)

(ii) Explain each concept briefly; your explanation should reflect the answer for your choice. (6 mks)

DTS224 25/26, Q5a, as printed.

WORKED

PQ 24/25 Q4D

PQ 25/26 Q5A

Q. Differentiate between data definition language (DDL) and data manipulation language (DML).

DDL is the part of a database language used to *define and change the structure* of the database: creating tables, altering columns, dropping objects, and stating constraints. Its statements are `CREATE`, `ALTER`, and `DROP`, and their effect is recorded in the data dictionary.

DML is the part used to *work with the data itself*: inserting new rows, updating and deleting existing ones, and retrieving them. Its statements are `INSERT`, `UPDATE`, `DELETE`, and `SELECT`.

In short: DDL changes the container, DML changes the contents.

2.4 Inside the DBMS: its components and its people

A DBMS is not one program but several parts working together, and it is run by people with defined roles. Two roles are regularly confused on the paper.

TEACH

COMPONENTS A DBMS PROVIDES

Asked to "list components of a DBMS", name from this set: the **DDL compiler**, the **DML compiler / query language processor**, the **query processor** (breaks a request into steps), the **database manager** (moves data between disk and memory), the **file manager**, the **data dictionary / system catalogue**, the **DML pre-compiler**, the **authorisation and integrity manager**, the **transaction manager**, and the **recovery / backup manager**. The **database manager** is the module that interfaces between the low-level stored data and the application programs and queries.

4 (b) Explain Database Manager and its Responsibilities

[3.5 Marks]

DTS224 19/20, Q4b, as printed.

(a) Extensively differentiate between the following terms:

- i. Database Manager and Database Administrator (2 marks)
- ii. SQL and NOSQL Databases (2 marks)
- iii. Composite and Multi-valued Attributes (2 marks)

DTS224 21/22, Q5a, as printed.

Q. Explain the Database Manager and its responsibilities, and differentiate it from the Database Administrator.

The **Database Manager** is a *software module* inside the DBMS. It is the interface between the low-level data stored on disk and the application programs and queries that ask for it. Its responsibilities are moving data between disk and main memory, and enforcing, at run time, the integrity and access controls the database is set up with. It is a program.

The **Database Administrator (DBA)** is a *person* with central control over the data and the programs that access it. Their responsibilities are: defining the schema, granting authorisation for data access, specifying integrity constraints, defining storage structures and access methods, and modifying the schema or physical organisation as needs change.

The one-line separation: the Database Manager is a **program** inside the DBMS; the Database Administrator is a **person** who runs the database.

2.5 The layered architecture: three levels of data abstraction

This is the central idea of the whole unit, and the answer to the "program-data dependence" problem from 2.1. A database hides its physical storage behind layers, so that a user sees only what they need and a change at one layer does not ripple up to the next. The classic picture is the **three-schema architecture**, also called the three levels of data abstraction.

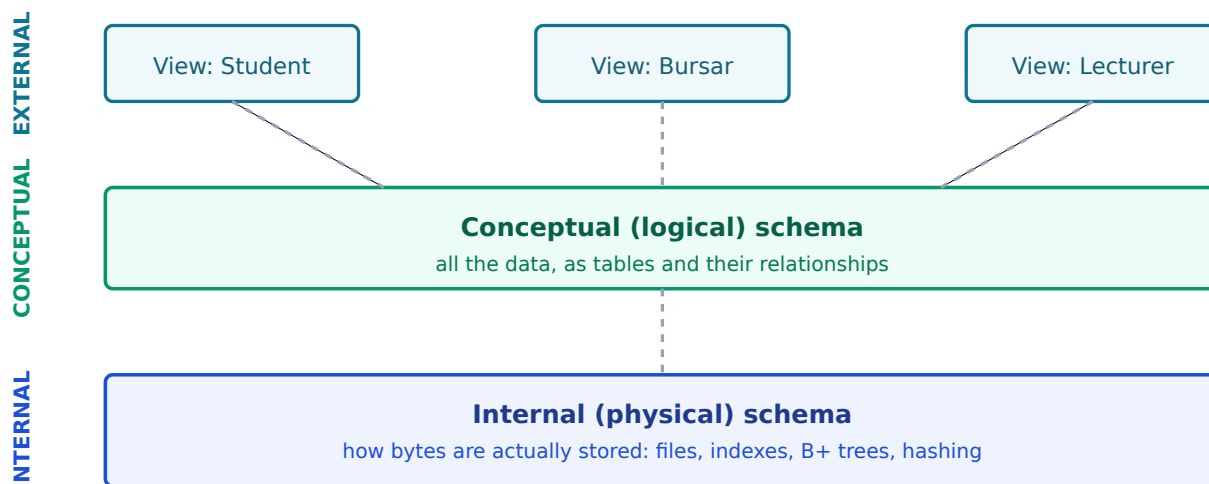


Figure 1.4 The three-schema architecture. Many external views sit on one conceptual schema, which sits on one internal schema. The dashed lines are mappings the DBMS maintains, and they are what make the levels independent.

● MUST MEMORISE

THE THREE LEVELS

External level (view level): the highest. Each user or program sees only the slice of data relevant to it, as rows and columns, tailored to its access rights. A student's view can compute a CGPA or an age without ever seeing a lecturer's salary. Many different external views can exist over the same data.

Conceptual level (logical level): the middle. It describes *all* the data in the database as tables and the relationships between them, without any concern for how it is stored. This is the level the database designer works at.

Internal level (physical level): the lowest. It describes *how* the data is actually stored: the file organisation, the indexes, whether hashing or a B+ tree is used. (Some texts name a separate physical level below the internal one; the exam wants the three: external, conceptual, internal.)

AS PRINTED

25/26 Q2C, 19/20 Q1B

- c. What is meant by data abstraction? Explain the three (3) levels of data abstraction with examples. (5 ½ mks)

DTS224 25/26, Q2c, as printed.

- 1(b) Explain (3) differences between user views, a conceptual schema, and an internal schema as different perspectives of the same database [6Marks]

DTS224 19/20, Q1b, as printed.

Q. What is meant by data abstraction? Explain the three levels of data abstraction with examples. (25/26 Q2c) Equivalently: explain the differences between user views, a conceptual schema, and an internal schema as different perspectives of the same database. (19/20 Q1b)

Data abstraction is the hiding of the complex details of how data is stored, showing each user only what they need. It is what lets a user work with a table without knowing anything about disks or files.

- 1 External level (user views).** What one user or program sees: a customised subset of the data. Example: the "Student" view shows a student their courses and results but not other students' records, and not any salary data.
- 2 Conceptual level (conceptual schema).** The community view: every table and relationship in the database, described independently of storage. Example: the whole university schema, STUDENT, COURSE, LECTURER, DEPARTMENT, and how they relate.
- 3 Internal level (internal schema).** The storage view: how those tables are physically held, which files, which indexes, which access method. Example: STUDENT is stored as an indexed file with a B+ tree index on matricNo.

They are three perspectives on one database: the same STUDENT data, seen as a tailored view, as a logical table, and as bytes in an indexed file.

2.6 Why the layers matter: data independence

The layers are not decoration. They deliver **data independence**: the ability to change the schema at one level without having to change the level above it. This is the precise cure for the file-based approach's program-data dependence.

● MUST MEMORISE

TWO KINDS OF DATA INDEPENDENCE

Physical data independence: the ability to change the internal (physical) schema, adding an index, changing the file organisation, without changing the conceptual schema or rewriting any application program. Comparatively easy to achieve.

Logical data independence: the ability to change the conceptual schema, adding a column or a table, without changing the external views or rewriting programs that do not use the change. Harder to achieve, because programs depend closely on the logical structure.

Example of logical independence: users A and B both read EmployeeNumber and EmployeeName. B adds a Salary column to the schema. A's view and A's programs are untouched.

2.7 Two sizes of database: personal and enterprise

TEACH

A **personal database system** supports a single application with a few tables, on one computer, for one user: simple. An **enterprise-level database system** supports many users at once and multiple applications, across many computers, with many tables and often many databases: complex. The difference the examiner wants is number of users, number of applications, and complexity.

AS PRINTED

25/26 Q3A

- a. (i) Would you choose a database system instead of simply storing data in operating system files? Justify your answer with two points. (5 marks)
(ii) When would it make sense not to use a database system? (2.5 marks)

DTS224 25/26, Q3a, as printed.

WORKED

PQ 25/26 Q3A

Q(i). Would you choose a database system instead of simply storing data in operating-system files? Justify with two points. Q(ii). When would it make sense not to use a database system?

(i) Yes, choose a database, because:

1. Controlled, consistent, shared access. A DBMS gives one shared copy of the data with concurrency control, so many users can read and write at once without corrupting it or duplicating it, which loose OS files cannot do safely.

2. Integrity, security, and independence built in. Constraints and access control are enforced centrally, and the layered architecture gives data independence, so the structure can change without rewriting every program. With OS files, all of this is left to hand-written code.

(ii) When not to use one

A database system carries cost and overhead (licensing, hardware, expertise, and slower simple access). For a small, single-user, single-application task with little data, few or no concurrency needs, and a fixed structure, plain files are cheaper and simpler, and a full DBMS is not worth its overhead. Real-time systems with very tight timing, or purely temporary data, are other cases where files can be preferable.

- 1. Two limitations of the file-based approach.**
 - 2. Which language creates a table, DDL or DML? Which inserts a row?**
 - 3. Name the three levels of data abstraction, highest first.**
 - 4. Which is harder to achieve, physical or logical data independence?**
 - 5. Database Manager or DBA: which one is a person?**
-

ANSWERS

1. Any two of: redundancy, inconsistency, isolation, program-data dependence, poor concurrency, weak security/integrity. 2. DDL creates the table (CREATE); DML inserts the row (INSERT). 3. External, conceptual, internal. 4. Logical. 5. The DBA is the person; the Database Manager is a software module.

You now have the vocabulary: data and information, DBMS, schema and instance, DDL and DML, the three levels, data independence. Module Two turns to the conceptual level and how you design it: the entity-relationship diagram, where a business description in English becomes a picture, and then, in Module Three, a set of tables. That picture is the single most drawn thing on the examination paper, so that is where we go next.

MODULE TWO

Conceptual Models

Turning a paragraph of business English into a diagram: entities, the lines between them, and the small marks at the ends of those lines that carry the whole meaning. This module is the most drawn thing on the paper.



The Entity-Relationship Model

A business hands you a description in words. Your job is to draw it: the things it cares about, how they connect, and the rules governing those connections. That drawing is the entity-relationship diagram, and drawing it correctly is half of this course's marks.

By the end of this unit you can: extract business rules from a description; draw entities, attributes, and relationships in a consistent notation; read and set cardinality with the crow's-foot symbols; and turn any of the recurring exam scenarios into a correct ER diagram.

1.1 From words to a model: business rules

A **conceptual model** is a high-level picture of the data an organisation cares about and how it relates, drawn without any concern for how a computer will store it. The **entity-relationship (E-R) model** is the most widely used one, and an **ER diagram (ERD)** is its picture. It is independent of any particular DBMS.

Everything in the diagram comes from **business rules**: statements, drawn from the organisation's policies and procedures, about the structure and behaviour of its data. A business rule is also called an **integrity constraint** once it is in the database. A rule such as "each project is contracted by exactly one client" is what fixes a cardinality on your diagram.

● MUST MEMORISE

CHARACTERISTICS OF A GOOD BUSINESS RULE

A good business rule is:

- ▶ **Declarative:** it states *what* should be true, not how to enforce it.
- ▶ **Precise:** it has one clear, unambiguous meaning.
- ▶ **Atomic:** it states one single thing, not several.
- ▶ **Consistent:** it does not contradict another rule.
- ▶ **Expressible:** it can be written in plain language.
- ▶ **Distinct:** it does not duplicate another rule.
- ▶ **Business-oriented:** it is stated in terms the business understands.

AS PRINTED

23/24 Q2B, 21/22 Q6B

(b) Business rule is important in data modelling, as it represent an organisation operation interaction among the stakeholders and the data flow. As a database administrator you have been asked by the management of Alex & Suite legal firm to come up with a standard business rule that will capture their organisation operation interaction and data flow. Explain Four important qualities you will need to consider in making your delivery. (4marks)

DTS224 23/24, Q2b, as printed.

(b) Mention four (4) characteristics of a good business rule (2 marks)

DTS224 21/22, Q6b, as printed.

Q. State four qualities you would consider in making a good business rule (21/22 asks for four characteristics of a good business rule).

Any four of the seven above, each named and given a phrase: a good business rule is **declarative** (states what is true, not how to enforce it), **precise** (one unambiguous meaning), **atomic** (states a single fact), and **business-oriented** (expressed in the organisation's own terms). Naming and explaining four earns full marks; a bare list of words does not.

1.2 The notation, all of it, on one page

There is no single world standard for ER notation. This manual uses the common crow's-foot style that the recent papers use. Learn these symbols cold: an answer with the right idea and the wrong symbol is still marked wrong.

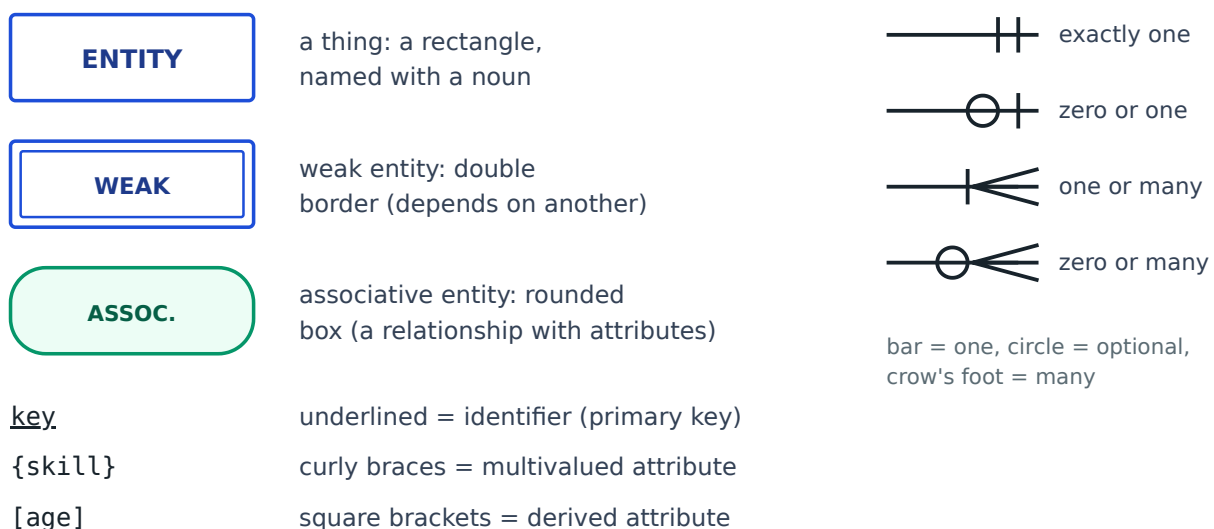


Figure 2.1. The basic ER notation used throughout this manual: entities (and weak and associative entities), the attribute markers, and the crow's-foot cardinality symbols read at the end nearest the entity they constrain.

1.3 Entities: the things

An **entity** is a person, place, object, event, or concept the organisation keeps data about. It is named with a noun, in capitals, singular: EMPLOYEE, PROPERTY, SALE. Two distinctions the examiner tests by name:

- ▶ **Entity type vs entity instance.** The entity *type* is the general category, EMPLOYEE, described once. An *instance* is one particular occurrence, the employee "S011, Tony Smith".
- ▶ **Strong vs weak entity.** A **strong (independent)** entity exists on its own and has its own identifier. A **weak (dependent)** entity cannot exist without another entity, its **identifying owner**, and has no full identifier of its own, only a **partial identifier**. It is drawn with a double border.

Q. Explain, with an example: weak entity type, strong entity type, unary relationship.

Weak entity type: an entity type whose existence depends on another entity type; a dependent entity that has no business meaning in the diagram without its owner.

Example: a DEPENDENT of an employee (it exists only while the EMPLOYEE does).

Strong entity type: an entity type that exists independently of any other; an independent entity with its own identifier. Example: EMPLOYEE.

Unary relationship: a relationship between the instances of a single entity type; also called a recursive relationship. Example: an EMPLOYEE "manages" another EMPLOYEE. (Diagram in section 1.5.)

1.4 Attributes: the details

An **attribute** is a property of an entity, named with a noun. The types you must distinguish, because the examiner asks you to:

● MUST MEMORISE

ATTRIBUTE TYPES

- ▶ **Simple:** cannot be broken down (age).
- ▶ **Composite:** made of parts (Name = firstName + lastName; Address = street + city + postcode).
- ▶ **Multivalued:** may hold more than one value per instance (an employee's {skills}). Shown in **curly braces**.
- ▶ **Derived:** calculated from other data ([age] from date of birth). Shown in **square brackets**.
- ▶ **Identifier (key):** distinguishes instances; no two share it. Shown underlined. A **composite identifier** is made of more than one attribute.

AS PRINTED

24/25 Q2A

(a) Differentiate between the following relational concepts with an example

- (i) Associative entity and ternary relationship. (2½marks)
- (ii) Multivalued attributes and derived attributes. (2½marks)

DTS224 24/25, Q2a, as printed.

Q. Differentiate, with an example: (i) multivalued vs derived attribute; (ii) composite vs multivalued attribute.

A **multivalued attribute** can take several values at once for one instance, for example an employee's {skills} = {Java, SQL, French}. A **derived attribute** holds a value worked out from other stored data, not stored in its own right, for example [age] computed from dateOfBirth and today's date. One holds many real values; the other holds a computed one.

A **composite attribute** is one attribute divided into sub-parts, for example Address into (street, city, postcode). A **multivalued attribute** is one attribute with several values of the same kind, for example {phoneNumbers}. Composite is one value in parts; multivalued is several values.

Attributes are worth drawing when the question is about the attributes themselves, as in this one, where the whole task is to represent an entity's attribute structure.

(c) An employee in an organization is represented by a unique ID, has a name which can be further broken down into first name and last name. The employee entity's date of employment is another valuable attribute, in addition to that, it has a payroll address which is comprised of the bank name, account number and sort code. The employee can have multiple skills and then the number of years employed can also be stored, although that can be gotten from the date employed. Represent this entity with an ER diagram showing the accurate representation of its attributes (3.5 marks)

DTS224 21/22, Q6c, as printed.

Q. An employee has a unique ID; a name broken into first and last name; a date of employment; a payroll address made of bank name, account number and sort code; multiple skills; and a number of years employed that can be derived from the date employed. Represent this entity with an ER diagram showing its attributes accurately.



Figure 2.2. EMPLOYEE with a composite Name, a composite payrollAddress, a multivalued {skills}, and a derived [yearsEmployed] (dashed oval), with empID as the underlined identifier.

1.5 Relationships and their degree

A **relationship** is an association between entity instances, named with a verb phrase on the line: "a STAFF *manages* a BRANCH". The **degree** of a relationship is the number of entity types it joins.

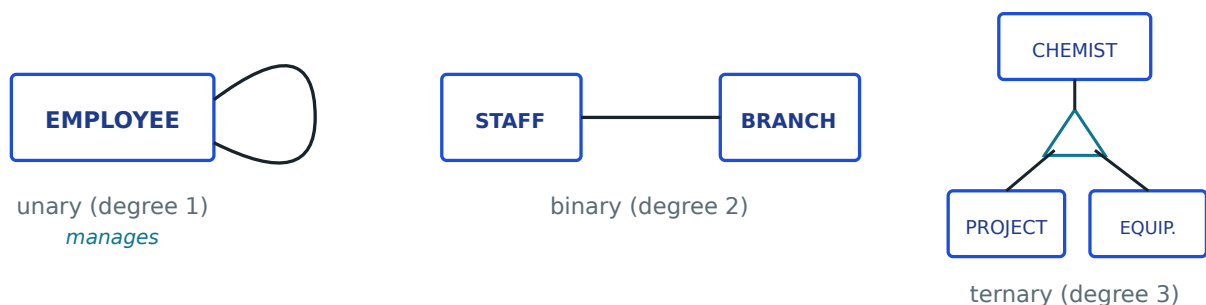


Figure 2.3. Degree is how many entity types a relationship joins: unary (an entity related to itself), binary (two, the commonest), ternary (three at once).

When a relationship itself has attributes (a grade for an enrolment, a date a certificate was completed), and especially when it is many-to-many, it is drawn as an **associative entity**: a rounded box on the relationship. An attribute cannot sit on a one-to-many relationship; it moves to the many side or becomes an associative entity.

Q. Explain with an example: attribute; ternary relationship; entity; multivalued attribute; associative entity; binary relationship. (Combined from Test 1 A and B.)

Attribute: a property or characteristic of an entity type of interest to the organisation (a PATIENT's Patient Name). **Entity:** a person, place, object, event, or concept about which the organisation keeps data (PATIENT). **Ternary relationship:** a simultaneous relationship among the instances of three entity types (CHEMIST uses EQUIPMENT on a PROJECT). **Binary relationship:** a relationship between the instances of two entity types, the most common kind (STUDENT enrolls in COURSE). **Multivalued attribute:** an attribute that may take more than one value for one instance ({skills}). **Associative entity:** an entity type that associates instances of other entity types and holds attributes peculiar to that relationship (ENROLMENT, holding grade).

1.6 Cardinality: the rule on the line

The most important marks on the diagram are the **cardinality constraints**: how many instances of one entity may, or must, relate to one instance of the other. Two numbers per end:

● MUST MEMORISE

MINIMUM AND MAXIMUM CARDINALITY

Minimum cardinality is participation. If it is **zero**, participation is **optional** (drawn with a circle). If it is **one**, participation is **mandatory** (drawn with a bar).

Maximum cardinality is **one** (a bar) or **many** (a crow's foot).

So each end carries two marks, read outer-then-inner: circle-then-crow's-foot is "zero or many", bar-then-crow's-foot is "one or many", bar-then-bar is "exactly one".

With the notation complete, the rest of this unit is drawing. Every scenario below is a real exam question; draw each yourself before reading the answer.

1.7 Worked: ConsultCo

AS PRINTED

20/21 Q1C, 19/20 Q2B, 15/16 Q2B

(c) Use the following business rules to create the ERD for a small consulting company named ConsultCo. Show all entities, relationships, optionalities, connectivities and cardinalities. A client can contract many projects, but each project is contracted by one client. Some clients in the company client list have not (yet) contracted any project(s). A project may have many (consulting company) employees assigned to it, and each employee may be assigned to many projects. Some employees are not assigned to any project. One employee leads each project. An employee may lead more than one project at a time. One employee coordinates each project. An employee may coordinate more than one project at a time. Some employees never lead or coordinate projects. Many of the consulting company employees have multiple job skills that are applicable to the consulting work. Each job description- system analyst, database designer, accountant and so on- can fit many employees. Some job descriptions are not yet filled. Some employees do not have a specific job classification. [10 marks]

QUESTION TWO

DTS224 20/21, Q1c, as printed.

2 (b) Use the following business rules to create the ERD for a small consulting company named ConsultCo. Show all entities, relationships, optionalities and cardinalities. A client can contract many projects, but each project is contracted by one client. Some clients in the company client list have not (yet) contracted any project(s). A project may have many (consulting company) employees assigned to it, and each employee may be assigned to many projects. Some employees are not assigned to any project. One employee leads each project. An employee may lead more than one project at a time. One employee coordinates each project. An employee may coordinate more than one project at a time. Some employees never lead or coordinate projects. Many of the consulting company employees have multiple job skills that are applicable to the consulting work. Each job description- system analyst, database designer, accountant and so on- can fit many employees. Some job descriptions are not yet filled. Some employees do not have a specific job classification. [12.5 Marks]

DTS224 19/20, Q2b, as printed.

2 (b) Use the following business rules to create the ERD for a small consulting company named ConsultCo. Show all entities, relationships, optionalities, connectivities and cardinalities. A client can contract many projects, but each project is contracted by one client. Some clients in the company client list have not (yet) contracted any project(s). A project may have many (consulting company) employees assigned to it, and each employee may be assigned to many projects. Some employees are not assigned to any project. One employee leads each project. An employee may lead more than one project at a time. One employee coordinates each project.

An

employee may coordinate more than one project at a time. Some employees never lead or coordinate projects. Many of the consulting company employees have multiple job skills that are

applicable to the consulting work. Each job description- system analyst, database designer, accountant and so on- can fit many employees. Some job descriptions are not yet filled. Some employees do not have a specific job classification. (10marks)

DTS224 15/16, Q2b, as printed.

EXAM QUESTION

PQ 20/21 Q1C · PQ 19/20 Q2B · PQ 15/16 Q2B

Use the following business rules to create the ERD for a consulting company named ConsultCo, showing all entities, relationships, optionalities and cardinalities. A client can contract many projects, but each project is contracted by one client. Some clients have not yet contracted any project. A project may have many employees assigned, and each employee may be assigned to many projects; some employees are assigned to none. One employee leads each project, and an employee may lead more than one. One employee coordinates each project, and may coordinate more than one. Some employees never lead or coordinate. Employees have many job skills, and each job description fits many employees; some job descriptions are unfilled and some employees have no job classification.

ANSWER

Four entities: CLIENT, PROJECT, EMPLOYEE, JOB. Read each rule into a line with its two cardinalities.

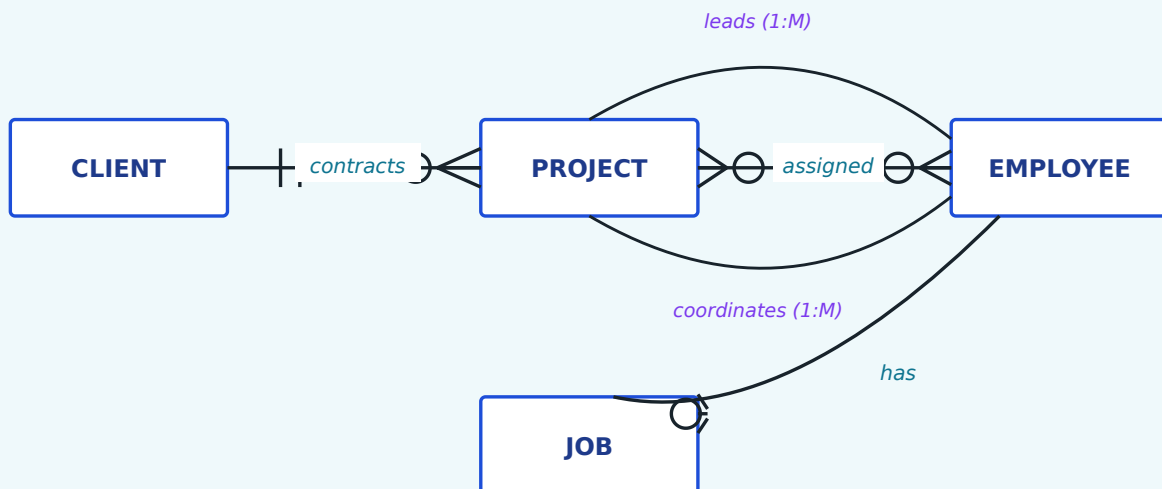


Figure 2.4. ConsultCo. CLIENT to PROJECT is one-to-many (a project has exactly one client; a client has zero or many projects). PROJECT to EMPLOYEE (assigned) is many-to-many, both optional. "leads" and "coordinates" are two separate one-to-many relationships from EMPLOYEE to PROJECT. EMPLOYEE to JOB is many-to-many, optional.

The trap here is drawing one relationship between EMPLOYEE and PROJECT. There are three (assigned, leads, coordinates), and each is a separate line because each is a different business rule.

1.8 Worked: hospital (physician, patient, ward)

AS PRINTED

24/25 Q2C

(c) Express the relationships in the business rule with just one connected ER diagram to show the

degree and cardinalities of the relationships.

A hospital has a large number of registered physicians. Attributes of PHYSICIAN include Physician ID (the identifier) and Specialty. Patients are admitted to the hospital by physicians.

Attributes of PATIENT include Patient ID (the identifier) and Patient Name. Any patient who is

admitted must have exactly one admitting physician. A physician may optionally admit any number of patients. Once admitted, a given patient must be treated by at least one physician. A

particular physician may treat any number of patients, or may not treat any patients.

Whenever a

physician treats a patient, the hospital wishes to record the treatment details.

Components of

Treatment Detail include Date, Time, and Results.

Did you draw more than one relationship between physician and patient? How would you include

in the ERD the need to represent the date a patient is admitted for each time he or she is admitted?

A patient is assigned to a particular ward, which can accommodate many patients at a time or is

vacant. A ward is characterized by its name and type. (9½marks)

Question Three

DTS224 24/25, Q2c, as printed.

EXAM QUESTION

PQ 24/25 Q2C · TEST 1 A(B)

A hospital has many PHYSICIANS (Physician ID, Specialty) and many PATIENTs (Patient ID, Patient Name). A patient who is admitted has exactly one admitting physician; a physician may admit any number of patients or none. Once admitted, a patient must be treated by at least one physician; a physician may treat any number of patients or none. When a physician treats a patient, the hospital records Treatment Detail (Date, Time, Results). A patient is assigned to one WARD (Name, Type); a ward holds many patients or is vacant. Draw one connected ER diagram showing degree and cardinalities.

ANSWER

Two relationships run between PHYSICIAN and PATIENT (admits, and treats), so draw both. "treats" is many-to-many and carries Treatment Detail, so it becomes an associative entity.

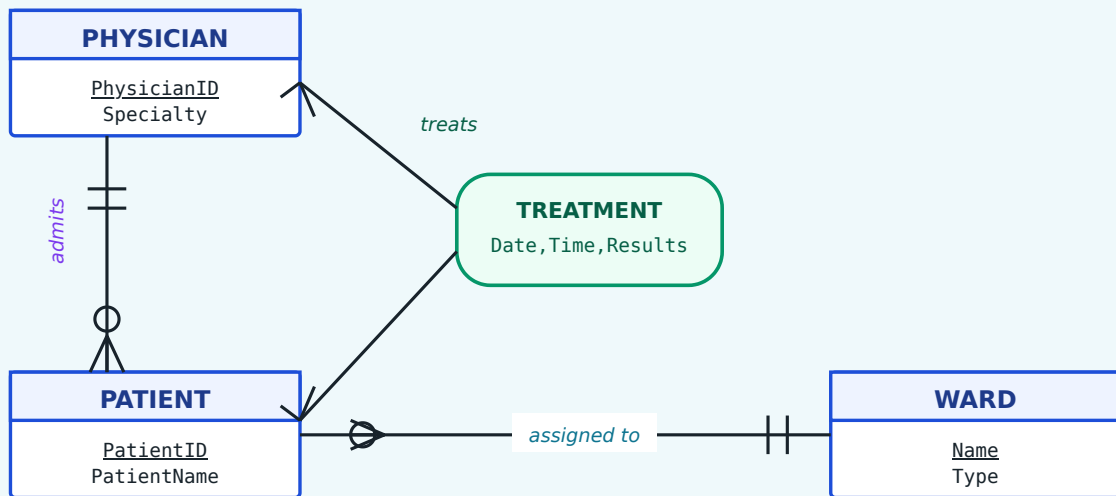


Figure 2.5. Hospital. PHYSICIAN "admits" PATIENT is one-to-many (a patient has exactly one admitting physician, mandatory; a physician admits zero or many). "treats" is many-to-many and carries Treatment Detail, so it is the associative entity TREATMENT. PATIENT is assigned to exactly one WARD; a WARD holds zero or many patients.

Yes, there is more than one relationship between physician and patient, which is exactly what the question's hint asks you to notice. The date a patient is admitted each time would be recorded as an attribute on the "admits" relationship (an associative entity if admissions repeat).

1.9 Worked: UPS package tracking

AS PRINTED

21/22 Q2A, 20/21 Q3A

(a) UPS prides itself on having up-to-date information on the processing and current location of each shipped item. To do this, UPS relies on a company-wide information system. Shipped items are the heart of the UPS product tracking information system. Shipped items can be characterized by item number (unique), weight, dimensions, insurance amount, destination, and final delivery date. Shipped items are received into the UPS system at a single retail center. Retail centers are characterized by their type, uniqueID, and address. Shipped items make their way to their destination via one or more standard UPS transportation events (i.e., flights, truck deliveries). These transportation events are characterized by a unique scheduleNumber, a type (e.g. flight, truck), and a deliveryRoute. create an Entity Relationship (E-R) diagram that captures this information about the UPS system. Be certain to indicate identifiers and cardinality constraints.

(9 marks)

DTS224 21/22, Q2a, as printed.

(a) UPS prides itself on having up-to-date information on the processing and current location of each shipped item. To do this, UPS relies on a company-wide information system. Shipped items are the heart of the UPS product tracking information system. Shipped items can be characterized by item number (unique), weight, dimensions, insurance amount, destination, and final delivery date. Shipped items are received into the UPS system at a single retail center. Retail centers are characterized by their type, uniqueID, and address. Shipped items make their way to their destination via one or more standard UPS transportation events (i.e., flights, truck deliveries). These transportation events are characterized by a unique scheduleNumber, a type (e.g. flight, truck), and a deliveryRoute. create an Entity Relationship (E-R) diagram that captures this information about the UPS system. Be certain to indicate identifiers and cardinality constraints. [9 marks]

DTS224 20/21, Q3a, as printed.

EXAM QUESTION

PQ 21/22 Q2A · PQ 20/21 Q3A

UPS tracks shipped items. A SHIPPED ITEM has itemNumber (unique), weight, dimensions, insuranceAmount, destination, and finalDeliveryDate. Each shipped item is received into the system at a single RETAIL CENTER (type, uniqueID, address). A shipped item travels to its destination via one or more TRANSPORTATION EVENTS (scheduleNumber unique, type, e.g. flight or truck, deliveryRoute). Create an ER diagram with identifiers and cardinality constraints.

ANSWER

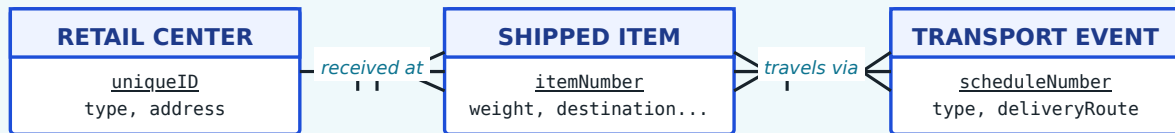


Figure 2.6. UPS. Each SHIPPED ITEM is received at exactly one RETAIL CENTER (one-to-many). A SHIPPED ITEM travels via one or many TRANSPORT EVENTS and an event carries many items (many-to-many).

1.10 Worked: supplier, item, product, order

AS PRINTED

21/22 Q6A

(a) Draw a detailed Entity-Relationship diagram for the following business rule. Show the cardinality constraints for each relationship.

A Supplier may supply many Items. Each Item is supplied by any number of suppliers. Each Item must be used in the assembly of at least one Product and may be used in many products. A Supplier may send many Shipments. However, each Shipment must be sent by exactly one Supplier. A Shipment must include one or more Items. An item may be included on several Shipments. A customer may submit any number of Orders. However, each Order must be submitted by exactly one Customer. An Order must request one or more products. A given Product may not be requested on any Order, or may be requested on one or more orders (12 marks)

DTS224 21/22, Q6a, as printed.

EXAM QUESTION

PQ 21/22 Q6A

A SUPPLIER may supply many ITEMS; each item is supplied by any number of suppliers. Each ITEM must be used in at least one PRODUCT and may be used in many. A SUPPLIER may send many SHIPMENTS; each shipment is sent by exactly one supplier. A SHIPMENT must include one or more ITEMS; an item may appear on several shipments. A CUSTOMER may submit any number of ORDERS; each order is submitted by exactly one customer. An ORDER must request one or more PRODUCTS; a product may be requested on none or many orders. Draw the ER diagram with cardinality constraints.

ANSWER

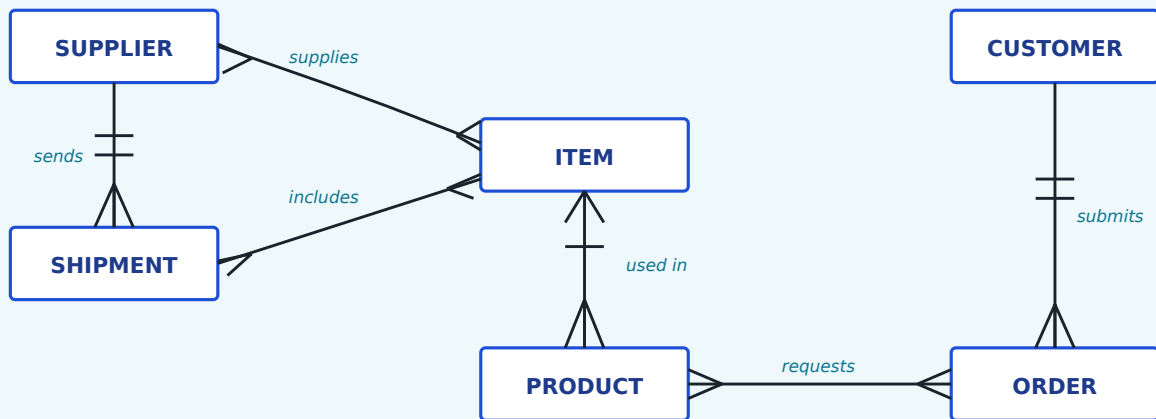


Figure 2.7. Supplier/item/product/order. All of supplies, used-in, includes, and requests are many-to-many; sends and submits are one-to-many. ITEM's participation in "used in" is mandatory (one or many), matching "must be used in at least one product".

1.11 Rapid ER practice: three more scenarios

Draw each before reading. All three are past questions.

WORKED

TEST 1 B(B) LAB: A TERNARY RELATIONSHIP

A CHEMIST (Employee ID, Name, Phone) works on one or more PROJECTs (Project ID, Start Date) using EQUIPMENT (Serial No, Cost). Record Assign Date, the date a given equipment item was assigned to a chemist on a project. A chemist must have at least one project and one equipment item; equipment and projects need not be assigned.

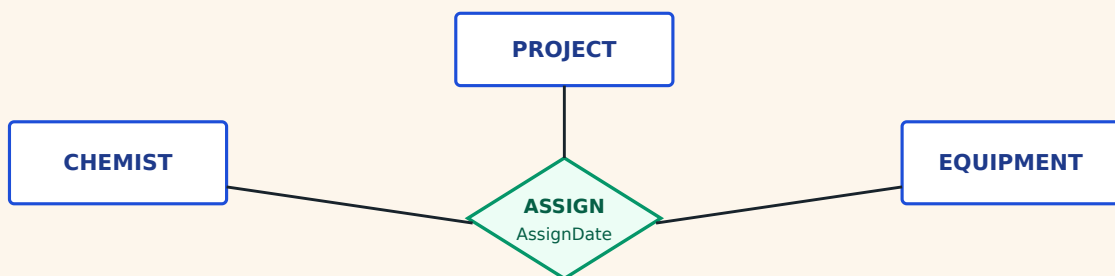


Figure 2.8. A single ternary relationship ASSIGN joins CHEMIST, PROJECT, and EQUIPMENT, drawn as an associative entity because it carries AssignDate.

A firm has SALES OFFICES (Office_Number, Location), each with one or more EMPLOYEEs (Employee_ID, Name). An employee is assigned to exactly one office; each office has one employee who manages it, and an employee manages only their own office. The firm lists PROPERTY (Property_ID, Location as address/city/state/zip); each property is listed with exactly one office; an office lists zero or many. Each property has one or more OWNERS (Owner_ID, Name); an owner owns one or many properties; Percent_Owned is an attribute of that relationship.

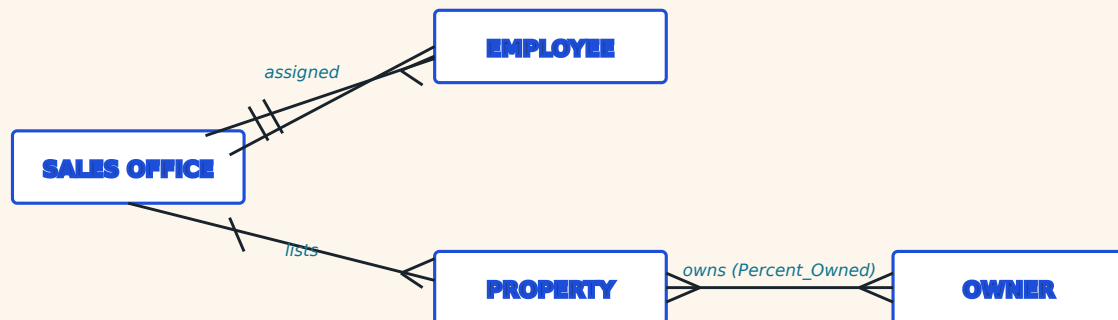


Figure 2.9. Real-estate firm. SALES OFFICE to EMPLOYEE is one-to-many (assigned) with a separate manages relationship; SALES OFFICE lists PROPERTY one-to-many; PROPERTY to OWNER is many-to-many with Percent_Owned on the relationship.

5 (c) Draw the ER diagram to represent the following business rules.

A vendor builds multiple types of tablet computers. Each has a type identification number and a name. The key specifications for each type include amount of storage space and display type. The company uses multiple processor types, exactly one of which is used for specific tablet computer type; obviously, the same processor can be used in multiple types of tablets. Each processor has a manufacturer and a manufacturer's unique code that identifies it. [2.5 Marks]

DTS224 19/20, Q5c, as printed.

A VENDOR builds multiple TABLET types (typeID, name, storageSpace, displayType). The company uses multiple PROCESSOR types, exactly one of which is used for a specific tablet type; the same processor can be used in many tablet types. Each processor has a MANUFACTURER and a manufacturer code.



Figure 2.10. VENDOR builds many TABLET types (one-to-many). Each TABLET uses exactly one PROCESSOR, but one PROCESSOR serves many tablets (many-to-one). Each PROCESSOR is made by exactly one MANUFACTURER.

1.12 Reading a diagram you are given

Some questions hand you a diagram and ask you to read the rules back out of it, the reverse skill. The course text's cellular-operator activity is the model: for each question, name the symbol that decides your answer.

Given an ERD of CUSTOMER, PLAN, HANDSET, MANUFACTURER, OPERATING SYSTEM, you answer by pointing at a cardinality mark:

- ▶ **Can a customer have unlimited plans?** Look at the maximum cardinality at the PLAN end of customer-plan. If it is a crow's foot, yes; if a bar, no.
- ▶ **Can a customer exist without a plan?** Look at the minimum at the PLAN end. A circle (zero) means yes; a bar (one) means no.
- ▶ **Can a plan exist without a known customer?** Look at the minimum at the CUSTOMER end. Circle means yes.
- ▶ **Can a handset be kept without a plan?** Minimum at the PLAN end of handset-plan: circle means yes.
- ▶ **Can one operating system run on many handset types?** Maximum at the HANDSET end of OS-handset: a crow's foot means yes.

The method is always the same: the answer to "can/must it..." is a single cardinality mark, and a full-mark answer names which mark and which end.

1.13 The course text's own questions

WORKED

TEXT M2U1 SELF-ASSESSMENT AND ASSIGNMENTS

Self-assessment

Is there a difference between data modelling and conceptual modelling? They are closely related: data modelling is the whole activity of documenting an organisation's data rules; conceptual modelling is the first, high-level stage of it that produces the ER diagram, before any logical or physical detail. So conceptual modelling is a part of data modelling, not a separate thing.

What determines how data are handled? The business rules: the organisation's policies and procedures, captured as entities, relationships, and constraints.

Relationship instance vs associative entity. Both link entity instances. A relationship instance is a single occurrence of a relationship (this student in this course). An associative entity is a relationship promoted to an entity because it has its own attributes or its own identifier, and can itself take part in relationships.

Assignments (book/author, and TEACHER)

Book and publisher (ACT1): BOOK (ISBN, title, price, pubDate) is published by one PUBLISHER (pubID, name); a publisher publishes many books. One-to-many. **Book and author (ACT2):** BOOK to AUTHOR is many-to-many (a book has one or many authors; an author has zero or many books, allowing prospective authors). **Royalties (ACT3):** the many-to-many carries royaltyPercent, isLeadAuthor, and sequencePosition, so it becomes an associative entity.

TEACHER identifier (TMA4): Trainer and Training Count show a teacher organises many workshops, so a WORKSHOP is a separate entity (or a multivalued attribute), and the identifier for TEACHER is Email (unique and stable), not Name (not guaranteed unique).

RECALL

COVER THE ANSWERS

1. What does a crow's foot mean? A bar? A circle?
2. How is a multivalued attribute shown? A derived one? An identifier?
3. What is the degree of a relationship between three entity types?
4. When does a relationship become an associative entity?
5. A patient has exactly one ward. Which two marks sit at the ward end?

ANSWERS

1. Many; one; optional (zero). 2. Curly braces; square brackets; underline. 3. Ternary (degree 3). 4. When it has its own attributes, or is many-to-many. 5. Two bars (exactly one): mandatory, maximum one.

MODULE TWO • UNIT 2

The Enhanced E-R Model

When some things are special kinds of other things, an employee who is either hourly or salaried, an offering that is either a product or a service, the plain ER model is not enough.

The enhanced ER model adds one construct, the supertype and its subtypes, and four rules that go with it. Those rules are examined every single year.

By the end of this unit you can: tell a supertype from a subtype; distinguish generalization from specialization; apply the completeness (total/partial) and disjointness (disjoint/overlap) constraints; use a subtype discriminator; and draw any supertype/subtype hierarchy the paper asks for.

2.1 Supertypes and subtypes

● MUST MEMORISE

THE TWO WORDS

A **subtype** is a meaningful subgrouping of the entities in an entity type, for example GRADUATE and UNDERGRADUATE within STUDENT. A **supertype** is the generic entity type that holds what the subgroups have in common, for example STUDENT. The relationship is drawn with a circle between the supertype and its subtypes, and the subtypes connect to it with the subset symbol.

You use a supertype/subtype structure when either (a) some attributes apply only to some instances (only salaried employees have an annual salary), or (b) some instances take part in a relationship that others do not (only consultants relate to a CONTRACT).

● MUST MEMORISE

ATTRIBUTE INHERITANCE

A subtype **inherits** every attribute and every relationship of its supertype. Attributes are placed at the highest level to which they apply: SSN at PERSON, dateHired at EMPLOYEE, rank at FACULTY. So a FACULTY instance has SSN, Name (from PERSON), dateHired, salary (from EMPLOYEE), and rank (from FACULTY). This is why the structure avoids redundancy: the common attributes are stored once, at the top.

2.2 Generalization and specialization

AS PRINTED

24/25 Q4A, 23/24 Q4A

- (a) Briefly differentiate the following pair of concepts with example
- (i) Total specialization and partial specialization. (2½marks)
 - (ii) Supertype and subtype. (2½marks)

DTS224 24/25, Q4a, as printed.

- (a) Briefly differentiate the following pair of concepts with example
- (i) Supertypes and subtypes (2½marks)
 - (ii) Generalization and specialization (2½marks)

DTS224 23/24, Q4a, as printed.

Q. Differentiate: (i) supertype and subtype; (ii) generalization and specialization.

Supertype vs subtype: a supertype is the general entity type holding common attributes; a subtype is a specialised subgroup of it holding its own extra attributes and inheriting the supertype's. VEHICLE is a supertype; CAR and TRUCK are subtypes.

Generalization vs specialization: both build the same hierarchy but from opposite ends. **Generalization** is *bottom-up*: you start with specialised entity types (CAR, TRUCK) and define a general supertype (VEHICLE) from what they share.

Specialization is *top-down*: you start with a general entity type (VEHICLE) and define subtypes (CAR, TRUCK) for the parts that differ.

2.3 The two constraints you are always asked for

Every EER question turns on two independent choices. Get them both and you have the diagram.

● MUST MEMORISE

COMPLETENESS AND DISJOINTNESS

Completeness (does every supertype instance have to be a subtype?)

- ▶ **Total specialization:** yes, every supertype instance must belong to some subtype. Drawn with a **double line** from the supertype to the circle.
- ▶ **Partial specialization:** no, a supertype instance need not belong to any subtype. Drawn with a **single line**.

Disjointness (can an instance be more than one subtype at once?)

- ▶ **Disjoint rule (d):** no, an instance is at most one subtype. Circle marked **d**.
- ▶ **Overlap rule (o):** yes, an instance may be several subtypes at once. Circle marked **o**.

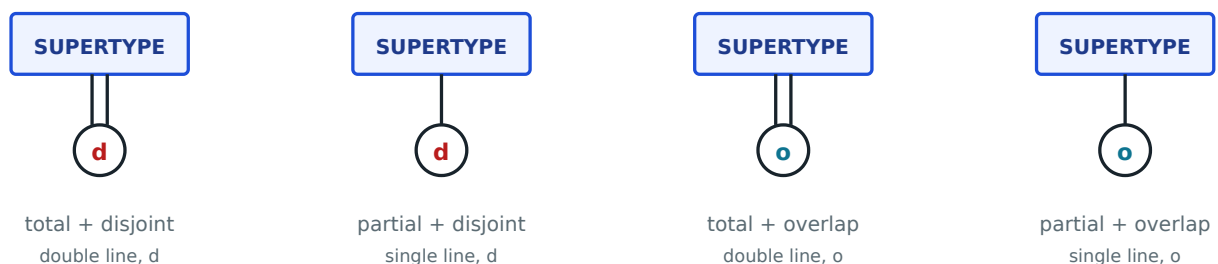


Figure 2.11. The four combinations. Completeness is the line (double = total, single = partial); disjointness is the letter (d = disjoint, o = overlap). Every EER question picks one of these four.

One more mark the paper asks for by name. The **subtype discriminator** (also called the discriminating attribute) is a supertype attribute whose value says which subtype a row belongs to. It is written beside the specialization circle. For a **disjoint** specialization it holds one value per row (for EMPLOYEE over SALARIED and HOURLY, a discriminator `empType` reads "S" or "H"). For an **overlap** specialization one value is not enough, so the discriminator is a set of yes/no flags, one per subtype.

- (b) Explain the following concepts with examples
- (i) Entity type (1½ marks)
 - (ii) Required attribute (1½ marks)
 - (iii) Disjoint rule (1½ marks)
 - (iv) Overlap rule (1½ marks)

DTS224 24/25, Q6b, as printed.

- (a) Differentiate between the following relational concepts with Example
- (i) Strong and weak entity (2½marks)
 - (ii) Disjoint and Overlap participation (2½marks)

DTS224 23/24, Q2a, as printed.

Q. Explain: total vs partial specialization; disjoint rule vs overlap rule; entity type; required attribute.

Total specialization: every instance of the supertype must be a member of at least one subtype. **Partial specialization:** a supertype instance need not be a member of any subtype. **Disjoint rule:** a supertype instance that is in one subtype cannot at the same time be in another. **Overlap rule:** a supertype instance may be a member of two or more subtypes at the same time. **Entity type:** a collection of entities sharing common properties, described once (EMPLOYEE). **Required attribute:** an attribute that must have a value for every instance (an employee's empID).

2.4 Worked: the four-constraint classic (CAMPER, BIKER, RUNNER)

This exact question appears in at least three papers. PERSON has subtypes CAMPER, BIKER, RUNNER. Draw a separate EER segment for each situation. The whole task is reading the words into the line and the letter.

- 1a. Extensively differentiate between the following terms:
- Database Manager and Database Administrator
 - Data Manipulation Language and Data Definition Language
 - SQL and NOSQL Databases
 - Composite and Multi-valued Attributes

DTS224 21/22, Test 1 Q1a, as printed.

2. At a weekend retreat, the entity type PERSON has three subtypes: CAMPER, BIKER, and RUNNER. Draw a separate Extended Entity Relationship (EER) diagram segment for each of the following situations:
- At a given time, a person must be exactly one of these subtypes.
 - A person may or may not be one of these subtypes. However, a person who is one of these subtypes cannot at the same time be one of the other subtypes.
 - A person may or may not be one of these subtypes. On the other hand, a person may be any two (or even three) of these subtypes at the same time.
 - At a given time, a person must be at least one of these subtype

DTS224 21/22, Test 1 Q2, as printed.

3. Explain the following terms:
- Open Database Connectivity (ODBC)
 - Data Mining
 - Knowledge-Based Systems
 - Locking Methods

QUESTION ONE

DTS224 21/22, Test 1 Q3, as printed.

- (b) At a weekend retreat, the entity type PERSON has three subtypes: CAMPER, BIKER, and RUNNER. Draw a separate Extended Entity Relationship (EER) diagram segment for each of the following situations:
- At a given time, a person must be exactly one of these subtypes. (2 marks)
 - A person may or may not be one of these subtypes. However, a person who is one of these subtypes cannot at the same time be one of the other subtypes. (2 marks)
 - A person may or may not be one of these subtypes. On the other hand, a person may be any two (or even three) of these subtypes at the same time. (2 marks)
 - At a given time, a person must be at least one of these subtype (2 marks)

DTS224 21/22, Q5b, as printed.

- 6 (d) At a weekend retreat, the entity type PERSON has three subtypes: CAMPER, BIKER, and RUNNER. Draw a separate EER diagram segment for each of the following situations: (6marks)
- At a given time, a person must be exactly one of these subtypes.
 - A person may or may not be one of these subtypes. However, a person who is one of these subtypes cannot at the same time be one of the other subtypes.
 - A person may or may not be one of these subtypes. On the other hand, a person may be

any two (or even three f these subtypes at the same time.
 (iv) At a given time, a person must be at least one of these subtype [4 Marks]
 DTS224 19/20, Q6d, as printed.

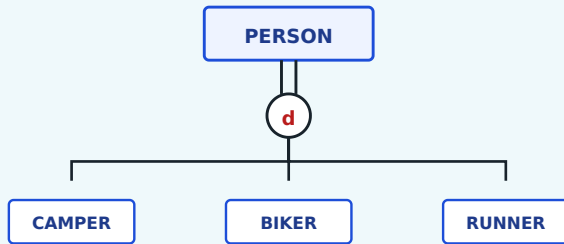
ANSWER

PQ 21/22 TEST

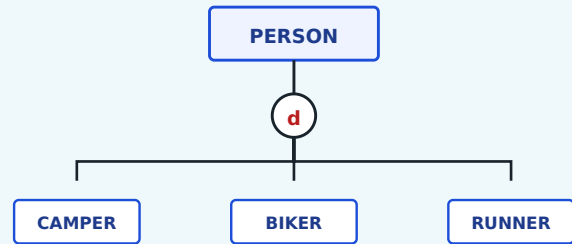
PQ 21/22 Q5B

PQ 19/20 Q6D

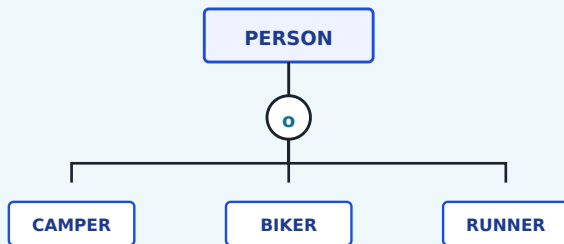
(a) must be exactly one = total + disjoint



(b) optional, not two = partial + disjoint



(c) optional, may be several = partial + overlap



(d) at least one = total + overlap

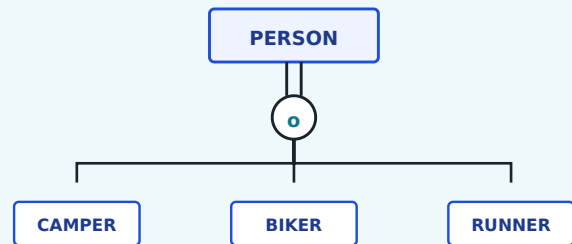


Figure 2.12. The four PERSON segments. "Exactly one" is total (double line) and disjoint (d). "May or may not, but not two" keeps disjoint but drops to partial (single line). "May be several" switches to overlap (o) and stays partial. "At least one" is total (double line) with overlap (o).

2.5 Worked: a multi-level hierarchy (university HR)

AS PRINTED

21/22 Q4A

(a) Model the following business rule based on supertype/subtype relationships. Show the constraints on the supertype/subtype relationships. List out clearly the attributes of each entity and model the supertype/subtype relationships.

Suppose you are asked to model the human resources in a university. Using specialization (a to-down approach) where the root general entity type is PERSON with attributes SSN (identifier), Name, Address, Gender and DOB. There are three types of persons that are of interest to the university which are: EMPLOYEE (persons who work for the university), STUDENT (persons who attend classes), and ALUMNUS (persons who have graduated). In this university a person might belong to more than one categories. Each employee has a value for Date hired and Salary and Student's Major department is recorded. For each ALUMNI the graduating degree is determined by year of graduation, designation and date of graduation. These two types of EMPLOYEE: FACULTY and STAFF have the following specific attributes respectively: Rank and Position. Also, a STUDENT could either be GRADUATE student with Test score or UNDERGRADUATE student with Class standing. (9.5 marks)

DTS224 21/22, Q4a, as printed.

EXAM QUESTION

PQ 21/22 Q4A

Model university human resources by specialization from PERSON (SSN, Name, Address, Gender, DOB). Three subtypes: EMPLOYEE (Date hired, Salary), STUDENT (Major), ALUMNUS (graduating degree: year, designation, date). A person may belong to more than one. EMPLOYEE further splits into FACULTY (Rank) and STAFF (Position). STUDENT further splits into GRADUATE (Test score) and UNDERGRADUATE (Class standing).

ANSWER

"May belong to more than one" fixes the top level as **overlap (o)**. The two further splits are each disjoint.

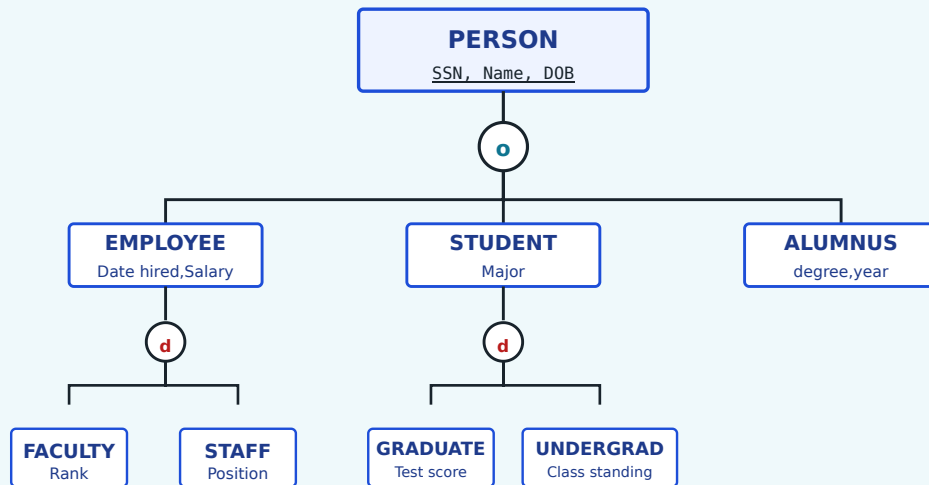


Figure 2.13. University HR. *PERSON* specializes with overlap (a person may be employee, student, and alumnus at once). *EMPLOYEE* and *STUDENT* each specialize further with the disjoint rule. Attributes sit at the level where they first apply.

2.6 Worked: technology-company offerings

AS PRINTED

25/26 Q5B

b. Develop an EER model for the following situation, using the crowfoot notation. (11 mks)

A technology company provides offerings to its customers. Offerings are of two separate types:

products and services. Offerings are identified by an offering ID and an attribute of description. In

addition, products are described by product name, standard price, and date of first release; services

are described by name of the company's unit responsible for the service and conditions of service.

There are repair, maintenance, and other types of services. A repair service has a cost and is the repair

of some product; a maintenance service has an hourly rate. Fortunately, some products never require

repair. However, there are many potential repair services for a product.

A customer may purchase an offering, and the company needs to keep track of when the offering was

purchased and the contact person for that offering with the customer. Unfortunately, not all offerings

are purchased. Customers are identified by customer ID and have descriptive data of name, address,

and phone number. When a service is performed, that service is billed to some customer.

Because

some customers purchase offerings for their clients, a customer may be billed for services he or she

did not purchase, as well as for ones that were purchased. When a customer is billed for a service

(although some may never require a service of any type), the company needs to keep track of the date

the service was performed, the date the bill is due, and the amount due.

QUESTION SIX

(17 ½ mks)

DTS224 25/26, Q5b, as printed.

EXAM QUESTION

PQ 25/26 Q5B

A company provides OFFERINGS (offering ID, description). An offering is either a PRODUCT (product name, standard price, first-release date) or a SERVICE (responsible unit, conditions). Services are of types REPAIR (cost; repairs some product) and MAINTENANCE (hourly rate), among others. Build the EER for the offering hierarchy.

ANSWER

An offering is exactly one of product or service: total (every offering is one) and disjoint (not both). SERVICE then specializes into REPAIR, MAINTENANCE, and others, so that inner split is partial (the "and other types" leaves room) and disjoint.

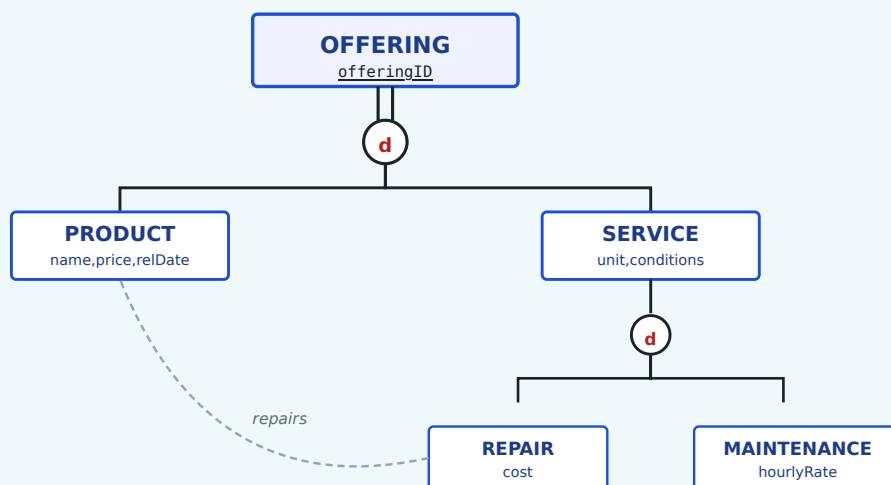


Figure 2.14. OFFERING is total and disjoint over PRODUCT and SERVICE. SERVICE is partial and disjoint over REPAIR, MAINTENANCE and others; a REPAIR repairs a PRODUCT (dashed relationship).

WORKED

TEXT M2U2 SELF-ASSESSMENT AND ASSIGNMENTS

The two most important constraints in supertype/subtype relationships?

Completeness (total or partial) and disjointness (disjoint or overlap).

What does partial specialization specify, and how does it differ from total?

Partial says a supertype instance need not belong to any subtype; total says every supertype instance must belong to at least one subtype. Partial allows "none of the above"; total forbids it.

The mental models for developing supertype/subtype relationships?

Generalization (bottom-up) and specialization (top-down).

When is each used, and what is attribute inheritance? (TMA) Use a subtype when some attributes or relationships apply only to some instances. Attribute inheritance means each subtype automatically has all attributes and relationships of its supertype, so shared data is stored once at the top.

1. Double line into the circle means what? Single line?
2. "d" in the circle means what? "o"?
3. Generalization: bottom-up or top-down?
4. "A person must be at least one of the subtypes and may be several." Which line and which letter?
5. Where is an attribute placed in a hierarchy?

ANSWERS

1. Total specialization; partial specialization. 2. Disjoint (at most one subtype); overlap (may be several). 3. Bottom-up. 4. Double line (total) and "o" (overlap). 5. At the highest level to which it applies.

MODULE TWO • UNIT 3

Semi-Structured Models: XML and JSON

Not all data fits neatly into a table with fixed columns. Semi-structured models keep some structure, tags and nesting, while letting every record differ. XML and JSON are the two you must be able to read and write.

By the end of this unit you can: say what semi-structured data is and when it is used; read and write both XML and JSON; and explain how XML data is stored, queried, and displayed.

3.1 Between rigid and formless

A **structured** model (the relational table) has a fixed schema: every row has the same columns. **Unstructured** data (a plain document, an image) has none. A **semi-structured** model sits between: it carries structure through tags or keys, but the structure can vary from one record to the next and change often.

- ▶ **Flexible schema.** Two forms: **wide-column**, where the set of attributes is not fixed and new ones are added as needed; and **sparse-column**, where there is a fixed but huge set of possible attributes and each record fills in only the few it uses, the rest left null.
- ▶ **Multivalued types.** An attribute may hold a set or array of values, for example interests = {basketball, cooking, jazz}, which a flat table forbids.
- ▶ **Nested types.** An attribute may itself be structured, for example name broken into first and last. The two models built on these ideas are JSON and XML.

3.2 JSON and XML, side by side

Both represent the same fact. JSON (JavaScript Object Notation) uses braces and key-value pairs; XML (eXtensible Markup Language) uses paired tags in angle brackets. The same staff record in each:

JSON

```
{
  "staffNo": "SG37",
  "name": { "first": "Ann", "last": "Beech" },
  "skills": ["admin", "sales"]
}
```

XML

```
<staff>
  <staffNo>SG37</staffNo>
  <name><first>Ann</first><last>Beech</last></name>
  <skills>admin</skills>
  <skills>sales</skills>
</staff>
```

Both are **self-describing** (the tags or keys name the data), **hierarchical** (values nest inside values), and need no fixed schema. XML tags always come in pairs, `<tag>...</tag>`, and can represent both relational rows and tree-shaped documents such as a purchase order with nested line items.

3.3 Storing, querying, and displaying XML

- ▶ **Storing.** Three ways: **shredding** it into relational tables (each element to a column, with relationship tables); a dedicated **XML column** type (optionally validated against an XSD schema); or a purpose-built **native XML database**.
- ▶ **Querying.** **XPath** expressions locate data by a path through the tree; **XQuery** is the full query language built on XPath. As the text puts it, "XQuery is to XML as SQL is to relational databases".
- ▶ **Displaying.** A stylesheet in **XSL** (Extensible Stylesheet Language), applied through **XSLT**, transforms XML into a display format such as an HTML table, so the same data can render on a phone, a tablet, or a page.

● MUST MEMORISE

KEY FEATURES OF SEMI-STRUCTURED DATA

- ▶ **Schema flexibility:** no fixed schema; fields vary per record.
- ▶ **Hierarchical structure:** data nests in trees or graphs.
- ▶ **Self-describing:** tags or keys carry the data's own metadata.
- ▶ **Partial organisation:** more manageable than unstructured data because the tags impose some order.

WORKED

TEXT M2U3 SELF-ASSESSMENT

Is XML the same as HTML? Justify. No. HTML has a *fixed* set of tags with a fixed meaning, and its job is to *display* content in a browser. XML lets you *define your own* tags, and its job is to *describe and carry* data, not to display it. HTML says how something looks; XML says what something is.

Explain the semi-structured data types. Flexible-schema records (wide-column and sparse-column), multivalued attributes (sets or arrays of values), and nested attributes (structured values within values), as realised by JSON and XML.

Four inconveniences of semi-structured models (TMA): no enforced schema so data quality is harder to guarantee; querying is less mature than SQL (XQuery is not widely adopted); larger storage because tags repeat on every record; and weaker support for the integrity constraints a relational DBMS enforces automatically.

- (a) Extensively differentiate between the following terms:
- i. Database Manager and Database Administrator (2 marks)
 - ii. SQL and NOSQL Databases (2 marks)
 - iii. Composite and Multi-valued Attributes (2 marks)

DTS224 21/22, Q5a, as printed.

WORKED

PQ 21/22 Q1(III)

PQ 21/22 Q5A

Q. Differentiate between SQL and NoSQL databases.

A **SQL (relational)** database stores data in tables with a fixed schema and relationships enforced by keys, and is queried with SQL; it favours strong consistency and structured data. A **NoSQL** database stores data in flexible forms (documents, key-value pairs, wide-columns, graphs) with little or no fixed schema, and scales out across many machines; it favours flexibility and horizontal scaling over strict structure. Use SQL when the data is regular and integrity matters most; use NoSQL when the data is varied, huge, or rapidly changing.

RECALL

COVER THE ANSWERS

1. Give the one-line difference between XML and HTML.
2. Name the three ways to store an XML document.
3. XQuery is to XML as what is to relational databases?
4. What does XSLT do?

ANSWERS

1. XML describes data with tags you define; HTML displays content with fixed tags. 2. Shred into relational tables; an XML column; a native XML database. 3. SQL. 4. Transforms XML into a display format such as an HTML table.

MODULE TWO · UNIT 4

Worked Conceptual Models

Everything in this module, applied. Five everyday systems, each turned from a sentence into an ER diagram. If you can do these cold, you can do any modelling question the paper sets.

By the end of this unit you can: read a short real-world description and produce a correct conceptual (ER) model for it, and say why you would build that model before touching a database.

4.1 Five systems at a glance

SYSTEM	ENTITIES	KEY RELATIONSHIPS
University	STUDENT, COURSE, LECTURER, DEPARTMENT	enrols in, teaches, belongs to
Hospital	PATIENT, DOCTOR, APPOINTMENT, TREATMENT	consults, prescribes, undergoes
Online retail	CUSTOMER, PRODUCT, ORDER, PAYMENT	places, contains, makes
Library	BOOK, MEMBER, LIBRARIAN, LOAN	borrows, returns
Hotel	GUEST, ROOM, STAFF, RESERVATION	books, checks in, has type/rate

4.2 Worked: hospital

WORKED**TEXT M2U4-SA1**

Q. Draw a simple ER diagram for a hospital with patients, doctors, and treatments.

```
graph LR; DOCTOR[DOCTOR] -- treats --> TREATMENT((TREATMENT  
date, diagnosis)); PATIENT[PATIENT] -- receives --> TREATMENT
```

Figure 2.15. Hospital. DOCTOR and PATIENT are joined through the associative entity TREATMENT (a doctor gives many treatments; a patient receives many), which carries the treatment's own attributes.

4.3 Worked: hotel reservation

WORKED

TEXT M2U4-TMA1

Q. Create a conceptual model for a hotel reservation system with guests, rooms, reservations, and staff.

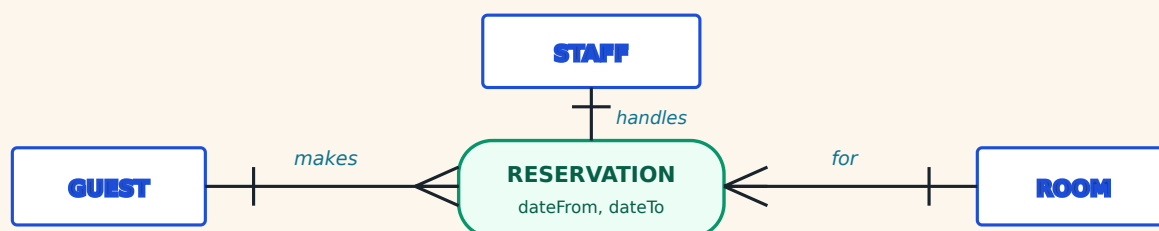


Figure 2.16. Hotel. A GUEST makes many RESERVATIONS; each reservation is for exactly one ROOM (a room takes many over time) and is handled by one STAFF member. ROOM carries type and rate; RESERVATION carries the dates.

WORKED

TEXT M2U4 LIBRARY ATTRIBUTES AND WHY-MODEL

List four attributes for each entity in a library system (TMA2). BOOK (ISBN, title, author, year); MEMBER (memberID, name, address, joinDate); LIBRARIAN (staffID, name, shift, desk); LOAN (loanID, dateOut, dueDate, returnDate).

Why create a conceptual model before building a database (TMA3)? It captures the business rules and the structure of the data in a form everyone can agree on, before any cost is sunk in tables or code. It surfaces mistakes early (a missing relationship, a wrong cardinality) when they are cheap to fix, improves communication between users and developers, and gives a DBMS-independent blueprint that the logical and physical designs are then derived from.

RECALL

COVER THE ANSWERS

1. In the hospital model, why is TREATMENT an associative entity?
2. A guest makes many reservations; each reservation is for one room. What is the degree and cardinality of "makes"?
3. Give one reason to build a conceptual model before the database.

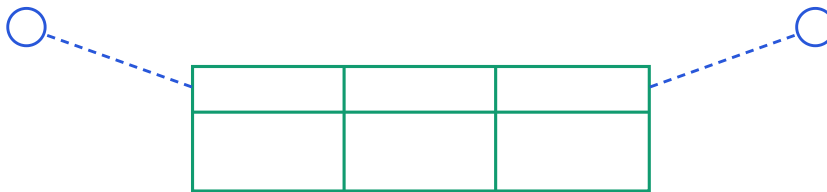
ANSWERS

1. Because the doctor-patient relationship is many-to-many and carries its own attributes (date, diagnosis). 2. Binary, one-to-many. 3. It catches design errors early and gives a shared, DBMS-independent blueprint.

MODULE THREE

Logical Design

The diagram becomes tables. This module gives you the vocabulary of the relational model, the keys and integrity rules that hold it together, and the exact procedure for turning any ER or EER diagram into a correct set of relations.



The Relational Model

A relation is a table, and that is nearly the whole idea. The marks are in the precise vocabulary, the several kinds of key, and the two integrity rules, all of which the examiner asks for by their exact names.

By the end of this unit you can: define relation, tuple, attribute, domain, degree, cardinality, and schema; distinguish super, candidate, primary, alternate, and foreign keys; state the entity and referential integrity rules; and list the properties of a well-formed relation.

1.1 The vocabulary

● MUST MEMORISE

RELATIONAL TERMS

TERM	MEANING	IN A TABLE
Relation	a table of related data	the whole table
Tuple	one record	a row
Attribute	a named property	a column
Domain	the set of allowed values for an attribute	a column's data type and range
Degree	number of attributes	number of columns
Cardinality	number of tuples	number of rows
Schema	the structure: relation name plus its attributes	the heading row, once

We write a schema as STAFF(staffNo, fName, lName, branchNo) with the primary key underlined.

AS PRINTED

25/26 Q4A, 24/25 Q1A, 15/16 Q4A

- a. Describe the relational Data model. (2 ½ mks)

DTS224 25/26, Q4a, as printed.

- (a) Define the following terms:

- (i) Relational model (1 mark)
- (ii) Relation (1 mark)
- (iii) Relational algebra (1 mark)
- (iv) Data model (1 mark)

DTS224 24/25, Q1a, as printed.

4. (a) Discuss each of the following concepts in the context of the relational data model
- (i) attribute and tuple
 - (ii) domain
 - (iii) degree and cardinality. (5marks)

DTS224 15/16, Q4a, as printed.

Q. Describe the relational data model; and define relation, relational algebra, data model, attribute and tuple, domain, degree and cardinality.

The **relational data model** represents all data as a collection of **relations** (two-dimensional tables of rows and columns), where relationships between data are expressed by common values, not by pointers. A **relation** is a table with named columns and un-ordered rows. **Relational algebra** is the set of operations (select, project, join, and so on) used to query relations, covered in Module Five. A **data model** is a collection of concepts for describing data, its relationships, and its constraints. An **attribute** is a column; a **tuple** is a row. A **domain** is the set of permitted values of an attribute. **Degree** is the number of attributes; **cardinality** is the number of tuples.

1.2 Keys

● MUST MEMORISE

THE FIVE KEYS

- ▶ **Super key:** any set of attributes that uniquely identifies a tuple (may include extra, unnecessary attributes).
- ▶ **Candidate key:** a minimal super key, one with no unnecessary attribute removed. A relation may have several.
- ▶ **Primary key (PK):** the one candidate key chosen to identify tuples. Cannot be null.
- ▶ **Alternate key:** a candidate key not chosen as the primary key.
- ▶ **Foreign key (FK):** an attribute in one relation whose values must match the primary key of another (or be null); this is what links tables.

From here on, schema listings write **PK** for the primary key and **FK** for a foreign key, so a relation such as ROOM(roomNo, hotelNo, price) is annotated PK roomNo, hotelNo · FK hotelNo -> HOTEL .

AS PRINTED

24/25 Q5A, 23/24 Q1A, 25/26 Q6A

(a) Define the following terms:

- | | |
|--------------------|----------|
| (i) Primary key | (1 mark) |
| (ii) Candidate key | (1 mark) |
| (iii) Foreign key | (1 mark) |
| (iv) Super key | (1 mark) |

DTS224 24/25, Q5a, as printed.

(a) Define the following relational concepts:

- | | |
|--------------------|----------|
| (i) Superkey | (1 mark) |
| (ii) Primary key | (1 mark) |
| (iii) Foreign key | (1 mark) |
| (iv) Candidate key | (1 mark) |

DTS224 23/24, Q1a, as printed.

a. Explain the following relational terms briefly: (3 mks)

- (i) Schema
- (ii) Alternate key
- (iii) Tuple

DTS224 25/26, Q6a, as printed.

WORKED

PQ 24/25 Q5A

· PQ 23/24 Q1A

· PQ 25/26 Q6A

Q. Define: primary key, candidate key, foreign key, super key, alternate key, schema, tuple.

All are in the boxes above. Compact model answers: a **super key** uniquely identifies a row but may carry spare attributes; a **candidate key** is a super key with nothing spare; the **primary key** is the chosen candidate key and is never null; an **alternate key** is a candidate key not chosen; a **foreign key** matches a primary key elsewhere and links the two relations; a **schema** is a relation's name and its attributes; a **tuple** is one row.

AS PRINTED

15/16 Q2A

2(a) What is the relationship between Candidate key and Foreign keys (5marks)
DTS224 15/16, Q2a, as printed.

WORKED

PQ 15/16 Q2A

Q. What is the relationship between a candidate key and a foreign key?

A candidate key uniquely identifies tuples *within* its own relation. A foreign key *refers* from one relation to a candidate key (in practice the primary key) of another. So a foreign key's whole purpose is to point at a candidate key elsewhere: the candidate key is the target, the foreign key is the pointer, and referential integrity is the rule that the pointer must always find its target.

1.3 The two integrity rules

● MUST MEMORISE**ENTITY AND REFERENTIAL INTEGRITY**

Entity integrity: no part of a primary key may be null. (If it were, the row could not be identified.)

Referential integrity: every foreign-key value must either match an existing primary-key value in the referenced relation, or be wholly null. (No row may point at a row that does not exist.)

- (b) Hotel (hotelNo, hotelName, City)
 Room (roomNo, hotelNo, type, price)
 Booking (hotelNo, guestNo, dateFrom, dateTo, roomNo)
 Guest (questNo, questName, questAddress)

Using the schema above to answer the following questions:

- (i) Identify the foreign keys in this schema (2marks)
 (ii) Explain how the entity integrity and referential integrity rules apply to these relations (4marks)
 (iii) Produce some sample tables for these relations that observe the relational integrity. (4marks)

DTS224 15/16, Q4b, as printed.

WORKED

PQ 15/16 Q4B

Q. Given Hotel(hotelNo, hotelName, city), Room(roomNo, hotelNo, type, price), Booking(hotelNo, guestNo, dateFrom, dateTo, roomNo), Guest(guestNo, questName, questAddress): (i) identify the foreign keys; (ii) explain how entity and referential integrity apply; (iii) give sample rows that observe integrity.

(i) Foreign keys

Room.hotelNo references Hotel.hotelNo. Booking.hotelNo references Hotel.hotelNo, Booking.guestNo references Guest.guestNo, and Booking.(roomNo, hotelNo) references Room.(roomNo, hotelNo).

(ii) The rules here

Entity integrity: no Hotel may have a null hotelNo; no Booking may have a null in any of hotelNo, guestNo, or dateFrom (its composite primary key). **Referential integrity:** a Booking's hotelNo must be a hotel that exists, its guestNo a guest that exists, and its (roomNo, hotelNo) a room that exists.

(iii) Sample rows that observe integrity

HOTEL

hotelNo	hotelName	city
H1	Grand	London
H2	Bayview	Glasgow

GUEST, ROOM, BOOKING (EACH FK MATCHES A ROW ABOVE)

Guest:	G1	John Kay	London
Room:	R1	H1	single 60.00
Booking:	H1	G1	2026-01-10 2026-01-12 R1

The Booking is legal because H1 is a real hotel, G1 a real guest, and (R1, H1) a real room. Change any one to a value not shown above and referential integrity is broken.

1.4 What makes a relation well-formed

AS PRINTED

24/25 Q2B

- (b) Relations are fundamental to relational database systems, as they represent data using tuples (rows) and attributes (columns). List six (6) properties or characteristics that define a well-structured (good) relation in a relational database. (3 marks)

DTS224 24/25, Q2b, as printed.

WORKED

PQ 24/25 Q2B SIX PROPERTIES OF A GOOD RELATION

Q. List six properties that define a well-structured relation.

1. Each **cell holds a single value** (the relation is atomic, no repeating groups).
2. Each **attribute has a distinct name**.
3. All values in an **attribute come from the same domain**.
4. The **order of attributes is irrelevant**.
5. The **order of tuples is irrelevant**.
6. Every **tuple is unique** (no two identical rows; guaranteed by the primary key).

RECALL

COVER THE ANSWERS

1. Degree vs cardinality of a relation?
2. Difference between a candidate key and a super key?
3. What is an alternate key?
4. State the entity integrity rule.
5. A foreign key value may be one of two things. Which?

ANSWERS

1. Degree = number of columns; cardinality = number of rows. 2. A candidate key is a super key with no spare attributes. 3. A candidate key not chosen as primary. 4. No part of a primary key may be null. 5. Matching an existing primary key, or wholly null.

MODULE THREE • UNIT 2

From Conceptual to Logical Design

The mechanical step every year: an ER or EER diagram is handed to you and you turn it into relations. There is a fixed set of rules; learn them and the question becomes bookkeeping.

By the end of this unit you can: map entities, every kind of relationship, multivalued attributes, and supertype/subtype structures into relations with correct primary and foreign keys.

2.1 The mapping rules

● MUST MEMORISE

ER TO RELATIONS

- ▶ **Each strong entity** becomes a relation; its identifier becomes the primary key.
- ▶ **A one-to-many relationship** puts the primary key of the "one" side into the "many" side as a foreign key.
- ▶ **A many-to-many relationship** becomes its own relation, whose primary key is the two entities' keys together, plus any relationship attributes.
- ▶ **A one-to-one relationship** puts the key of one side into the other as a foreign key (usually on the optional side).
- ▶ **A multivalued attribute** becomes its own relation, keyed by the entity's key plus the attribute.
- ▶ **A weak entity** becomes a relation whose primary key includes the identifying owner's key.
- ▶ **A supertype/subtype** maps either to one relation with all attributes (plus a type flag), or to a relation per subtype sharing the supertype's key. The common choice: one relation for the supertype and one per subtype, each subtype keyed by the supertype's key.

2.2 Worked: employee, certificate, course

AS PRINTED

19/20 Q1C

1(c) Transform the diagram below to a relational schema that shows referential integrity constraints.

[6.5Marks]

DTS224 19/20, Q1c, as printed.

EXAM QUESTION

PQ 19/20 Q1C

Transform to a relational schema showing referential integrity: EMPLOYEE (Employee ID, Employee Name, Birth Date) earns, through the associative entity CERTIFICATE (Certificate Number, Date Completed), a COURSE (Course ID, Course Title, Topic). An employee earns many certificates; each certificate is for one course.

ANSWER

Two strong entities become relations. CERTIFICATE is an associative entity between EMPLOYEE and COURSE, so it becomes a relation carrying both foreign keys.

```
EMPLOYEE(EmployeeID, EmployeeName, BirthDate)
    PK EmployeeID

COURSE(CourseID, CourseTitle, Topic)
    PK CourseID

CERTIFICATE(CertificateNumber, DateCompleted, EmployeeID, CourseID)
    PK CertificateNumber
    FK EmployeeID -> EMPLOYEE
    FK CourseID   -> COURSE
```

Referential integrity: every EmployeeID and CourseID in CERTIFICATE must match a real EMPLOYEE and COURSE.

2.3 Worked: the bank card (an EER with subtypes)

AS PRINTED

24/25 Q4B

- (b) Transform the conceptual design for the bank card in Figure 2 into a logical design.
(6marks)

DTS224 24/25, Q4b, as printed.

EXAM QUESTION

PQ 24/25 Q4B

Transform the bank-card conceptual design to a logical design. CUSTOMER (Customer ID, Name, Address) holds a CARD ACCOUNT (Account ID, Exp Date, Card Type). CARD ACCOUNT is a disjoint supertype over DEBIT CARD (Bank No) and CREDIT CARD (Cur Bal). A CREDIT CARD is billed through CHARGES (Charge Date, Charge Time, Amount) to a MERCHANT (Merch ID, Merch Addr).

ANSWER

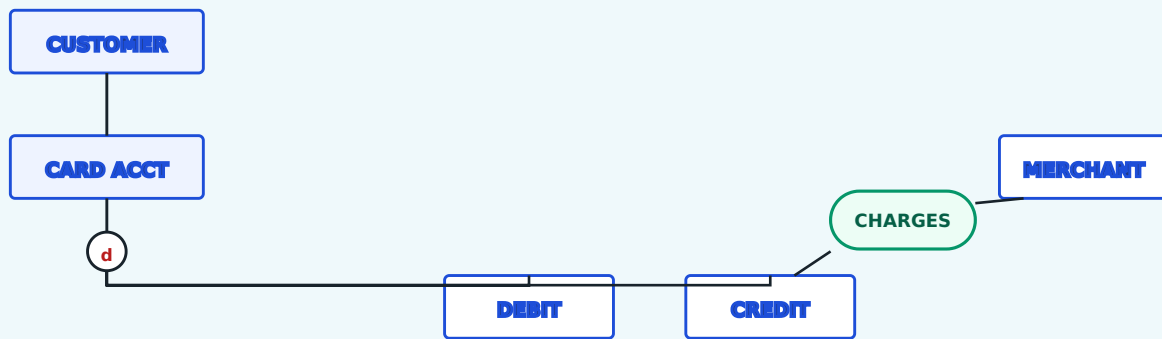


Figure 3.1. The given bank-card EER, redrawn: CARD ACCOUNT is a disjoint supertype over DEBIT and CREDIT; CHARGES links CREDIT CARD to MERCHANT.

```

CUSTOMER(CustomerID, Name, Address)          PK CustomerID
CARD_ACCOUNT(AccountID, ExpDate, CardType, CustomerID)
    PK AccountID    FK CustomerID -> CUSTOMER
DEBIT_CARD(AccountID, BankNo)                PK AccountID    FK -> CARD_ACCOUNT
CREDIT_CARD(AccountID, CurBal)                PK AccountID    FK -> CARD_ACCOUNT
MERCHANT(MerchID, MerchAddr)                 PK MerchID
CHARGES(AccountID, MerchID, ChargeDate, ChargeTime, Amount)
    PK (AccountID, MerchID, ChargeDate)
    FK AccountID -> CREDIT_CARD    FK MerchID -> MERCHANT
  
```

Each subtype becomes a relation sharing the supertype's key (AccountID). CHARGES, a many-to-many with attributes, becomes its own relation.

2.4 Worked: the university dining service

AS PRINTED

25/26 Q2D

d. Transform the EER for a University Dining Service in Figure 2 into a set of relational schemas. Show the prim (Use the short text notation). (8 mks)

QUESTION THREE

(17 ½ mks)

DTS224 25/26, Q2d, as printed.

EXAM QUESTION

PQ 25/26 Q2D

Transform the University Dining Service EER to relational schemas: MENU (Menu ID, Description, Type) contains many DISH (Dish ID, Name, Prep Time, and a multivalued Ingredient) and is served at EVENT (Event ID, Date, Location, Time). STAFF (Emp ID, Name, Salary, multivalued Skill) work a WORK SCHEDULE (Start, End, Position) at events, and a staff member supervises other staff.

ANSWER

Entities become relations; the multivalued Ingredient and Skill each become their own relation; the many-to-many "contains" and "served at" become relations; "supervises" is a unary one-to-many, so a supervisor foreign key goes into STAFF.

```
MENU(MenuID, Description, Type)      PK MenuID
DISH(DishID, Name, PrepTime)        PK DishID
DISH_INGREDIENT(DishID, Ingredient)  PK both, FK DishID -> DISH
MENU_DISH(MenuID, DishID)           PK both (contains, M:N)
EVENT(EventID, Date, Location, Time) PK EventID
MENU_EVENT(MenuID, EventID)         PK both (served at, M:N)
STAFF(EmpID, Name, Salary, SupervisorID) PK EmpID, FK SupervisorID -> STAFF
STAFF_SKILL(EmpID, Skill)           PK both, FK EmpID -> STAFF
WORK_SCHEDULE(EmpID, EventID, Start, End, Position)
    PK (EmpID, EventID), FK EmpID -> STAFF, FK EventID -> EVENT
```

2.5 Worked: vacation property (subtypes) and more

AS PRINTED

23/24 Q4C, 23/24 Q4B

(c) Giving an EER diagram for Vacation Property Rentals in Figure 2 Transform the EER diagram to a set of relations and develop its relational/logical schema. (6½marks)

Question Five

DTS224 23/24, Q4c, as printed.

(b) Transform the conceptual design for patient and treatment in Figure1 into a logical design

(6marks)

DTS224 23/24, Q4b, as printed.

Vacation Property Rentals. RENTER signs a RENTAL AGREEMENT which books a PROPERTY; PROPERTY is a disjoint supertype over BEACH (Blocks to Beach) and MOUNTAINS (Activity), and has a unary relationship on itself.

```

RENTER(RenterID, FirstName, MI, LastName, Address, Phone)    PK RenterID
RENTAL_AGREEMENT(AgreementID, BeginDate, EndDate, RenterID, PropertyID)
    PK AgreementID  FK RenterID -> RENTER  FK PropertyID -> PROPERTY
PROPERTY(PropertyID, Street, City, State, Zip, Rooms, BaseRate, PropertyType, NearbyID)
    PK PropertyID   FK NearbyID -> PROPERTY (the recursive link)
BEACH_PROPERTY(PropertyID, BlocksToBeach)                    PK PropertyID  FK -> PROPERTY
MOUNTAINS_PROPERTY(PropertyID, Activity)                     PK PropertyID  FK -> PROPERTY

```

Patient and treatment (23/24 Q4b). PHYSICIAN treats PATIENT through the associative PATIENT TREATMENT, which references a TREATMENT catalogue.

```

PHYSICIAN(PhysicianID, PhysicianName)                        PK PhysicianID
PATIENT(PatientID, PatientName)                              PK PatientID
TREATMENT(TreatmentCode, Description)                        PK TreatmentCode
PATIENT_TREATMENT(PatientID, PhysicianID, TreatmentCode, PTDate, PTTime, PTResults)
    PK (PatientID, PhysicianID, PTDate)
    FK PatientID -> PATIENT  FK PhysicianID -> PHYSICIAN  FK TreatmentCode -> TREATMENT

```

1. Convert the following ER diagram into a set of tables (logical Design).
DTS224 20/21, mid-semester Q1, as printed.

1(d) Convert the following ER diagram into a set of tables. (10marks)
DTS224 15/16, Q1d, as printed.

The general recipe applies to the Project/Organisation/Employee diagrams these questions give. Work top-down through the mapping rules: every entity a relation, every one-to-many a foreign key on the many side, every many-to-many a link relation, every subtype a relation sharing the supertype key. For example, an ORGANISATION executes many PROJECTs and employs many EMPLOYEEs, with a WorksFor link and a payment history:

```

ORGANISATION(OrgID, Name, Address, City, State)    PK OrgID
PROJECT(ProjectID, Name, Purpose, StartDate, EndDate, OrgID)
    PK ProjectID  FK OrgID -> ORGANISATION (Executes, 1:M)
EMPLOYEE(SSN, Name, DOB, Address, City)            PK SSN
WORKS_FOR(SSN, OrgID, Cost)                         PK (SSN, OrgID)
    FK SSN -> EMPLOYEE  FK OrgID -> ORGANISATION (WorksFor, M:N)

```

- 1. Where does the foreign key go in a one-to-many relationship?**
 - 2. How is a many-to-many relationship mapped?**
 - 3. How is a multivalued attribute mapped?**
 - 4. How is a supertype with two subtypes usually mapped?**
 - 5. A unary one-to-many (staff supervises staff): how is it mapped?**
-

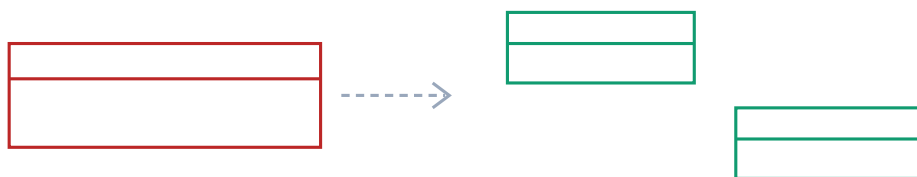
ANSWERS

1. Into the "many" side, as a foreign key to the "one" side. 2. As its own relation keyed by both entities' keys plus any relationship attributes. 3. As its own relation keyed by the entity key plus the attribute. 4. One relation for the supertype and one per subtype, each subtype keyed by the supertype's key. 5. A supervisor foreign key added to the same relation, referencing its own primary key.

MODULE FOUR

Functional Dependencies and Normalization

One table crammed with everything hides errors that only surface when you insert, update, or delete. Normalization is the disciplined splitting of that table into well-behaved relations, and it is the single most reliable source of marks on the paper.



Normalization

A procedure with a fixed set of steps. You are given a table and its functional dependencies; you produce a set of relations in third normal form, or Boyce-Codd normal form, showing your working. Master one worked example and the rest are the same moves.

By the end of this unit you can: define full, partial, and transitive functional dependencies; draw a functional-dependency diagram; and normalize any given table to 3NF or BCNF, stating every step and the resulting relations.

1.1 Why: the update anomalies

Put everything in one table and three faults appear. **Insertion anomaly:** you cannot record a new property until someone inspects it, because inspection data shares the row. **Update anomaly:** a staff member's name, repeated on every inspection row, must be changed in every one or the copies disagree. **Deletion anomaly:** deleting the last inspection of a property loses the property's address too. Normalization removes all three by giving each fact one home.

1.2 Functional dependencies

An attribute B is **functionally dependent** on A, written $A \rightarrow B$, if each value of A has exactly one value of B. A is the **determinant**. Three kinds are examined by name:

● MUST MEMORISE

THREE FUNCTIONAL DEPENDENCIES

- ▶ **Full functional dependency:** B depends on the *whole* of a composite determinant A, not on any part of it. (propertyNo, iDate) $\rightarrow$ iTime is full: you need both.
- ▶ **Partial functional dependency:** B depends on only *part* of a composite key. With key (propertyNo, iDate), propertyNo $\rightarrow$ pAddress is partial: pAddress needs only propertyNo.
- ▶ **Transitive functional dependency:** B depends on the key only *through* another non-key attribute. (propertyNo, iDate) $\rightarrow$ staffNo $\rightarrow$ sName makes sName transitively dependent.

- (b) Explain the following Terms with example
- (i) Full Functional Dependency. (1½marks)
 - (ii) Partial Functional Dependency. (1½marks)
 - (iii) Transitive functional Dependency. (1½marks)

DTS224 24/25, Q3b, as printed.

- (b) Explain the following Terms with example
- (i) Full Functional Dependency (1½marks)
 - (ii) Partial Functional Dependency (1½marks)
 - (iii) Transitive functional Dependency (1½marks)

DTS224 23/24, Q3b, as printed.

- b. Use Figure 1 to explain the concept of determinant in the context of normalization. Show all possible determinants, the corresponding attributes and the type of functional dependency involved. (2½ mks)

DTS224 25/26, Q1b, as printed.

WORKED

PQ 24/25 Q3B · PQ 23/24 Q3B · PQ 25/26 Q1B (DETERMINANT)

Q. Explain full, partial, and transitive functional dependency with examples; and explain a determinant.

A **determinant** is the attribute (or set of attributes) on the left of a functional dependency, the thing that determines another. In fd propertyNo → pAddress, propertyNo is the determinant. The three dependency types are in the box above; the one-line hooks are: **full** needs the whole composite key, **partial** needs only part of it, **transitive** reaches the key only through a non-key attribute. (For 25/26's EMPLOYEE2, the determinants are EmpID → Name, DeptName, Salary, and the composite (EmpID, CourseTitle) → DateCompleted, the only full dependency.)

1.3 The normal forms

● MUST MEMORISE

1NF TO BCNF

- ▶ **1NF (first normal form):** every cell holds a single value; no repeating groups.
- ▶ **2NF (second normal form):** in 1NF, and no **partial** dependency (every non-key attribute depends on the whole key).
- ▶ **3NF (third normal form):** in 2NF, and no **transitive** dependency (no non-key attribute depends on another non-key attribute).
- ▶ **BCNF (Boyce-Codd normal form):** in 3NF, and **every determinant is a candidate key**. The strictest of the four.

The pattern is always the same: 2NF removes partial dependencies, 3NF removes transitive dependencies, BCNF removes any determinant that is not a candidate key.

|| (a) What is Normalization? (1½marks)
DTS224 24/25, Q3a, as printed.

|| (a) What is Normalization (1½marks)
DTS224 23/24, Q3a, as printed.

Q. What is normalization? Normalization is a step-by-step process that organises the attributes of a database into well-structured relations by removing redundancy and the update, insertion, and deletion anomalies it causes, using functional dependencies to decide how to split the tables.

1.4 Worked in full: StaffPropertyInspection to 3NF

AS PRINTED

24/25 Q3C

(c) Given the StaffPropertyInspection Relation in Table 1 and the information below answer the

following questions:

(i) Draw the functional dependency diagram for the relation. (2marks)

(ii) Normalize the relation to 3rd Normal Form.

The keys are:

(propertyNo, iDate) (PK)

(staffNo, iDate, iTime) (CK)

(carReg, iDate, iTime) (CK)

(9½marks)

Table 1: StaffPropertyInspection Relation

propertyNo	pAddress	iDate	iTime	comments	staffNo	carReg
PG4		18-Oct-03	10.00	Need to replace crockery.	SG37	M231 JGR
PG4	6 Lawrence	22-Apr-04	09.00	In good order.	SG14	M533
HDR						
PG4	St, Glasgow.	1-Oct-04	12.00	Damp rot in bathroom.	SG14	N721
HFR						
PG16	5 Novar Dr,	22-Apr-04	13.00	Replace living room carpet.	SG14	
M533 HDR						
PG16	Glasgow.	24-Oct-04	14.00	Good condition.	SG37	N721
HFR						

Functional dependencies:

fd1 propertyNo, iDate → iTime, comments, staffNo, sName, carReg (Primary key)

fd2 propertyNo → pAddress (Partial dependency)

fd3 staffNo → sName (Transitive dependency)

fd4 staffNo, iDate → carReg (Transitive dependency)

fd5 carReg, iDate, iTime → propertyNo, pAddress, comments, staffNo, sName (Candidate key)

fd6 staffNo, iDate, iTime → propertyNo, pAddress, comments (Candidate key)

Question Four

DTS224 24/25, Q3c, as printed.

EXAM QUESTION

PQ 25/26 Q6 · PQ 24/25 Q3C

Given StaffPropertyInspection(propertyNo, pAddress, iDate, iTime, comments, staffNo, carReg, sName) with primary key (propertyNo, iDate): draw the FD diagram, explain transitive dependency, and normalize to 3NF.

FDs: fd1 (propertyNo, iDate) → iTime, comments, staffNo, carReg, sName; fd2 propertyNo → pAddress; fd3 staffNo → sName; fd4 (staffNo, iDate) → carReg.

ANSWER

FUNCTIONAL-DEPENDENCY DIAGRAM

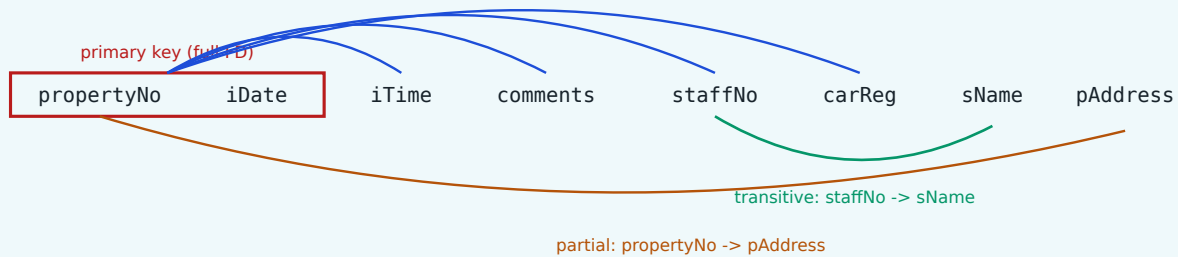


Figure 4.1. FD diagram. Blue arcs are the full dependency on the whole key; the amber arc is the partial dependency (pAddress on propertyNo alone); the green arc is the transitive dependency (sName on staffNo).

Transitive dependency here: (propertyNo, iDate) determines staffNo, and staffNo determines sName, so sName depends on the key only through staffNo. That is what 3NF removes.

STEP TO 2NF (REMOVE THE PARTIAL DEPENDENCY)

pAddress depends on propertyNo alone (part of the key), so lift it out:

```
Property(propertyNo, pAddress)           PK propertyNo
PropertyInspection(propertyNo, iDate, iTime, comments, staffNo, carReg, sName)
    PK (propertyNo, iDate)  FK propertyNo -> Property
```

STEP TO 3NF (REMOVE THE TRANSITIVE DEPENDENCY)

sName depends on non-key staffNo, so lift staffNo → sName into its own relation:

```
Property(propertyNo, pAddress)           PK propertyNo
Staff(staffNo, sName)                   PK staffNo
PropertyInspection(propertyNo, iDate, iTime, comments, staffNo, carReg)
    PK (propertyNo, iDate)  FK propertyNo -> Property  FK staffNo -> Staff
```

Three relations, each fact in one place. carReg stays with the inspection because it is identified by that inspection (propertyNo, iDate). This is the final 3NF answer.

1.5 Worked in full: Patient_Appointment to BCNF

AS PRINTED

20/21 Q2B, 15/16 Q3

(b) Describe and illustrate the process of normalizing the table below to Boyce-Codd Normal form,

giving the following information

[12 marks]

Candidate keys:

PNo, ADate (Primary key)

ADate, Atime, SurgeonNo

StaffNo, ADate, Atime

Patient_Appointment Table

StaffNo	DentistName	PNo	PName	ADate	ATime	SurgeonNo
S011	Tony Smith	P100	James White	12-Sep-08	10.00	S15
S011	Tony Smith	P105	Jill Bell	12-Sep-08	12.00	S15
S024	Hellen Chris	P108	Tom Fred	12-Sep-08	10.00	S10
S024	Hellen Chris	P108	Tom Fred	14-Sep-98	14.00	S10
S032	Rose Kelvin	P105	Jill Bell	14-Sep-98	16.00	S15
S032	Rose Kelvin	P110	John Stone	15-Sep-08	18.00	S13

Functional dependency:

PNo, ADate, → StaffNo, ATime, DentistName, SurgeonNo

PNo → PName

StaffNo → DentistName

StaffNo, ADate → SurgeonNo

ADate, ATime, SurgeonNo → PNo, PName, StaffNo, DentistName

StaffNo, ADate, ATime → PNo, PName

QUESTION THREE

DTS224 20/21, Q2b, as printed.

3. Given the Patient_Appointment Table below

- (i) Determine the candidate keys for the table (3marks)
- (ii) Normalizing the table below to Boyce-Codd Normal form and Show the final decomposed relations with the data. (12marks)

Table 1: Patient_Appointment Table

Functional dependency

PNo, ADate, → StaffNo, ATime, DentistName, SurgeonNo (pk)

PNo → PName (pd)

StaffNo → DentistName Td

StaffNo, ADate → SurgeonNo

Date, ATime, SurgeonNo → PNo, PName, StaffNo, DentistName (ck)

StaffNo, ADate, ATime → PNo, PName (ck)

[note: the printed page leaves the body of Table 1 blank. The caption and the functional dependencies are printed, but no data rows appear between them.]

DTS224 15/16, Q3, as printed.

Patient_Appointment(StaffNo, DentistName, PNo, PName, ADate, ATime, SurgeonNo). Determine the candidate keys and normalize to Boyce-Codd normal form. FDs: (PNo, ADate) → StaffNo, ATime, DentistName, SurgeonNo; PNo → PName; StaffNo → DentistName; (StaffNo, ADate) → SurgeonNo; (ADate, ATime, SurgeonNo) → PNo, PName, StaffNo, DentistName; (StaffNo, ADate, ATime) → PNo, PName.

ANSWER

CANDIDATE KEYS

Three sets each determine every attribute: **(PNo, ADate)** [chosen as primary key], **(ADate, ATime, SurgeonNo)**, and **(StaffNo, ADate, ATime)**.

TO 2NF (REMOVE PARTIAL DEPENDENCY)

PNo → PName depends on part of the key (PNo, ADate), so lift it out:

```
Patient(PNo, PName)                PK PNo
Appointment(PNo, ADate, StaffNo, ATime, DentistName, SurgeonNo)
    PK (PNo, ADate)    FK PNo -> Patient
```

TO 3NF (REMOVE TRANSITIVE DEPENDENCIES)

StaffNo → DentistName (non-key to non-key) and (StaffNo, ADate) → SurgeonNo both violate 3NF. Lift each out:

```
Patient(PNo, PName)                PK PNo
Staff(StaffNo, DentistName)         PK StaffNo
SurgeonAssign(StaffNo, ADate, SurgeonNo) PK (StaffNo, ADate)
Appointment(PNo, ADate, StaffNo, ATime)
    PK (PNo, ADate)    FK PNo -> Patient    FK StaffNo -> Staff
```

TO BCNF (EVERY DETERMINANT A CANDIDATE KEY)

Check each relation's determinants. Patient (PNo), Staff (StaffNo), SurgeonAssign (StaffNo, ADate) are each keyed by their determinant. In Appointment, both (PNo, ADate) and (StaffNo, ADate, ATime) are candidate keys, and they are the only determinants, so every determinant is a candidate key. The four relations above are already in BCNF.

1.6 Worked: Customer_renter, and a dependency-only question

AS PRINTED

23/24 Q3C

(c) Given the Customer_renter Relation and the information below answer the following question

- Draw the functional dependency diagram for the relation (2marks)
- Normalize the relation to 3rd Normal forms. The primary key is Customer_No, Property_No.

(9½marks)

Customer_renter Relation

Customer_No	CName	Property_No	Paddress	RentStart	RentFinish	Rent	Owner_No	OName
CR76	John	PG4	6, Lawrence	1-Jul-04	31-Aug-06	350	CO40	Tina
	Kay		St. Ikoyi					Murphy
		PG16		1-Sep-06	1-Sep-08	450	CO93	
			5, Novar St,					Tony Shaw
			Oshodi					
CR56	Fred	PG4	6, Lawrence	1-Sep-02	10-Jun-04	350	CO40	Tina
	Wey	PG36	St. Ikoyi	10-Oct-04	1-Dec-06	375	CO93	
		PG16	2, Fred St.	1-Jan-06	10-Aug-06	450	CO93	Tony
			Apapa					Tony Shaw
			5, Novar St,					
			Oshodi					

Functional dependencies:

fd1: Customer_No, Property_No → RentStart, RentFinish (Pk)

fd2: Customer_No → CName (Pd)

fd3: Property_No → Paddress, Rent, Owner_No, OName (Pd)

fd4: Owner_No → OName (Td)

fd5: Customer_No, RentStart → Property_No, Paddress, Rent, Owner_No, OName (Ck)

fd6: Property_No, RentStart → Customer_No, CName, RentFinish (Ck)

Question Four

DTS224 23/24, Q3c, as printed.

WORKED
PQ 23/24 Q3C CUSTOMER_RENTER TO 3NF

Customer_renter(Customer_No, CName, Property_No, Paddress, RentStart, RentFinish, Rent, Owner_No, OName), PK (Customer_No, Property_No). FDs: Customer_No $\rightarrow$ CName (partial); Property_No $\rightarrow$ Paddress, Rent, Owner_No, OName (partial); Owner_No $\rightarrow$ OName (transitive).

```
Customer(Customer_No, CName)           PK Customer_No
Owner(Owner_No, OName)                 PK Owner_No
Property(Property_No, Paddress, Rent, Owner_No)
    PK Property_No  FK Owner_No -> Owner
Rental(Customer_No, Property_No, RentStart, RentFinish)
    PK (Customer_No, Property_No)
    FK Customer_No -> Customer, FK Property_No -> Property
```

2NF lifts the two partial dependencies (CName on Customer_No, and the property attributes on Property_No); 3NF lifts the transitive Owner_No $\rightarrow$ OName into Owner.

AS PRINTED
21/22 Q2C

(c) Suppose you are given a relation R with four attributes ABCD. And functional dependency $C \rightarrow D$, $C \rightarrow A$, $B \rightarrow CF$, assuming those are the only dependencies that hold for R, do the following:

- Identify the candidate key(s) for R.
- Identify the best normal form that R satisfies (1NF, 2NF, 3NF, or BCNF).
- If R is not in BCNF, decompose it into a set of BCNF relations that preserve the dependencies.

(4 Marks)

QUESTION THREE

DTS224 21/22, Q2c, as printed.

WORKED
PQ 21/22 Q2C R(A,B,C,D) TO BCNF

Q. R has attributes ABCD with FDs $C \rightarrow D$, $C \rightarrow A$, $B \rightarrow C$. Find the candidate key, the best normal form R is in, and decompose to BCNF.

Candidate key. B determines C (given), C determines A and D, so B determines A, C, D, hence B determines everything: **B is the sole candidate key.**

Best normal form. Non-key attributes are A, C, D. $B \rightarrow C \rightarrow A$ and $B \rightarrow C \rightarrow D$ are transitive dependencies, so R is not in 3NF. There is no partial dependency (the key B is a single attribute), so R is in **2NF but not 3NF**.

Decompose to BCNF. The offending determinant is C (not a candidate key). Split it off:

```
R1(C, A, D)   PK C       (from C -> A, C -> D)
R2(B, C)      PK B       FK C -> R1      (from B -> C)
```

Now every determinant (C in R1, B in R2) is a candidate key of its relation, and the join of R1 and R2 on C reconstructs R, so the decomposition is lossless and dependency-preserving.

3. Examine the Patient Medication Form for the Wellmeadows Hospital case study shown in the

Figure below

- i. Identify the functional dependencies represented by the attributes shown in the form
- ii. State any assumptions you make about the data and the attributes shown in this form
- iii. Describe and illustrate the process of normalizing the attributes to produce a set of well-designed 3NF relations. (Use PatientNo, DrugNumber as primary key).

ALPHA SEMESTER EXAMINATION (3 hours, attempt any four questions)

QUESTION ONE

DTS224 20/21, mid-semester Q3, as printed.

WORKED

PQ 20/21 MID Q3

WELLMEADOWS ·

TEXT M4U1

Wellmeadows patient-medication form (20/21 mid). Same method: from the form, the FDs are patientNo → patient details; drugNo → drug name, description; and (patientNo, drugNo) → the prescription details. Normalizing to 3NF gives Patient, Drug, and Prescription relations, with the prescription keyed by (patientNo, drugNo).

The course text's normalization exercises (Pine Valley INVOICE, DreamHome Customer_Rental) follow the identical UNF → 1NF → 2NF → 3NF path: remove repeating groups, then partial dependencies, then transitive dependencies, ending in CUSTOMER, PRODUCT, ORDER, ORDER_LINE (Pine Valley) or Customer, Property, Rental, Owner (DreamHome).

⚠ TRAP

PARTIAL VS TRANSITIVE

The examiner's favourite snare is labelling a dependency wrongly. A **partial** dependency is on *part of the key*; a **transitive** dependency is on a *non-key attribute*. If the determinant is a piece of the primary key, it is partial (fix in 2NF). If the determinant is an ordinary attribute, it is transitive (fix in 3NF). Read the determinant, not the dependent.

1. Which anomaly loses data when you delete a row?
 2. What does 2NF remove? 3NF? BCNF?
 3. staffNo is not part of the key; staffNo $\rightarrow$ sName. Partial or transitive?
 4. Give the BCNF condition in one line.
 5. For R(ABCD) with $B \rightarrow C \rightarrow A$, $C \rightarrow D$, what is the candidate key?
-

ANSWERS

1. The deletion anomaly. 2. 2NF removes partial dependencies; 3NF removes transitive dependencies; BCNF removes determinants that are not candidate keys. 3. Transitive (the determinant is a non-key attribute). 4. Every determinant is a candidate key. 5. B.

MODULE FIVE

Relational Theory and Query Languages

Now you ask the database questions. First in the pure notation of relational algebra, the theory the exam still tests, then in SQL, the language you actually type. Both come down to the same handful of operations.

σ π $\bowtie$ ----- SELECT
WHERE

Relational Algebra

A small set of operations that each take one or two relations and produce a new relation. Chain them and you can express any query. The exam asks you to define the operations and to write expressions, so this unit does both.

By the end of this unit you can: define the six core operations with their symbols, and write a relational-algebra expression for any query over a given schema.

1.1 The operations and their symbols

● MUST MEMORISE

SIX OPERATIONS, PLUS TWO

OPERATION	SYMBOL	DOES
Selection	σ (sigma)	picks <i>rows</i> that meet a condition: $\sigma_{\text{price} < 40}(\text{Room})$
Projection	π (pi)	picks <i>columns</i> : $\pi_{\text{name, city}}(\text{Hotel})$
Union	$\cup$	all rows in either relation (same columns), duplicates removed
Set difference	$-$	rows in the first relation but not the second
Cartesian product	$\times$	every row of one paired with every row of the other
Join	$\bowtie$ (bowtie)	a product kept only where a matching condition holds

Two more you will use: **rename** ρ (rho), which gives a relation or its columns a new name, and **division** $\div$, which finds rows associated with *all* rows of another relation ("works on all projects").

AS PRINTED

25/26 Q4B, 15/16 Q5A

| b. Explain six (6) Relational Algebra operations with examples. (6 mks)

DTS224 25/26, Q4b, as printed.

| 5. (a) Define the 3 basic relational algebra operation (4marks)

DTS224 15/16, Q5a, as printed.

WORKED

PQ 25/26 Q4B

PQ 15/16 Q5A

Q. Explain six relational algebra operations with examples (15/16 asks for the three basic ones).

The three **basic** ones the older paper wants are **selection** (σ , choose rows), **projection** (π , choose columns), and **join** ($\bowtie$, combine two relations on a match). The full six add **union** ($\cup$, combine two union-compatible relations), **set difference** ($-$, rows in one but not the other), and **Cartesian product** ($\times$, all pairings). Each takes relations and returns a relation, which is why they chain.

1.2 Worked: College, Student, Apply

AS PRINTED

24/25 Q1B

(b) Consider the following schema:

College (C_Name, State, Enrollment)

Student (S_ID, S_Name, CGPA, HighSch_Size)

Apply (S_ID, College_Name, Course, Decision)

Write the queries in relational algebra to:

(i) Find all students with a CGPA greater than 4.5. (1½marks)

(ii) Find all students with a CGPA greater than 4.5 and a high school size of less than 50. (1½marks)

(iii) Produce the name of students who applied to CST college for the course, Computer Science (1½marks)

(iv) Generate a list of the names and CGPA of students with a high school size larger than 1000

who applied to a computer science course at college with an enrollment size greater than

20000 and were rejected. (2 marks)

DTS224 24/25, Q1b, as printed.

EXAM QUESTION

PQ 24/25 Q1B

College(C_Name, State, Enrollment); Student(S_ID, S_Name, CGPA, HighSch_Size); Apply(S_ID, College_Name, Course, Decision). Write in relational algebra: (i) students with CGPA > 4.5; (ii) with CGPA > 4.5 and high-school size < 50; (iii) names of students who applied to CST for Computer Science; (iv) names and CGPA of students with high-school size > 1000 who applied for Computer Science at a college with enrollment > 20000 and were rejected.

ANSWER

(i) $\sigma_{CGPA > 4.5} (Student)$

(ii) $\sigma_{CGPA > 4.5 \wedge HighSch_Size < 50} (Student)$

(iii) $\pi_{S_Name} (Student \bowtie \sigma_{College_Name='CST' \wedge Course='Computer Science'} (Apply))$

(iv) $\pi_{S_Name, CGPA} ($
 $\sigma_{HighSch_Size > 1000} (Student)$
 $\bowtie \sigma_{Course='Computer Science' \wedge Decision='Reject'} (Apply)$
 $\bowtie \sigma_{Enrollment > 20000} (p \text{ College_Name} \leftarrow C_Name (College)))$

The joins in (iii) and (iv) match on S_ID (Student to Apply) and on the college name; the rename in (iv) lines up College's C_Name with Apply's College_Name so the join has a common attribute. $\wedge$ is "and".

1.3 Worked: Cartesian product and rename

AS PRINTED

24/25 Q1C

(c) Consider the relation Boys and Girls in Figure 1.

Boys		Girls	
Name	State	Name	State
Adamu	Kogi	Kemi	Kwara
Obi	Imo	Hadiza	Kaduna
		Chioma	Rivers

Figure 1: MN, Boys and Girls Relations.

perform the following operations on the relations:

- (i) Cartesian product on the Boys and Girls relation. (6 marks)
- (ii) Rename the output of (i) as people. (1 mark)

Question Two

DTS224 24/25, Q1c, as printed.

EXAM QUESTION

PQ 24/25 Q1C

Boys(Name, State) = {(Adamu, Kogi), (Obi, Imo)}. Girls(Name, State) = {(Kemi, Kwara), (Hadiza, Kaduna), (Chioma, Rivers)}. (i) Perform the Cartesian product; (ii) rename the result as people.

ANSWER

(i) Boys $\times$ Girls pairs every boy row with every girl row, so $2 \times 3 = 6$ rows, each with four columns (renamed to avoid the clash on Name and State):

B.Name	B.State	G.Name	G.State
Adamu	Kogi	Kemi	Kwara
Adamu	Kogi	Hadiza	Kaduna
Adamu	Kogi	Chioma	Rivers
Obi	Imo	Kemi	Kwara
Obi	Imo	Hadiza	Kaduna
Obi	Imo	Chioma	Rivers

(ii) Rename the whole result: $\text{people} \leftarrow \rho (\text{Boys} \times \text{Girls})$.

1.4 Worked: employees, departments, projects (with division)

AS PRINTED

25/26 Q4C

c. Consider the following relational schemas and write the relation algebra for the following tasks

(i-iii). (9 mks)

Emp (Fname, Lname, SSN, Dno)

Dept (DName, DNum, Mgr_SSN)

D_Loc (DNo, Location)

Project (PName, PNo, DNo, P_Loc)

Works_on (ESSN, PNo, Hours)

(i) Retrieve the names of all employees who work for the 'Headquarters' Department.

(ii) For every project located in 'Abuja', list the project number, controlling department and the manager's name.

(iii) Find the names of the employees who work on all the projects controlled by DNo = A314.

QUESTION FIVE

(17 ½ mks)

DTS224 25/26, Q4c, as printed.

EXAM QUESTION

PQ 25/26 Q4C

Emp(Fname, Lname, SSN, Dno); Dept(DName, DNum, Mgr_SSN); D_Loc(DNo, Location); Project(PName, PNo, DNo, P_Loc); Works_on(ESSN, PNo, Hours). Write: (i) names of all employees who work for the Headquarters department; (ii) for every project located in Abuja, the project number, controlling department, and manager's name; (iii) names of employees who work on all projects controlled by DNo = A3/4.

ANSWER

```
(i)  π Fname, Lname (
      Emp ⋈ Dno=DNum σ DName='Headquarters' (Dept) )

(ii) π PNo, DName, Fname, Lname (
      σ P_Loc='Abuja' (Project)
      ⋈ DNo=DNum Dept
      ⋈ Mgr_SSN=SSN Emp )

(iii) Works on ALL projects of A3/4 is a DIVISION:
      A34 ← π PNo ( σ DNo='A3/4' (Project) )
      EP  ← π ESSN, PNo ( Works_on )
      Result ← π Fname, Lname ( (EP ÷ A34) ⋈ ESSN=SSN Emp )
```

Division is the "for all" operator: $EP \div A34$ returns the ESSNs that appear with *every* PNo in A34, that is, the employees working on all of that department's projects.

(b) Using the schema in 4(c), generates SQL statement for the following queries:
(8marks)

- (i) How many hotels are there?
- (ii) List full details of all hotels in London
- (iii) List number of rooms in each hotel
- (iv) List the names and address of all guests living in London, alphabetically ordered

by

name. use pattern matching operator.

DTS224 15/16, Q5b, as printed.

WORKED

PQ 15/16 Q5B HOTEL ALGEBRA

Over Hotel(hotelNo, hotelName, city), Room(roomNo, hotelNo, type, price), Guest(guestNo, guestName, guestAddress): (i) list all hotels; (ii) single rooms below 2000 per night; (iii) names and addresses of all guests.

- (i) Hotel
- (ii) σ type='single' $\wedge$ price < 2000 (Room)
- (iii) π guestName, guestAddress (Guest)

RECALL

COVER THE ANSWERS

- Which operation chooses rows? Which chooses columns?
- What symbol is a join? A selection?
- When do you reach for division?
- Boys has 2 rows, Girls has 3. How many rows in Boys $\times$ Girls?

ANSWERS

1. Selection (σ) chooses rows; projection (π) chooses columns. 2. Join is $\bowtie$; selection is σ . 3. For "works on all / for every" queries. 4. 6 (2×3).

MODULE FIVE • UNIT 2

Structured Query Language (SQL)

The language you actually type. SQL both defines the tables (DDL) and works with the data in them (DML). The exam gives you a schema and a list of English requests; your job is to write the statement for each. Every keyword below is introduced before it is used.

By the end of this unit you can: create, alter, and drop tables with keys and constraints; insert, update, and delete rows; and write a *SELECT* for any request using *WHERE*, *ORDER BY*, *GROUP BY*, the aggregate functions, *LIKE*, *IN*, *BETWEEN*, joins, subqueries, and views.

2.1 DDL: defining the tables

● MUST MEMORISE

CREATE, ALTER, DROP

```
CREATE TABLE Branch (  
    branchNo CHAR(4) PRIMARY KEY,  
    city VARCHAR(20) NOT NULL );  
  
ALTER TABLE Staff DROP COLUMN DOB;           -- remove a column  
ALTER TABLE Staff ADD prefRooms INT;         -- add a column  
DROP TABLE Branch;                           -- remove the whole table
```

PRIMARY KEY enforces entity integrity; FOREIGN KEY ... REFERENCES enforces referential integrity; NOT NULL, UNIQUE, DEFAULT, and CHECK are the other constraints.

2.2 DML: changing the data

● MUST MEMORISE

INSERT, UPDATE, DELETE

```
INSERT INTO Emp VALUES (101, 'Azi Esther', 40, 15000);  
UPDATE Emp SET Salary = Salary * 1.10;           -- every row: 10% rise  
DELETE FROM Dept WHERE Dname = 'Toy';           -- only matching rows
```

An UPDATE or DELETE with no WHERE touches every row. Text values are quoted; numbers are not.

2.3 SELECT: asking questions

● MUST MEMORISE

THE CLAUSES, IN ORDER

```
SELECT columns (or * for all, or COUNT/SUM/AVG/MIN/MAX)  
FROM table(s)  
WHERE row condition (=, <, >, AND, OR, LIKE, IN, BETWEEN, IS NULL)  
GROUP BY column (one result row per group)  
HAVING group condition  
ORDER BY column [ASC|DESC];
```

LIKE '%London%' matches any text containing London; IN ('a', 'b') matches a list; BETWEEN 200 AND 300 is an inclusive range; COUNT(*) counts rows. A **join** combines tables on a matching column; a **subquery** is a SELECT inside another; a **view** is a stored SELECT (CREATE VIEW).

2.4 Worked: DreamHome (the compulsory 25/26 set)

AS PRINTED

25/26 Q1C

c. Consider the relational schema and write the SQL queries to answer the business questions labelled (i-vi) with standard SQL syntax.

BRANCH (branch_no, street, city, postcode)

STAFF (staff_no, fName, lName, position, sex, DOB, salary, branch_no)

PROPERTYFORRENT (property_no, street, city, postcode, type, rooms, rent, staff_no, branch_no).

- (i) List all staff members whose salary is greater than ₦100,000, ordered from highest salary to lowest salary. (1½ mks)
- (ii) Display the total number of staff working in each branch. Include the branch number and total staff count. (2 mks)
- (iii) Find the average salary of staff in each branch. Display branch number and average salary. (1½ mks)
- (iv) Display the details of all properties having more than 3 rooms and a monthly rent above ₦80,000. (2 mks)
- (v) Show the property number, property type, rent, and the full name of the staff member responsible for each property. (2 mks)
- (vi) Display all branches that currently have no staff assigned to them. (2 mks)

QUESTION TWO

(17 ½ mks)

DTS224 25/26, Q1c, as printed.

EXAM QUESTION

PQ 25/26 Q1C

BRANCH(branch_no, street, city, postcode); STAFF(staff_no, fName, lName, position, sex, DOB, salary, branch_no); PROPERTYFORRENT(property_no, street, city, postcode, type, rooms, rent, staff_no, branch_no). Write SQL: (i) staff earning over 100000, highest first; (ii) staff count per branch; (iii) average salary per branch; (iv) properties with more than 3 rooms and rent above 80000; (v) property number, type, rent, and the responsible staff member's full name; (vi) branches with no staff.

ANSWER

- (i) `SELECT * FROM STAFF WHERE salary > 100000 ORDER BY salary DESC;`
- (ii) `SELECT branch_no, COUNT(*) AS total FROM STAFF GROUP BY branch_no;`
- (iii) `SELECT branch_no, AVG(salary) AS avg_salary FROM STAFF GROUP BY branch_no;`
- (iv) `SELECT * FROM PROPERTYFORRENT WHERE rooms > 3 AND rent > 80000;`
- (v) `SELECT p.property_no, p.type, p.rent, s.fName, s.lName
FROM PROPERTYFORRENT p JOIN STAFF s ON p.staff_no = s.staff_no;`
- (vi) `SELECT branch_no FROM BRANCH
WHERE branch_no NOT IN (SELECT branch_no FROM STAFF);`

(vi) is the anti-join: the branches whose number never appears in STAFF. This is the one most people miss.

2.5 Worked: BankInfo (create, insert, alter, view, select)

AS PRINTED

24/25 Q5B

(b) Use the BankInfo relational schema to answer questions (i-vii) with Structured Query Language.

Staff(staffNo, fName, lName, position, sex, DOB, salary, branchNo)

Branch(branchNo, street, city, postcode)

PrivateOwner(ownerNo, fName, lName, address, telNo)

- (i) Write an SQL statement to create the Staff and Branch relations enforcing the referential constraints (3 marks)
- (ii) Populate the first three (3) tuples of the staff relations created in (i). (1½ marks)
- (iii) Remove the column (attribute) DOB from the staff relation. (1½ mark)
- (iv) Create a view of staff details named `adhocStaff` at branch B003 excluding salaries. (1½ marks)
- (v) Find all owners with the string 'Glasgow' in their address. (1½ marks)
- (vi) List addresses of all branch offices in London or Glasgow. (1½ marks)
- (vii) List all staff with a salary greater than 10,000. (1½ marks)
- (viii) List staff who work in branch at '163 Main St'. (1½ marks)

Question Six

DTS224 24/25, Q5b, as printed.

EXAM QUESTION

PQ 24/25 Q5B

Staff(staffNo, fName, lName, position, sex, DOB, salary, branchNo); Branch(branchNo, street, city, postcode); PrivateOwner(ownerNo, fName, lName, address, telNo). (i) create Staff and Branch enforcing referential integrity; (ii) insert three staff rows; (iii) drop the DOB column; (iv) create a view `adhocStaff` of B003 staff excluding salaries; (v) owners with Glasgow in their address; (vi) branches in London or Glasgow; (vii) staff earning over 10000; (viii) staff in the branch at 163 Main St.

ANSWER

```
(i) CREATE TABLE Branch (
    branchNo CHAR(4) PRIMARY KEY,
    street VARCHAR(30),
    city VARCHAR(20),
    postcode VARCHAR(8) );
CREATE TABLE Staff (
    staffNo CHAR(5) PRIMARY KEY,
    fName VARCHAR(20), lName VARCHAR(20),
    position VARCHAR(15), sex CHAR(1),
    DOB DATE, salary DECIMAL(8,2),
    branchNo CHAR(4),
    FOREIGN KEY (branchNo) REFERENCES Branch(branchNo) );

(ii) INSERT INTO Staff VALUES
    ('SL21','John','White','Manager','M','1945-10-01',30000,'B005');
    -- repeat for the second and third rows

(iii) ALTER TABLE Staff DROP COLUMN DOB;

(iv) CREATE VIEW adhocStaff AS
    SELECT staffNo, fName, lName, position, sex, branchNo
    FROM Staff WHERE branchNo = 'B003';

(v) SELECT * FROM PrivateOwner WHERE address LIKE '%Glasgow%';

(vi) SELECT * FROM Branch WHERE city IN ('London','Glasgow');

(vii) SELECT * FROM Staff WHERE salary > 10000;

(viii) SELECT * FROM Staff WHERE branchNo =
    (SELECT branchNo FROM Branch WHERE street = '163 Main St');
```

2.6 Worked: EmployeeInfo (DDL and DML together)

AS PRINTED

23/24 Q1B

(b) Use the EmployeeInfo relational database schema to answer questions (i-vii)

EmployeeInfo Schema

Emp (Eid, Ename, Age, Salary)

Works (Eid, Did, per_time)

Dept (Did, Dname, Budget, managerid)

(i) Write an SQL statement to create the relations in the EmployeeInfo schema (4½ marks)

(ii) Write an SQL statement to add a 40 years Azi Esther whose employee_Id is 101, and with salary of 15000. (1½marks)

(iii) Write an SQL statement to give every employee a 10 percent salary increase. (1½ marks)

(iv) Write an SQL statement to delete the Toy department. (1½mark)

(v) Write an SQL query to increment the budget of the account department to 8 million. (1½marks)

(vi) Write an SQL statement to remove the Dept relation completely from the EmployeeInfo

Database (1½marks)

(vii) Write an SQL statement to make the column Eid come first when the Emp relation is queried. (1½marks)

Question Two

DTS224 23/24, Q1b, as printed.

EXAM QUESTION

PQ 23/24 Q1B

Emp(Eid, Ename, Age, Salary); Works(Eid, Did, per_time); Dept(Did, Dname, Budget, managerId). Write SQL to: (i) create the relations; (ii) add a 40-year-old Azi Esther, Eid 101, salary 15000; (iii) give every employee a 10% rise; (iv) delete the Toy department; (v) set the Account department budget to 8 million; (vi) remove the Dept relation; (vii) list Emp with Eid first.

ANSWER

```
(i) CREATE TABLE Emp (
      Eid INT PRIMARY KEY, Ename VARCHAR(30),
      Age INT, Salary DECIMAL(10,2) );
CREATE TABLE Dept (
      Did INT PRIMARY KEY, Dname VARCHAR(20),
      Budget DECIMAL(12,2), managerId INT );
CREATE TABLE Works (
      Eid INT, Did INT, per_time INT,
      PRIMARY KEY (Eid, Did),
      FOREIGN KEY (Eid) REFERENCES Emp(Eid),
      FOREIGN KEY (Did) REFERENCES Dept(Did) );

(ii) INSERT INTO Emp VALUES (101, 'Azi Esther', 40, 15000);
(iii) UPDATE Emp SET Salary = Salary * 1.10;
(iv) DELETE FROM Dept WHERE Dname = 'Toy';
(v) UPDATE Dept SET Budget = 8000000 WHERE Dname = 'Account';
(vi) DROP TABLE Dept;
(vii) SELECT Eid, Ename, Age, Salary FROM Emp;
```

(vii) column order in the result is chosen by naming the columns in the SELECT; changing the stored order would mean recreating the table.

2.7 Worked: the Hotel schema

AS PRINTED

20/21 Q1B, 15/16 Q5B

(b) Using the schema below, generate SQL statement for the following queries:

Hotel (hotelNo, hotelName, City)

Room (roomNo, hotelNo, type, price)

Booking (hotelNo, guestNo, dateFrom, dateTo, roomNo)

Guest (guestNo, guestName, guestAddress)

(i) How many hotels are there?

(ii) List full details of all hotels in London

(iii) List number of rooms in each hotel

[4.5 marks]

DTS224 20/21, Q1b, as printed.

(b) Using the schema in 4(c), generates SQL statement for the following queries:

(8marks)

(i) How many hotels are there?

(ii) List full details of all hotels in London

(iii) List number of rooms in each hotel

(iv) List the names and address of all guests living in London, alphabetically ordered

by

name. use pattern matching operator.

DTS224 15/16, Q5b, as printed.

Hotel(hotelNo, hotelName, city); Room(roomNo, hotelNo, type, price);
Booking(hotelNo, guestNo, dateFrom, dateTo, roomNo); Guest(guestNo, guestName, guestAddress).

```
-- how many hotels
SELECT COUNT(*) FROM Hotel;
-- full details of all hotels in London
SELECT * FROM Hotel WHERE city = 'London';
-- number of rooms in each hotel
SELECT hotelNo, COUNT(*) AS rooms FROM Room GROUP BY hotelNo;
-- guests in London, alphabetically by name (pattern match)
SELECT guestName, guestAddress FROM Guest
WHERE guestAddress LIKE '%London%' ORDER BY guestName;
-- double or family rooms below 40 per night, cheapest first
SELECT * FROM Room WHERE type IN ('double','family') AND price < 40
ORDER BY price;
-- bookings with no dateTo
SELECT * FROM Booking WHERE dateTo IS NULL;
```

2.8 Worked: products, and a correlated subquery

AS PRINTED

19/20 Q4A, 21/22 Q1C

4. (a) Given the following relations;

Customer_t(Customer_ID, Customer_Name, Customer_Address, City, State, Postal_Code)

Orderline_t(Order_ID, Product_ID, Order_Quantity)

Order_t(Order_ID, Order_Date, Customer_ID)

Product_t(Product_ID, Product_Description, Product_Finish,

Standard_Price, Product_Line_ID)

Write the SQL statements to answer the following queries

- i. List products, finish, and unit price for all desks and all tables that cost more than 300 in the PRODUCT view
- ii. Which products in the PRODUCT view have a standard price between 200 and 300.
- iii. How many different items were ordered on order number 1004
- iv. Which products have a standard price of less than 275.
- v. What is the address of the customer named Home Furnishings. Use an alias NAME for customer.
- vi. List the unit price, product name and product ID for all products in the product table.
- vii. What is the average standard price for product in inventory [14 Marks]

DTS224 19/20, Q4a, as printed.

(c) Write DML statements to achieve the following; (4.5 Marks)

- i. Find all staff whose salary is larger than the salary of every member of staff at branch B003
- ii. List the properties that are handled by staff who work in the branch at '22 Deer Road'
- iii. List full details of all branch

QUESTION TWO

DTS224 21/22, Q1c, as printed.

Product_t(Product_ID, Product_Description, Product_Finish, Standard_Price, Product_Line_ID); Customer_t(Customer_ID, Customer_Name, Customer_Address, ...); Orderline_t(Order_ID, Product_ID, Order_Quantity).

```
-- desks and tables costing more than 300
SELECT Product_Description, Product_Finish, Standard_Price FROM Product_t
WHERE Product_Description IN ('Desk','Table') AND Standard_Price > 300;
-- products priced between 200 and 300
SELECT * FROM Product_t WHERE Standard_Price BETWEEN 200 AND 300;
-- how many different items on order 1004
SELECT COUNT(*) FROM Orderline_t WHERE Order_ID = 1004;
-- address of a named customer, aliasing the table as NAME
SELECT Customer_Address FROM Customer_t AS NAME
WHERE Customer_Name = 'Home Furnishings';
-- average standard price
SELECT AVG(Standard_Price) FROM Product_t;
```

The 21/22 paper adds a **correlated subquery**: find staff earning more than every staff member at branch B003:

```
SELECT * FROM Staff
WHERE salary > ALL (SELECT salary FROM Staff WHERE branchNo = 'B003');
```

⚠ TRAP

NULL AND COUNT

To find rows with a missing value you must write `IS NULL`, never `= NULL` (nothing equals NULL, not even NULL). And `COUNT(column)` skips NULLs in that column, while `COUNT(*)` counts every row: choose the one the question means.

RECALL

COVER THE ANSWERS

1. Which clause filters rows? Which filters groups?
2. Write the condition for "address contains Glasgow".
3. How do you count rows per branch?
4. What enforces referential integrity in a CREATE TABLE?
5. How do you find bookings with no end date?

ANSWERS

1. WHERE filters rows; HAVING filters groups. 2. address LIKE '%Glasgow%'. 3. SELECT branch, COUNT(*) FROM ... GROUP BY branch. 4. A FOREIGN KEY ... REFERENCES clause. 5. WHERE dateTo IS NULL.

WORKED PAPER 2025/2026

The 2025/2026 Paper, Fully Worked

The most recent paper, the one whose shape the examination now follows, solved from end to end. Every question below is first reproduced exactly as the examiner printed it, then read apart, then answered. Sit it closed-book first; then read the working.



Questions One to Three

Rubric as printed: **Attempt QUESTION ONE and any three (3) questions, in 3 hours.** Question One is compulsory and carries 17½ marks, as does every other question. Each question below appears three times over: the examiner's words, a reading of them, then the answer.

AS PRINTED

QUESTION ONE, COMPULSORY

a. Briefly explain the following with symbolic examples:

- (i) Super type
- (ii) Specialisation
- (iii) Total specialisation
- (iv) Disjoint rule (4 mks)

DTS224 2025/2026, Question One (a).

b. Use Figure 1 to explain the concept of determinant in the context of normalization. Show all possible determinants, the corresponding attributes and the type of functional dependency involved. (2½ mks)

DTS224 2025/2026, Question One (b).

c. Consider the relational schema and write the SQL queries to answer the business questions labelled (i-vi) with standard SQL syntax.

BRANCH (branch_no, street, city, postcode)
 STAFF (staff_no, fName, lName, position, sex, DOB, salary, branch_no)
 PROPERTYFORRENT (property_no, street, city, postcode, type, rooms, rent, staff_no, branch_no).

- (i) List all staff members whose salary is greater than ₦100,000, ordered from highest salary to lowest salary. (1½ mks)
- (ii) Display the total number of staff working in each branch. Include the branch number and total staff count. (2 mks)
- (iii) Find the average salary of staff in each branch. Display branch number and average salary. (1½ mks)
- (iv) Display the details of all properties having more than 3 rooms and a monthly rent above ₦80,000. (2 mks)
- (v) Show the property number, property type, rent, and the full name of the staff member responsible for each property. (2 mks)
- (vi) Display all branches that currently have no staff assigned to them. (2 mks)

QUESTION TWO (17 ½ mks)

DTS224 2025/2026, Question One (c).

Two things to notice before you answer. The paper spells the marks four different ways in this question alone, and part (c) prices salaries in Naira: it asks for salary greater than ₦100,000, not 100000. Quote the figure the examiner used. Figure 1 is printed as a bare row of attribute boxes with arrows beneath it; the arrows ARE the functional dependencies you are being asked to read.

- GIVEN** four EER terms; Figure 1, the EMPLOYEE2 relation with EmpID and CourseTitle underlined as a composite key; and a three-relation schema (BRANCH, STAFF, PROPERTYFORRENT).
- ASKED** to **explain with symbolic examples** (not merely define, so each term needs a worked instance); to **show all possible determinants** and name the **type** of each dependency; and to write six SQL queries.
- MARKS** $4 + 2\frac{1}{2} + 11 = 17\frac{1}{2}$. The SQL is worth nearly two thirds of a compulsory question, so it is the part to practise, not the definitions.
- TAUGHT** Module Two Unit 2 (EER terms), Module Four Unit 1 (determinants), Module Five Unit 2 (SQL).
- TRAP** in (b) the marks are only $2\frac{1}{2}$, yet "show ALL possible determinants" means both EmpID alone and the composite. Naming one earns half. In (c)(vi) "no staff assigned" is an anti-join: NOT IN or a LEFT JOIN with IS NULL, never a plain join.

(a) EER terms, with symbolic examples. Super type: a generic entity type holding the attributes common to its subtypes, for example EMPLOYEE over HOURLY and SALARIED. **Specialisation:** the top-down process of defining subtypes of a supertype (EMPLOYEE specialised into HOURLY and SALARIED). **Total specialisation:** every supertype instance must belong to at least one subtype, drawn as a double line. **Disjoint rule:** an instance may belong to at most one subtype, drawn as a circle marked d. (Module Two, Unit 2.)

(b) Determinants in EMPLOYEE2. A determinant is the attribute, or set of attributes, on the left of a functional dependency. In EMPLOYEE2(EmpID, CourseTitle, Name, DeptName, Salary, DateCompleted):

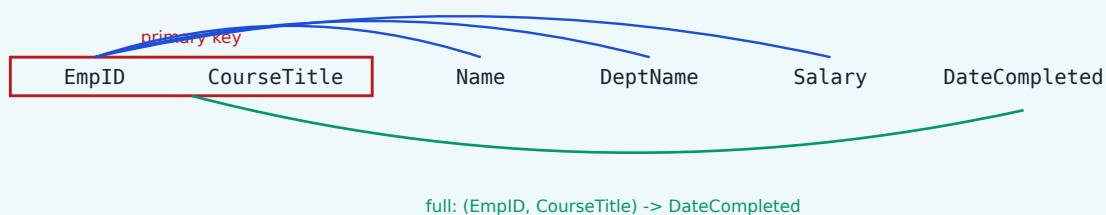


Figure P.1. EMPLOYEE2. EmpID (a partial determinant) $\rightarrow$ Name, DeptName, Salary. The whole key (EmpID, CourseTitle) $\rightarrow$ DateCompleted is the only full dependency.

EmpID determines Name, DeptName and Salary: a **partial** dependency, since EmpID is only part of the key. The composite (**EmpID, CourseTitle**) determines DateCompleted: a **full** functional dependency. Because a partial dependency exists, the relation is in 1NF but not 2NF.

(c) The six SQL queries. Solved at length in Module Five, Unit 2.4.

- (i) `SELECT * FROM STAFF WHERE salary > 100000 ORDER BY salary DESC;`
- (ii) `SELECT branch_no, COUNT(*) FROM STAFF GROUP BY branch_no;`
- (iii) `SELECT branch_no, AVG(salary) FROM STAFF GROUP BY branch_no;`
- (iv) `SELECT * FROM PROPERTYFORRENT WHERE rooms > 3 AND rent > 80000;`
- (v) `SELECT p.property_no, p.type, p.rent, s.fName, s.lName
FROM PROPERTYFORRENT p JOIN STAFF s ON p.staff_no = s.staff_no;`
- (vi) `SELECT branch_no FROM BRANCH
WHERE branch_no NOT IN (SELECT branch_no FROM STAFF);`

The amounts are written 100000 and 80000 in SQL because the Naira sign and the thousands commas are presentation, not data; say so in one line under your answer and you have shown you read the question rather than ignored the currency.

- a. Explain two (2) limitations of the file-based approach to data processing. (2 mks)

DTS224 2025/2026, Question Two (a).

- b. What is a Database Management System (DBMS). (2 mks)

DTS224 2025/2026, Question Two (b).

- c. What is meant by data abstraction? Explain the three (3) levels of data abstraction with examples. (5 ½ mks)

DTS224 2025/2026, Question Two (c).

- d. Transform the EER for a University Dining Service in Figure 2 into a set of relational schemas. Show the prim (Use the short text notation). (8 mks)

QUESTION THREE

(17 ½ mks)

DTS224 2025/2026, Question Two (d).

Part (d) carries a deletion the setter left standing. The paper prints "Show the prim" struck through, mid-sentence, exactly as reproduced above. It appears to be the start of a deleted instruction (*show the primary keys*) that survived typesetting. Answer what remains, and show the primary keys anyway: it costs one line and it is plainly what was intended. Note too that (b) ends in a full stop rather than a question mark.

GIVEN nothing but definitions for (a) to (c); for (d), Figure 2, a crow's-foot EER for a University Dining Service with five entity types.

ASKED to **explain**, then to **transform** a diagram into relational schemas. "Transform" means produce the relations, not describe how you would.

MARKS $2 + 2 + 5\frac{1}{2} + 8 = 17\frac{1}{2}$. Part (d) alone is worth more than (a), (b) and (c) together, and it is the only part that cannot be bluffed.

TAUGHT Module One Units 2.1 to 2.5 (file-based limits, DBMS, abstraction); Module Three Unit 2.4 (the transform).

TRAP the multivalued attributes in Figure 2 are printed in braces, {Ingredient} and {Skill}. Each becomes its OWN relation, not a column. Miss that and the transform loses several marks even though the rest is right.

(a) Two limitations of the file-based approach. Data redundancy and inconsistency, since the same fact is stored and updated in several files; and program-data dependence, since a file's structure is coded into every program that reads it. (Module One, Unit 2.1.)

(b) A DBMS. The software that lets users define, create, maintain and control access to a database, for example MySQL or Oracle. (Module One, Unit 2.2.)

(c) Data abstraction, three levels. Hiding storage detail so each user sees only what they need: the **external** level (per-user views), the **conceptual** level (all tables and relationships), and the **internal** level (physical storage). (Module One, Unit 2.5.)

(d) The University Dining transform. Worked in full at Module Three, Unit 2.4. Each multivalued attribute becomes its own relation, and each many-to-many becomes a link relation:

MENU(menuID, menuDescription, menuType)	PK menuID
DISH(dishID, dishName, prepTime)	PK dishID
DISH_INGREDIENT(dishID, ingredient)	PK (dishID, ingredient)
MENU_DISH(menuID, dishID)	PK (menuID, dishID)
EVENT(eventID, eventDate, eventLocation, eventTime)	PK eventID
MENU_EVENT(menuID, eventID)	PK (menuID, eventID)
STAFF(empID, name, salary, supervisorID)	PK empID FK supervisorID -> STAFF
STAFF_SKILL(empID, skill)	PK (empID, skill)
WORK_SCHEDULE(empID, eventID, startTime, endTime, position)	
PK (empID, eventID)	

- a. (i) Would you choose a database system instead of simply storing data in operating system files? Justify your answer with two points. (5 marks)
- (ii) When would it make sense not to use a database system? (2.5 marks)

DTS224 2025/2026, Question Three (a).

- b. Draw an EER diagram for the following description of a law firm:

Each case handled by the firm has a unique case number; a date opened, date closed, and judgment description are also kept on each case. A case is brought by one or more plaintiffs,

and the same plaintiff may be involved in many cases. A plaintiff has a requested judgment characteristic. A case is against one or more defendants, and the same defendant may be involved in many cases. A plaintiff or defendant may be a person or an organisation. Over time, the same person or organisation may be a defendant or a plaintiff in cases. In either situation, such legal entities are identified by an entity number, and other attributes are name

and net worth. (10 marks)

QUESTION FOUR

(17 ½ mks)

DTS224 2025/2026, Question Three (b).

Question Three is the one place in this paper where the examiner writes "marks" instead of "mks" and uses a decimal, "2.5 marks", instead of the vulgar fraction used everywhere else. It is the same 17½ total. Do not read 2.5 as 25 and spend half your time here.

GIVEN a judgement call in (a), and in (b) a narrative description of a law firm.

ASKED (a) asks "**Would you choose ... Justify** with two points", so an unjustified yes scores nothing; (a)(ii) asks when it would make sense **not** to, the word printed in italics because it inverts the question. (b) asks you to **draw**.

MARKS 5 + 2.5 + 10 = 17½. The diagram is worth ten, so budget accordingly.

TAUGHT Module One Units 2.1 and 2.7; Module Two Unit 2 (EER).

TRAP the same person or organisation may be a plaintiff in one case and a defendant in another. That is **two** many-to-many relationships between the same pair of entity types, not one. Drawing a single "involved in" line is the error the question is built to catch.

(a) Database or operating-system files. (i) Choose a database: it gives one shared, consistent copy with concurrency control, and enforces integrity, security and data independence centrally, none of which loose files provide. (ii) It would not be worth it for a small, single-user, single-application task with little data and a fixed structure, where a full DBMS is pure overhead. (Module One, Units 2.1 and 2.7.)

(b) EER for the law firm. Each CASE (caseNumber, dateOpened, dateClosed, judgmentDescription) is brought by one or more plaintiffs and is against one or more defendants. A plaintiff or defendant is a LEGAL ENTITY (entityNumber, name, netWorth), which is either a PERSON or an ORGANISATION. Because the same entity can be plaintiff in some cases and defendant in others, draw **two** many-to-many relationships between LEGAL ENTITY and CASE. requestedJudgment is a property of the plaintiff relationship, not of the entity, so it sits on that relationship.

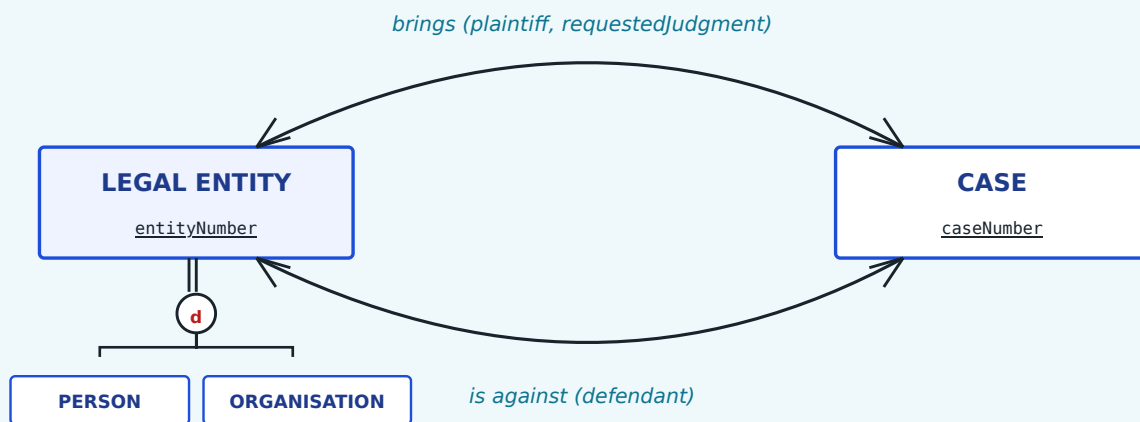


Figure P.2. Law firm. LEGAL ENTITY is a total, disjoint supertype over PERSON and ORGANISATION, and relates to CASE twice: "brings" (as plaintiff, carrying requestedJudgment) and "is against" (as defendant), both many-to-many.

WORKED PAPER 2025/2026

Questions Four to Six

The remaining three, any two of which you would answer alongside Question One. Full working, with the module named where a topic is treated at length.

- a. Describe the relational Data model. (2 ½ mks)

DTS224 2025/2026, Question Four (a).

- b. Explain six (6) Relational Algebra operations with examples. (6 mks)

DTS224 2025/2026, Question Four (b).

- c. Consider the following relational schemas and write the relation algebra for the following tasks

(i-iii). (9 mks)

Emp (Fname, Lname, SSN, Dno)

Dept (DName, DNum, Mgr_SSN)

D_Loc (DNo, Location)

Project (PName, PNo, DNo, P_Loc)

Works_on (ESSN, PNo, Hours)

- (i) Retrieve the names of all employees who work for the 'Headquarters' Department.
 (ii) For every project located in 'Abuja', list the project number, controlling department and the manager's name.
 (iii) Find the names of the employees who work on all the projects controlled by DNo = A314.

QUESTION FIVE

(17 ½ mks)

DTS224 2025/2026, Question Four (c).

The paper writes "relation algebra" in (c) where (b) writes "Relational Algebra"; both mean the same thing. In (c)(iii) the department is printed as "DNo = A314", in italics, though A314 is not a number and the schema declares DNum on Dept and DNo on D_Loc. Answer it as the department whose identifier is A314 and say in one line which attribute you took it to be.

GIVEN five relation schemas: Emp, Dept, D_Loc, Project and Works_on.

ASKED to **describe**, to **explain six operations with examples** (six, and each needs an example), and to **write the relation algebra**, meaning expressions in symbols, not English and not SQL.

MARKS $2\frac{1}{2} + 6 + 9 = 17\frac{1}{2}$. Six operations for six marks is one mark each with an example; do not write an essay on selection and run out of time.

TAUGHT Module Three Unit 1 (the model); Module Five Unit 1 (algebra, all eight operations and their symbols).

TRAP (iii) says "work on **all** the projects controlled by" a department. "All" is the signature of **division** ($\div$). A join and a count will not do it and is the commonest wrong answer.

(a) The relational data model. It represents all data as relations, that is two-dimensional tables of rows and columns, with relationships between them expressed by shared values rather than by pointers, and it is manipulated by relational algebra. (Module Three, Unit 1.)

(b) Six operations, with examples. Selection (σ , choose rows: $\sigma_{\text{rent} < 500}(\text{PROPERTY})$); projection (π , choose columns: $\pi_{\text{fName}, \text{lName}}(\text{STAFF})$); union ($\cup$, all rows of either); set difference ($-$, rows of one not in the other); Cartesian product ($\times$, every pairing); and join ($\bowtie$, a product kept only where a condition holds). (Module Five, Unit 1.1.)

(c) The three algebra expressions. Worked at Module Five, Unit 1.4.

```
(i)   HQ      ← σ DName='Headquarters' (Dept)
      Result ← π FName, Lname ( Emp ⋈ Dno=DNum HQ )

(ii)  Abuja   ← σ P_Loc='Abuja' (Project)
      Result ← π PNo, DNo, Mgr_SSN ( Abuja ⋈ Dno=DNum Dept )

(iii) A314    ← π PNo ( σ DNo='A314' (Project) )
      Result ← π FName, Lname ( ( π ESSN, PNo (Works_on) ÷ A314 ) ⋈ ESSN=SSN Emp )
```

In (ii) the manager's name needs one more join, Dept to Emp on Mgr_SSN = SSN, if the examiner wants the name rather than the SSN. Say which you read it as. In (iii) the division is what "all the projects" demands.

- a. Given the following database concept in (I-III), answer question (i-ii):
- I. The data definition language.
 - II. The data manipulation language.
 - III. The data model.
- (i) Which of the following enumerations (I) to (III) plays an important role in representing information about the real world in a database? (½ mk)
- (ii) Explain each concept briefly; your explanation should reflect the answer for your choice. (6 mks)

DTS224 2025/2026, Question Five (a).

- b. Develop an EER model for the following situation, using the crowfoot notation. (11 mks)

A technology company provides offerings to its customers. Offerings are of two separate types:

products and services. Offerings are identified by an offering ID and an attribute of description. In

addition, products are described by product name, standard price, and date of first release; services

are described by name of the company's unit responsible for the service and conditions of service.

There are repair, maintenance, and other types of services. A repair service has a cost and is the repair

of some product; a maintenance service has an hourly rate. Fortunately, some products never require

repair. However, there are many potential repair services for a product.

A customer may purchase an offering, and the company needs to keep track of when the offering was

purchased and the contact person for that offering with the customer. Unfortunately, not all offerings

are purchased. Customers are identified by customer ID and have descriptive data of name, address,

and phone number. When a service is performed, that service is billed to some customer.

Because

some customers purchase offerings for their clients, a customer may be billed for services he or she

did not purchase, as well as for ones that were purchased. When a customer is billed for a service

(although some may never require a service of any type), the company needs to keep track of the date

the service was performed, the date the bill is due, and the amount due.

QUESTION SIX

(17 ½ mks)

DTS224 2025/2026, Question Five (b).

Part (a)(i) asks which of I to III "plays an important role in representing information about the real world", and is worth half a mark: it wants one word, not a paragraph. The six marks are in (ii), which asks you to explain each concept and says the explanation "should reflect the answer for your choice", so the three explanations must argue for the one you picked.

GIVEN three enumerated concepts (DDL, DML, data model); and a long narrative about a technology company's offerings.

ASKED to **choose** then **explain**; and to **develop an EER model using the crowfoot notation**. The notation is specified, so a Chen diagram scores badly however correct.

MARKS $\frac{1}{2} + 6 + 11 = 17\frac{1}{2}$. The model is worth eleven, nearly two thirds.

TAUGHT Module One Unit 2.3 and Module Three Unit 1 (DDL/DML); Module Two Unit 2.6 (this exact EER, worked).

TRAP read the completeness constraints out of the words. "Offerings are of two separate types" is total and disjoint; but "There are repair, maintenance, and **other** types of services" makes SERVICE only **partial**. And "some products never require repair" plus "many potential repair services for a product" fixes the optionality and the cardinality of that relationship.

(a) The choice and the three concepts. (i) The **data model**. (ii) A **data model** is the set of concepts (entities, relationships, attributes and constraints) used to describe the structure and meaning of data, so it is the one that represents information about the real world. The **data definition language** defines that structure to the DBMS (CREATE, ALTER, DROP), and the **data manipulation language** operates on the data held in it (SELECT, INSERT, UPDATE, DELETE). Neither language models reality; they realise and query a model. (Module One, Unit 2.3.)

(b) Crow's-foot EER for the technology company. Worked in full at Module Two, Unit 2.6. OFFERING (offeringID, description) is **total and disjoint** over PRODUCT (productName, standardPrice, dateOfFirstRelease) and SERVICE (unitName, condition-OfService). SERVICE is **partial and disjoint** over REPAIR (cost) and MAINTENANCE (hourlyRate), because the paper allows "other types of services". A REPAIR *repairs* a PRODUCT: optional on the product side (some products never require repair), many on the repair side (many potential repair services for a product). CUSTOMER (customerID, name, address, phoneNumber) *purchases* OFFERING, an optional many-to-many carrying purchaseDate and contactPerson; and a SERVICE performed is *billed to* a CUSTOMER, carrying datePerformed, dateDue and amountDue, which is a separate relationship from the purchase because a customer may be billed for a service they did not purchase.

- a. Explain the following relational terms briefly: (3 mks)
- (i) Schema
 - (ii) Alternate key
 - (iii) Tuple

DTS224 2025/2026, Question Six (a).

- b. Given the StaffPropertyInspection Relation, answer the following questions:

- (i) Draw the functional dependency diagram for the relation. (2½mks)
- (ii) Briefly explain the concept of transitive Functional dependency. (2 mk)
- (iii) Normalise the relation to the 3rd Normal Forms. The primary keys are: (propertyNo, iDate)

Note: All actions involved in the normalization process should be well stated. The output relations

and the tables should be presented clearly with identified primary key (10 mk)

Table 1: StaffPropertyInspection

propertyNo	pAddress	iDate	iTime	comments	staffNo	carReg
PG4	6 Lawrence St,	18-Oct-03	10.00	Need to replace	SG37	M231
JGR	Kaifa					
	Glasgow.			crockery.		
		22-Apr-04	09.00	In good order.	SG14	M533 HDR
Damilola						
PG16	5 Novar Dr,	22-Apr-04	13.00	Replace living room	SG14	M533
HDR	Damilola					
	Glasgow.			carpet.		
		24-Oct-04	14.00	Good condition.	SG37	N721 HFR
Kaifa						

Functional dependencies:

- fd1 propertyNo, iDate → iTime, comments, staffNo, sName, carReg (Primary key)
- fd2 propertyNo → pAddress (Partial dependency)
- fd3 staffNo → sName (Transitive dependency)
- fd4 staffNo, iDate → carReg (Transitive dependency)
- fd5 carReg, iDate, iTime → propertyNo, pAddress, comments, staffNo, sName (Candidate key)
- fd6 staffNo, iDate, iTime → propertyNo, pAddress, comments (Candidate key)

DTS224 2025/2026, Question Six (b).

Table 1 heads its last column **staffName**, but every functional dependency beneath it calls that attribute **sName**. The question therefore names one attribute two ways, and it is not a transcription slip: it is printed. Say in one line which you are using and use it consistently. Note also that the paper labels fd3 and fd4 both "(Transitive dependency)" and fd2 "(Partial dependency)"; those labels are the examiner's, and part (ii) asks you to explain transitive dependency, so quote fd3 as your example and let fd4 alone.

GIVEN three relational terms; and StaffPropertyInspection with four data rows, six named functional dependencies and the primary key stated as (propertyNo, iDate).

ASKED to **draw** the FD diagram, to **briefly explain** transitive dependency (so two or three sentences, not a page), and to **normalise to 3NF** with **all actions well stated** and the output tables presented with their primary keys identified. The Note is itself worth marks.

MARKS $3 + 2\frac{1}{2} + 2 + 10 = 17\frac{1}{2}$. The normalisation is ten, and the Note tells you those ten are for the **working**, not just the final tables.

TAUGHT Module Three Unit 1 (the terms); Module Four Unit 1.4, where this exact relation is normalised step by step.

TRAP the key is given, so do not spend time deriving it. Work 1NF to 2NF to 3NF in named steps and show the relation you split off at each one; a correct final answer with no working cannot score ten.

ANSWER

QUESTION SIX

(a) The three terms. Schema: the structure of a relation, its name together with its attributes, for example STAFF(staffNo, fName, lName, branchNo). **Alternate key:** a candidate key that was not chosen as the primary key. **Tuple:** one row of a relation, one complete record. (Module Three, Unit 1.)

(b)(i) and (iii) are worked step by step at Module Four, Unit 1.4. Taking the attribute as sName throughout, and the key as (propertyNo, iDate):

(ii) Transitive dependency. A non-key attribute depends on the key only through another non-key attribute. Here staffNo $\rightarrow$ sName: sName is determined by staffNo, and staffNo is itself determined by the key, so sName reaches the key only via staffNo. That is what 3NF removes, by moving sName into a relation of its own keyed on staffNo.

The 3NF result:

Property(propertyNo, pAddress)	PK propertyNo
Staff(staffNo, sName)	PK staffNo
PropertyInspection(propertyNo, iDate, iTime, comments, staffNo, carReg)	
PK (propertyNo, iDate) FK propertyNo $\rightarrow$ Property, FK staffNo $\rightarrow$ Staff	

Steps to state: 1NF, the table already holds single values, so it qualifies once the repeating property address is understood as belonging to the property. 2NF, remove the partial dependency fd2 (propertyNo $\rightarrow$ pAddress) into Property. 3NF, remove the transitive dependency fd3 (staffNo $\rightarrow$ sName) into Staff. carReg is left on the inspection because fd4 makes it depend on staffNo together with iDate, which is not a determinant available in either split relation.

Question One is compulsory and mixes EER terms, a functional-dependency reading and SQL, so it rewards broad preparation. For your three chosen questions, pick the ones whose long parts you are surest of: the transforms (Q2d, Q5b), the algebra (Q4c) and the normalisation (Q6b) each carry the most marks and are the most mechanical once learned. Draw every diagram large and label every cardinality; an unlabelled line scores little.

Watch the mark text as you triage. This paper does not write marks the same way twice: most parts read "mks", Question Three alone reads "marks" and prints 2.5 as a decimal, and Question Six drops to the singular "mk". None of that changes what a part is worth, but a candidate skimming for the big numbers can misread 2.5 as 25.

MOCK EXAMINATION ONE

Sit This Under Exam Conditions

Two hours, no notes, no computer. Attempt any four of the six questions. This paper follows the 2024/2025 shape. The answers begin on a fresh page, so you can mark yourself honestly.

Question Paper

DTS224 Data Management I. Time: 2 hours. Attempt any four (4) questions. Each question carries 25 marks.

QUESTION ONE

- (a) Define: relational model, tuple, attribute, domain. **(4)**
- (b) Given Book(bookID, title, author, year), Member(memberID, name, city), Loan(bookID, memberID, dateOut): write relational algebra for (i) all books published after 2015; (ii) names of members in Ibadan; (iii) titles of books on loan. **(9)**
- (c) $A(x) = \{1, 2\}$, $B(y) = \{p, q, r\}$. Give $A \times B$ and its cardinality. **(6)**
- (d) State the entity integrity rule and the referential integrity rule. **(6)**

QUESTION TWO

- (a) Differentiate with an example: (i) associative entity and ternary relationship; (ii) multivalued and composite attribute. **(6)**
- (b) List six properties of a well-structured relation. **(6)**
- (c) A clinic keeps DOCTORS (doctorID, name) and PATIENTs (patientID, name). A doctor sees many patients through an APPOINTMENT (date, time, notes); a patient may see many doctors. Draw one connected ER diagram with cardinalities. **(13)**

QUESTION THREE

- (a) What is normalization? Explain full, partial, and transitive functional dependency with an example each. **(8)**
- (b) Given Order(orderNo, productNo, orderDate, custNo, custName, qty, unitPrice) with $\text{custNo} \rightarrow \text{custName}$ and $\text{orderNo} \rightarrow \text{orderDate}$, custNo : draw the FD diagram and normalize to 3NF, showing the final relations. **(17)**

QUESTION FOUR

- (a) Differentiate: (i) total and partial specialization; (ii) supertype and subtype. **(6)**
- (b) VEHICLE(vehID, make, price) is a disjoint, total supertype over CAR (numDoors) and TRUCK (capacity). Transform this EER into a logical (relational) design showing keys and foreign keys. **(10)**
- (c) Describe a DBMS with two examples, and differentiate DDL from DML. **(9)**

QUESTION FIVE

- (a) Define primary key, candidate key, foreign key, super key. **(4)**
- (b) Given Emp(empNo, name, salary, deptNo) and Dept(deptNo, dName, city): write SQL to (i) create both tables enforcing referential integrity; (ii) insert one employee; (iii) give all employees a 5% rise; (iv) create a view of Lagos-department employees excluding salary; (v) list employees earning over 50000, highest first; (vi) count employees per department. **(21)**

QUESTION SIX

- (a) You are an intern building an inventory system for a small shop on a tight one-month schedule with the owner closely involved. (i) Explain what an information system is. (ii) Give three reasons Rapid Application Development suits this project. (iii) State six characteristics of a good business rule. **(13)**
- (b) Explain with examples: entity type, required attribute, disjoint rule, overlap rule. **(12)**

MOCK EXAMINATION ONE

Answers

Full solutions. Compare yours line by line, and where you missed a mark, go back to the module named.

ANSWER

QUESTION ONE

(a) Relational model: data as relations (tables) with relationships by shared values. Tuple: a row. Attribute: a column. Domain: the set of allowed values of an attribute.

(b)

- ```
(i) σ year > 2015 (Book)
(ii) π name (σ city='Ibadan' (Member))
(iii) π title (Book ⋈ Loan)
```

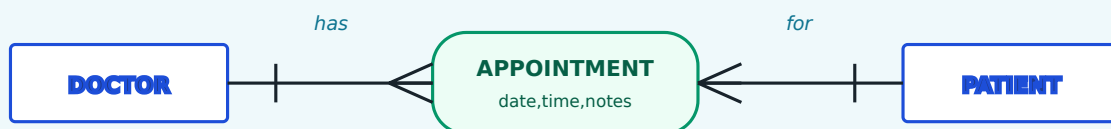
**(c)**  $A \times B = \{(1,p),(1,q),(1,r),(2,p),(2,q),(2,r)\}$ ; cardinality  $2 \times 3 = 6$ .

**(d)** Entity integrity: no part of a primary key may be null. Referential integrity: a foreign-key value must match an existing primary key or be wholly null.

**(a)(i)** An associative entity is a relationship (usually many-to-many) promoted to an entity because it has its own attributes; a ternary relationship simply relates three entity types at once. **(ii)** A multivalued attribute holds several values of one kind ({phones}); a composite attribute is one value split into parts (Name = first + last).

**(b)** Single-valued cells; distinct attribute names; values from one domain; attribute order irrelevant; tuple order irrelevant; every tuple unique.

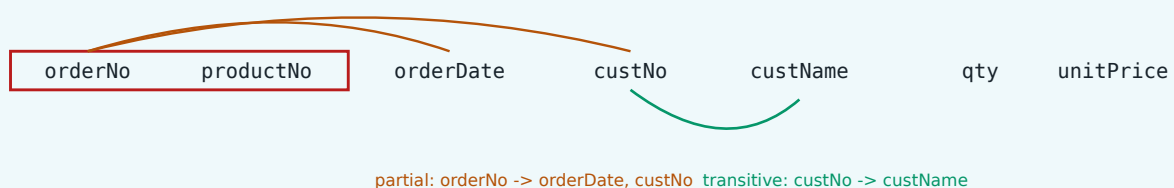
**(c)** A doctor sees many patients and a patient sees many doctors, and the pairing carries date, time, and notes, so APPOINTMENT is an associative entity:



Mock 1, Q2(c). DOCTOR and PATIENT joined through the associative entity APPOINTMENT, many-to-many both ways.

**(a)** Normalization organises attributes into well-structured relations, removing redundancy and anomalies using functional dependencies. Full: depends on the whole composite key. Partial: depends on part of the key. Transitive: depends on the key only through a non-key attribute.

**(b)** The key is (orderNo, productNo). custNo, custName, orderDate depend on orderNo alone (partial); custName depends on custNo (transitive).



Mock 1, Q3(b). FD diagram: amber = partial (on orderNo), green = transitive (custName on custNo).

```

Customer(custNo, custName) PK custNo
OrderHead(orderNo, orderDate, custNo) PK orderNo FK custNo -> Customer
OrderLine(orderNo, productNo, qty, unitPrice)
 PK (orderNo, productNo) FK orderNo -> OrderHead

```

**(a)(i)** Total specialization: every supertype instance must be in some subtype; partial: it need not be. **(ii)** Supertype: the general type holding common attributes; subtype: a specialised subgroup with its own extra attributes, inheriting the supertype's.

**(b)** Each subtype becomes a relation sharing the supertype's key:

|                             |          |                     |
|-----------------------------|----------|---------------------|
| VEHICLE(vehID, make, price) | PK vehID |                     |
| CAR(vehID, numDoors)        | PK vehID | FK vehID -> VEHICLE |
| TRUCK(vehID, capacity)      | PK vehID | FK vehID -> VEHICLE |

**(c)** A DBMS is the software that defines, creates, maintains, and controls access to a database (MySQL, Oracle). DDL defines structure (CREATE, ALTER, DROP); DML changes data (INSERT, UPDATE, DELETE, SELECT).

**(a)** Super key: any attribute set that identifies a row uniquely. Candidate key: a minimal super key. Primary key: the chosen candidate key, never null. Foreign key: an attribute matching a primary key elsewhere.

```
(i) CREATE TABLE Dept (
 deptNo CHAR(4) PRIMARY KEY, dName VARCHAR(20), city VARCHAR(20));
CREATE TABLE Emp (
 empNo CHAR(5) PRIMARY KEY, name VARCHAR(30),
 salary DECIMAL(9,2), deptNo CHAR(4),
 FOREIGN KEY (deptNo) REFERENCES Dept(deptNo));
(ii) INSERT INTO Emp VALUES ('E001','Ada Obi',60000,'D01');
(iii) UPDATE Emp SET salary = salary * 1.05;
(iv) CREATE VIEW lagosStaff AS
 SELECT empNo, name, deptNo FROM Emp
 WHERE deptNo IN (SELECT deptNo FROM Dept WHERE city = 'Lagos');
(v) SELECT * FROM Emp WHERE salary > 50000 ORDER BY salary DESC;
(vi) SELECT deptNo, COUNT(*) FROM Emp GROUP BY deptNo;
```

**(a)(i)** An information system is an organised combination of people, data, processes, and technology that collects, stores, processes, and delivers information to support an organisation. **(ii)** RAD suits this project because the timeline is short (RAD delivers working software fast through prototyping), resources are limited (it reuses components and avoids heavy up-front documentation), and the owner is involved (RAD depends on continuous user feedback). **(iii)** A good business rule is declarative, precise, atomic, consistent, expressible, distinct, and business-oriented (any six).

**(b)** Entity type: a collection of entities with common properties (STUDENT). Required attribute: one that must have a value for every instance (studentID). Disjoint rule: an instance may be at most one subtype. Overlap rule: an instance may be several subtypes at once.

## MOCK EXAMINATION TWO

# Sit This One After the First

*This paper follows the 2025/2026 shape: Question One is compulsory, then any three of the rest. Two hours. Sit it closed-book before reading the answers.*

# Question Paper

*DTS224 Data Management I. Time: 2 hours. Answer Question One (compulsory) and any three others. Each question carries 17½ marks.*

## QUESTION ONE

**COMPULSORY**

- (a) Explain, with symbolic examples: generalization, specialization, subtype discriminator, completeness constraint. **(6)**
- (b) StudentGrade(matricNo, courseCode, sName, cTitle, grade) has matricNo → sName and courseCode → cTitle. Identify all determinants and the type of dependency each gives. **(4)**
- (c) Over Sales(saleID, item, amount, staffID, region): write SQL to (i) list sales above 5000, largest first; (ii) total sales per region; (iii) average sale per staff; (iv) sales of the item 'Laptop'; (v) staff with sale over 20000; (vi) regions with no sales recorded. **(7½)**

## QUESTION TWO

- (a) Explain two limitations of the file-based approach. **(4)**
- (b) What is a DBMS, and what are the three levels of data abstraction? **(6½)**
- (c) A hospital EER has WARD (wardNo, name), PATIENT (patientNo, name, multivalued allergy), and an ADMISSION (date, bed) linking each patient to one ward. Transform it into relational schemas. **(7)**

## QUESTION THREE

- (a) Would you store data in a database rather than operating-system files? Justify with two points, and say when a database would not be worth it. **(7½)**
- (b) A university library lends BOOKs to MEMBERs. A member may borrow many books over time; each loan records a date out and due date. A member is either a STUDENT (matricNo) or a STAFF (staffNo). Draw the EER diagram. **(10)**

## QUESTION FOUR

- (a) Describe the relational data model. **(2½)**
- (b) Explain six relational algebra operations with examples. **(6)**
- (c) Over Emp(eNo, name, dNo), Dept(dNo, dName, loc): write relational algebra for (i) names of employees in the 'Research' department; (ii) departments in 'Abuja'; (iii) names and locations of all employees. **(9)**

## QUESTION FIVE

- (a) Which of DDL, DML, or the data model represents real-world information? Explain each briefly. **(6½)**
- (b) A bank has CUSTOMERs who hold ACCOUNTs. An account is either a SAVINGS account (interestRate) or a CURRENT account (overdraftLimit), never both. Draw the crow's-foot EER. **(11)**

## QUESTION SIX

- (a) Explain: relation, degree, cardinality. **(3)**
- (b) Employee(empNo, projNo, eName, pName, hours, deptNo, deptName) has empNo → eName, deptNo; projNo → pName; deptNo → deptName. Draw the FD diagram and normalize to 3NF. **(14½)**

## MOCK EXAMINATION TWO

# Answers

*Full solutions. Mark honestly, and revisit the named module for anything you missed.*

## ANSWER

## QUESTION ONE (COMPULSORY)

- (a)** Generalization: bottom-up, defining a supertype from shared subtypes. Specialization: top-down, defining subtypes of a supertype. Subtype discriminator: a supertype attribute whose value says which subtype a row belongs to. Completeness constraint: whether every supertype instance must belong to a subtype (total) or need not (partial).
- (b)** Determinants: matricNo → sName (partial, on part of the key); courseCode → cTitle (partial); the full key (matricNo, courseCode) → grade (full dependency). Both single-attribute dependencies are partial, so the relation is not in 2NF.
- (c)**

```
(i) SELECT * FROM Sales WHERE amount > 5000 ORDER BY amount DESC;
(ii) SELECT region, SUM(amount) FROM Sales GROUP BY region;
(iii) SELECT staffID, AVG(amount) FROM Sales GROUP BY staffID;
(iv) SELECT * FROM Sales WHERE item = 'Laptop';
(v) SELECT DISTINCT staffID FROM Sales WHERE amount > 20000;
(vi) SELECT region FROM Region
 WHERE region NOT IN (SELECT region FROM Sales); -- assumes a Region table
```

## ANSWER

## QUESTION TWO

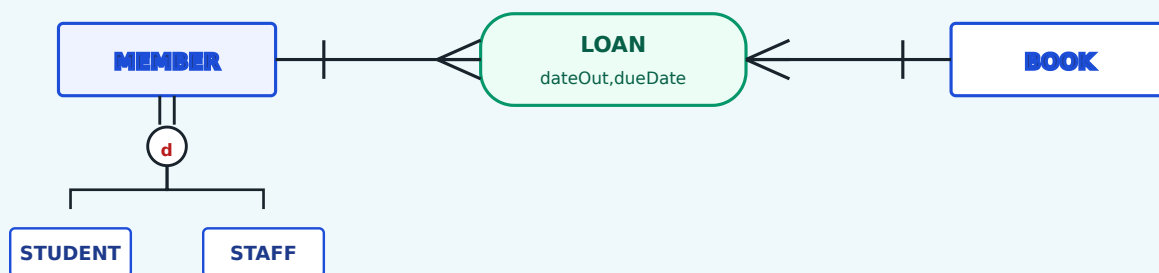
- (a) Data redundancy/inconsistency, and program-data dependence.
- (b) A DBMS is the software that defines, creates, maintains, and controls access to a database. The three levels of abstraction are external (per-user views), conceptual (all tables and relationships), and internal (physical storage).
- (c) The multivalued allergy and the admission each become their own relation:

|                                                                 |                                  |
|-----------------------------------------------------------------|----------------------------------|
| WARD(wardNo, name)                                              | PK wardNo                        |
| PATIENT(patientNo, name)                                        | PK patientNo                     |
| PATIENT_ALLERGY(patientNo, allergy)                             | PK both, FK patientNo -> PATIENT |
| ADMISSION(patientNo, date, wardNo, bed)                         |                                  |
| PK (patientNo, date) FK patientNo -> PATIENT, FK wardNo -> WARD |                                  |

## ANSWER

## QUESTION THREE

- (a) Yes: a database gives one shared, consistent copy with concurrency control, and central integrity/security/independence. Not worth it for a tiny, single-user, fixed-structure task where a DBMS is pure overhead.
- (b) MEMBER is a disjoint supertype over STUDENT and STAFF, and borrows BOOK many-to-many through the associative entity LOAN:



*Mock 2, Q3(b). MEMBER (disjoint over STUDENT/STAFF) borrows BOOK many-to-many through LOAN.*

## ANSWER

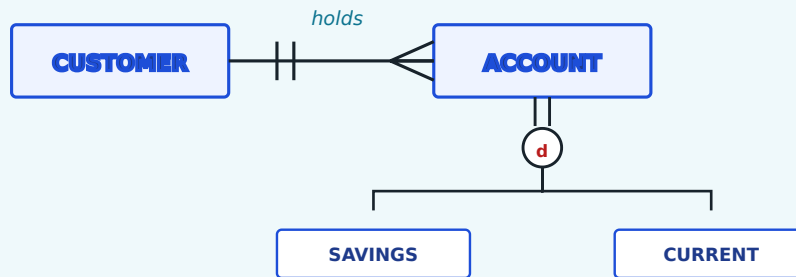
## QUESTION FOUR

- (a) Data as relations (tables), relationships by shared values, queried by relational algebra or SQL.
- (b) Selection ( $\sigma$ ), projection ( $\pi$ ), union ( $\cup$ ), set difference ( $-$ ), Cartesian product ( $\times$ ), join ( $\bowtie$ ), each taking relations and returning a relation.

```
(i) π name (σ dName='Research' (Dept) $\bowtie$ Emp)
(ii) σ loc='Abuja' (Dept)
(iii) π name, loc (Emp $\bowtie$ Dept)
```

**(a)** The **data model** represents real-world information: it is the set of concepts (entities, relationships, constraints) describing what the data means. DDL defines the structure; DML manipulates the data.

**(b)** CUSTOMER holds ACCOUNT one-to-many; ACCOUNT is total and disjoint over SAVINGS and CURRENT:



*Mock 2, Q5(b). CUSTOMER holds ACCOUNT (one-to-many); ACCOUNT is total (double line) and disjoint (d) over SAVINGS and CURRENT.*

**(a)** Relation: a table. Degree: number of attributes (columns). Cardinality: number of tuples (rows).

**(b)** Key (empNo, projNo). empNo → eName, deptNo and projNo → pName are partial; deptNo → deptName is transitive. 2NF lifts the partials; 3NF lifts the transitive:

|                                    |                    |                   |
|------------------------------------|--------------------|-------------------|
| Emp(empNo, eName, deptNo)          | PK empNo           | FK deptNo -> Dept |
| Dept(deptNo, deptName)             | PK deptNo          |                   |
| Proj(projNo, pName)                | PK projNo          |                   |
| WorksOn(empNo, projNo, hours)      | PK (empNo, projNo) |                   |
| FK empNo -> Emp, FK projNo -> Proj |                    |                   |

## MOCK EXAMINATION THREE

# One More, To Be Sure

*The hardest of the three, mixing every skill: a deeper specialization, a Boyce-Codd decomposition with two candidate keys, a subquery, and a division. Answer any four. If you can pass this, the real paper holds no surprises.*

# Question Paper

*DTS224 Data Management I. Time: 2 hours. Attempt any four (4) questions. Each carries 25 marks.*

## QUESTION ONE

- (a) Distinguish disjoint from overlap, and total from partial specialization. **(6)**
- (b) A PUBLICATION (pubID, title) is either a BOOK (isbn) or a JOURNAL (issn), never both, and every publication is one of the two. A BOOK is further either a TEXTBOOK (subject) or a NOVEL (genre). Draw the EER diagram with all constraints. **(19)**

## QUESTION TWO

- (a) State the BCNF condition. **(3)**
- (b) Teaching(student, course, teacher) has (student, course) → teacher and teacher → course. Find all candidate keys, state the highest normal form it satisfies, and decompose it to BCNF. **(22)**

## QUESTION THREE

Over Student(sID, sName, level), Course(cID, cTitle, unit), Enrol(sID, cID, score): write SQL to (i) list students with their course titles; (ii) count students per course; (iii) students who scored above the average score; (iv) courses no student is enrolled in; (v) the highest score in each course. **(25)**

## QUESTION FOUR

- (a) Define, with an example, the division operation of relational algebra. **(5)**
- (b) Over Takes(sID, cID) and Offered(cID, dept): write relational algebra for the students who have taken every course offered by the 'CIS' department. **(12)**
- (c) Write the algebra for the names of all students, given Student(sID, sName). **(8)**

## QUESTION FIVE

A delivery company handles PARCELS for CUSTOMERs. A customer sends many parcels; each parcel is sent by exactly one customer. Each parcel is carried by one COURIER, and a courier carries many parcels. A courier is assigned to one ROUTE; a route has many couriers. Draw the ER diagram with identifiers and cardinalities. **(25)**

## QUESTION SIX

- (a) Explain the relationship between a candidate key and a foreign key, and state the referential integrity rule. **(9)**
- (b) Transform the ER model of Question Five into a set of relations, showing all primary and foreign keys. **(16)**

# Answers

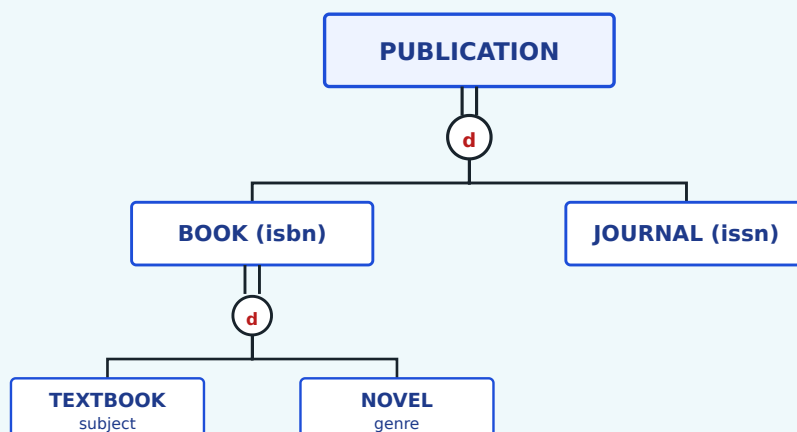
Full solutions to the hardest paper. If these come out clean, you are ready.

## ANSWER

## QUESTION ONE

**(a)** Disjoint: an instance is at most one subtype; overlap: it may be several. Total: every supertype instance must be a subtype; partial: it need not be.

**(b)** PUBLICATION is total and disjoint over BOOK and JOURNAL; BOOK is total and disjoint over TEXTBOOK and NOVEL:



Mock 3, Q1(b). A two-level total, disjoint specialization: PUBLICATION over BOOK/JOURNAL, and BOOK over TEXTBOOK/NOVEL.

## ANSWER

## QUESTION TWO

**(a)** A relation is in BCNF if it is in 3NF and every determinant is a candidate key.

**(b) Candidate keys:** (student, course) determines teacher, so it is a key; and since teacher → course, (student, teacher) determines course and therefore everything, so it too is a key. **Highest normal form:** the dependency teacher → course has a non-key determinant, but course is a prime attribute (part of the key (student, course)), so the relation is in **3NF**. It is **not** in BCNF, because teacher is a determinant that is not a candidate key. **BCNF decomposition:**

|                      |                                           |
|----------------------|-------------------------------------------|
| R1(teacher, course)  | PK teacher                                |
| R2(student, teacher) | PK (student, teacher)    FK teacher -> R1 |

This is the classic case where BCNF costs a dependency: (student, course) → teacher is no longer enforced by one relation. That trade-off is itself an exam point.

## ANSWER

## QUESTION THREE

- (i) `SELECT s.sName, c.cTitle`  
`FROM Student s JOIN Enrol e ON s.sID = e.sID`  
`JOIN Course c ON e.cID = c.cID;`
- (ii) `SELECT cID, COUNT(*) FROM Enrol GROUP BY cID;`
- (iii) `SELECT * FROM Student WHERE sID IN`  
`(SELECT sID FROM Enrol WHERE score > (SELECT AVG(score) FROM Enrol));`
- (iv) `SELECT * FROM Course WHERE cID NOT IN (SELECT cID FROM Enrol);`
- (v) `SELECT cID, MAX(score) FROM Enrol GROUP BY cID;`

## ANSWER

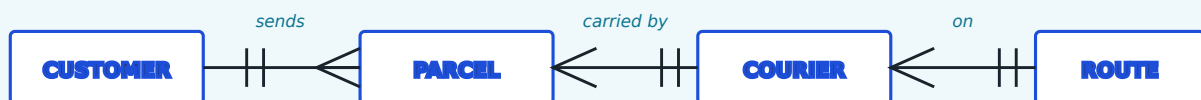
## QUESTION FOUR

**(a)** Division  $R \div S$  returns the tuples in R that are associated with *every* tuple in S.  
 Example: (Enrolment  $\div$  all-courses) gives students enrolled in every course.

- (b) `CIS  $\leftarrow \pi$  cID (  $\sigma$  dept='CIS' (Offered) )`  
`Result  $\leftarrow$  Takes  $\div$  CIS`
- (c)  `$\pi$  sName (Student)`

## ANSWER

## QUESTION FIVE



*Mock 3, Q5. CUSTOMER sends PARCEL (one-to-many); PARCEL carried by one COURIER (many-to-one); COURIER on one ROUTE (many-to-one).*

## ANSWER

## QUESTION SIX

**(a)** A foreign key refers from one relation to the candidate (primary) key of another; the candidate key is the target, the foreign key the pointer. Referential integrity: every foreign-key value must match an existing primary key or be wholly null.

**(b)** Each one-to-many puts the "one" side's key into the "many" side:

|                                                                                  |                                                     |
|----------------------------------------------------------------------------------|-----------------------------------------------------|
| <code>CUSTOMER(custID, name, address)</code>                                     | <code>PK custID</code>                              |
| <code>ROUTE(routeID, description)</code>                                         | <code>PK routeID</code>                             |
| <code>COURIER(courierID, name, routeID)</code>                                   | <code>PK courierID    FK routeID -&gt; ROUTE</code> |
| <code>PARCEL(parcelID, weight, custID, courierID)</code>                         |                                                     |
| <code>PK parcelID    FK custID -&gt; CUSTOMER, FK courierID -&gt; COURIER</code> |                                                     |